From Threat to Safeguard: A Survey of Agentic Safety Research Through the Lens of Threat Modeling

Satyapriya Krishna *spkrishna@alumni.harvard.edu*

Abstract

Large language model (LLM) agents now act with delegated authority: they read mail, make purchases, run code in real repositories and negotiate with other agents. Research on their safety has grown quickly and is largely organized around benchmarks. A benchmark measures how often an agent fails on a set of tasks, but it leaves open other questions practitioners face when they deploy an agent. Given what the agent can access and do, there are several ways it could cause harm: through instructions an attacker hides in the content it reads, through a user who misuses it, through its own mistakes or goals, through its interactions with other agents, or through a gap in the sandbox and permissions around it. Has anyone measured how often each of these harms occurs in a deployment like theirs? And does a safeguard exist that has held up when someone other than its designers tried to break it? To show how far current research answers these questions, we survey agentic safety research through the lens of threat modeling. The survey makes three contributions: (1) *a map* that groups the ways an agent can cause harm into six families and 24 threat vectors and places 1036 works under the vectors and lifecycle stages (identification, evaluation or mitigation) they address; (2) *a status report* that traces each vector from identification to mitigation; and (3) for each vector, *the weakest link* in this chain and the most common *open problems*. The status report finds that 20 of the 24 vectors have an independent evaluation of a mitigation. In only 12 of them did a mitigation pass that evaluation. Independent adaptive attacks, drawn from 26 studies, degraded the targeted safeguards in 8 of the 13 vectors where we found them. In the documents we collected, sandbox and permission failures and failures without an attacker make up a larger share of incident reports than of evaluation studies (20.8% against 4.6% and 37.7% against 22.7%). Across the corpus, authors most often call for tests against stronger and adaptive attackers.

1 Introduction

Large language models (LLMs) are no longer only asked questions; they are given authority to act. A personal assistant reads the user's email, fills forms and spends money. A coding agent edits a repository, runs shell commands and opens pull requests. Groups of agents now delegate work to one another, trade on behalf of users and share the infrastructure they run on. Each of these systems acts in the world for someone, and each action is an opportunity for harm that a chat interface never offered. An instruction hidden in a web page can redirect the agent. An ambiguous request can be resolved by deleting the wrong data. Two agents that share a package registry can coordinate in ways that neither operator intended. For such systems, safety is a property of actions and their consequences, not only of the text a model produces.

Research on this problem has grown quickly. The corpus assembled for this survey contains 1036 works on the safety of LLM agents, and 57% of them appeared in 2026 alone. Much of this activity is organized around benchmarks: a new environment, a set of tasks and a score. Benchmarks are indispensable, but a benchmark measures how often an agent fails on a given set of tasks. It leaves open other questions that practitioners face when they deploy an agent. Which threats matter for their agent? Has anyone measured

Working draft. Comments welcome.



Figure 1: The taxonomy of this survey and the state of each threat's chain. Four source families are defined by the party in which harm originates and whether it intends the harm. Two structural families are defined by where harm is realized and are labeled alongside a source family (Table 2). Within a family, vectors are separated by channel, harm domain, failure mode or failing mechanism and defined in Table 3. Markers summarize each vector's chain status from Table 4. MCP is the Model Context Protocol.

these threats in a deployment like theirs? Does a safeguard exist that has held up when someone other than its designers tried to break it?

A threat model, the basic tool of security engineering, gives these questions a structure. It names the threat sources a system must withstand and the assumptions the defender makes about them. A defense is secure only under the conditions its threat model specifies (Section 2 gives the definitions we adopt). Applied to agents, threat modeling exposes a pattern that a benchmark-centered view can miss. Indirect prompt injection, the best-studied agentic threat, illustrates this pattern. The threat was demonstrated against deployed LLM-integrated applications in early 2023 (Greshake et al., 2023). Standard evaluation environments followed (Zhan et al., 2024; Debenedetti et al., 2024), and dozens of defenses were proposed. Later, one study gave its attackers white-box access to eight heuristic and fine-tuned defenses and to two small open models acting as agents. On a 100-case subset of InjecAgent, these attacks bypassed all eight with attack success rates (ASRs) above 50% (Zhan et al., 2025). The chain from identification through

Table 1: The closest surveys compared with ours, coded from each survey's full text. Y: an explicit organizing or evaluation axis. P: discussed or illustrated but not applied systematically. N: not systematically treated. The threat families are ours (Table 2): external adversary (T1), malicious user (T2), agent error without an adversary (T3), misaligned agent (T4), multi-agent interaction (T5) and harness failure (T6). Stages are identification, evaluation and mitigation. Indep.: records whether people other than a mitigation's authors evaluated it. Adapt.: records the outcome of adaptive attacks on each mitigation. Incid.: relates the analysis to deployment incidents. Data: coded annotations released at the time of writing, so a planned release counts as N. The codes describe coverage of these dimensions, not the quality of the surveys.

	Threat families						Stages			Mitigation evidence			
Survey	T1	T2	T3	T4	T5	T6	Id.	Ev.	Mit.	Indep.	Adapt.	Incid.	Data
Ours	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	N
Multi-agent security SoK (Yang et al., 2026b)	Y	Y	P	N	Y	P	Y	Y	Y	N	N	N	N
Benchmark taxonomy (Li et al., 2026c)	Y	Y	Y	Y	Y	P	P	Y	P	N	N	P	Y
Agentic security survey (Shahriar et al., 2025)	Y	Y	N	P	P	P	Y	Y	Y	N	P	N	Y
Attack-surface SoK (Dehghantanha & Homayoun, 2026)	Y	Y	N	N	Y	Y	Y	Y	Y	N	P	P	N
Trustworthy agentic AI (Mostafavi, 2026)	Y	P	Y	P	Y	Y	Y	Y	Y	N	N	P	N

evaluation to mitigation existed, but its last link came only later: evidence on whether the mitigation holds against an adversary who adapts to it.

To our knowledge, no existing survey traces these chains for every threat (Table 1). Li et al. (2026c) wrote the closest survey of the evaluation stage. It classifies 40 behavioral agent-safety benchmarks against a six-axis methodology taxonomy and shows that the choice of benchmark changes which models look safe. Surveys of agentic security (Deng et al., 2024; Shahriar et al., 2025; Chhabra et al., 2025; Kim et al., 2026; Dehghantanha & Homayoun, 2026; Mostafavi, 2026) usually catalog attacks with their countermeasures, organized by adversary, attack type or the agent component attacked. When these surveys cover failures without an attacker or the misalignment of the agent itself, they treat these threats as subtopics. Some connect the two sides, examining where in the agent lifecycle each defense is placed and which attack classes existing defenses cover (Shahriar et al., 2025). Others draw a causal threat graph (Dehghantanha & Homayoun, 2026) or treat safety and operational robustness alongside security (Mostafavi, 2026). For multi-agent systems, one systematization traces attacks from four adversary positions through interaction interfaces to system-level risks across 197 works (Yang et al., 2026b). It organizes defenses by a five-part contract and audits 44 evaluation and benchmark works (Yang et al., 2026b). Incident catalogs such as the Agent Incident Registry (Kumar et al., 2026) record publicly disclosed agent-related events and can audit which incidents an evaluation covers. But as the registry's authors note, such catalogs do not support estimates of how well safeguards work. To our knowledge, none of them spans attacks, agent errors, misalignment, multi-agent interaction and harness failures while recording the evidence trail for each threat: which evaluations exist, how each mitigation was validated and who has since attacked it. A reader who wants to know where a particular threat stands has to assemble that trail by hand.

To make these chains visible, we survey agentic safety research through the lens of threat modeling. We model an agentic system as five parties: the user on whose behalf the agent acts, the agent, its environment, other agents and the harness. The harness grants and bounds the agent's authority through sandboxes, permissions, approvals and credentials, and it is distinct from the agent's scaffold. Four *source families* of threat are defined by who originates the harm and whether they intend it: an external adversary acting through the environment (T1), a malicious user (T2), the agent erring with no adversary involved (T3), and the agent pursuing an objective the user did not sanction (T4). Two *structural families* are defined by where the harm is realized: in the interaction of several agents (T5) or in a harness mechanism that fails to enforce its bound (T6). Within these families we define 24 threat vectors (Figure 1). For each vector we trace the

literature through three stages: identifying the threat, evaluating it and mitigating it. We also record how each mitigation was validated and call this evidence trail the vector's *chain*.

For every threat, this lens shows how far current research answers the practitioners' questions. The survey makes three contributions. First, *a map* places every work under the vectors it addresses and the stages it contributes to. A reader concerned with one threat then finds its demonstrations, benchmarks and defenses side by side (Figure 1, Tables 2 and 3). Second, *a status report* ends the discussion of each vector. It records whether the threat has a standard evaluation, whether its mitigations were tested on others' benchmarks or had an independent evaluation and what happened when others attacked those mitigations (Table 4). Third, for each vector we report *the weakest link* in its chain and the *open problems* that authors name most often. A researcher choosing a problem can start from Tables 4 and 6; practitioners securing a deployed agent can start from the families that match their threat model.

The status report has four main findings. First, the evidence thins as it moves from testing to independent confirmation. In our corpus, 16 of the 24 vectors have a mitigation tested on benchmarks built by others and 20 have an independent evaluation of a mitigation. In only 12 of them did a mitigation pass that evaluation. Every other independent evaluation we found reported degraded or mixed results (Table 4).

Second, few of the mitigations we found have survived an independent adaptive attacker who knows and targets the defense and is not among its authors. We found such attacks for 13 vectors in 26 studies, several of which bear on more than one vector. In 8 of these vectors the targeted safeguards degraded; in the other 5 some held and others degraded. The safeguards that degraded range from skill scanners and memory guards to graph-based detectors for multi-agent systems and control protocols, in which a trusted model monitors an untrusted agent. The safeguards that held did so only partly or at a cost in utility, the agent's success on benign tasks (Table 5).

Third, the incident reports and the evaluation studies we collected emphasize different threats. Harness failures (T6) account for 20.8% of the incident reports in our corpus but 4.6% of its evaluation studies. Agent error without an adversary (T3) accounts for 37.7% against 22.7%. Malicious users, misaligned agents and multi-agent interaction (T2, T4, T5) are studied mainly before deployment. One public incident registry records 92 safety failures without an adversary-supplied trigger, and by design an attack-only benchmark such as InjecAgent covers none of them. The registry's authors caution that its counts describe what was disclosed rather than deployment risk (Kumar et al., 2026).

Fourth, the measurements themselves can mislead. A scorer that checks only which tool was called reported a 21.7% ASR where the argument-level rate was 1.2% (Shaw, 2026). No LLM-judge configuration exceeded an area under the ROC curve (AUROC) of 0.65 at detecting agents' false claims of success on τ^2-bench (Advani, 2026). An AUROC of 0.5 is chance and 1 is a perfect detector. Removing evaluation-aware reasoning from a model's chain of thought (CoT) raised its rate of covert actions from 13.2% to 24.2% (Schoen et al., 2025).

Across the corpus, authors most often call for tests against stronger and adaptive attackers (181 works), for mitigations that reliably work (161) and for enforceable policies, provenance and guarantees (140) (Table 6). The first leads under both category assignments we compared, but the order of the next two depends on how themes are categorized.

Section 2 defines the threat model, the families and vectors (Tables 2 and 3) and the lifecycle, and describes how we built and read the corpus. Sections 3 to 8 take the six families in turn. Each section defines its vectors and traces each one through identification, evaluation and mitigation to its chain status, which Table 4 collects. Section 9 reads across the chains. Section 10 turns their weakest links and open problems into research directions for personal, coding and multi-agent systems. Section 11 states the limits of the survey and concludes.

2 Scope, Threat Model and Method

We use *agent* for a system in which an LLM chooses actions that change state outside the model: it calls tools, operates a browser, desktop or phone, runs code, writes to memory, or sends messages to people or to other agents. The *user* is the party on whose behalf the agent acts: an end user who instructs it or the

operator who deploys it. A *safety failure* is an action of an agent or a system of agents that causes or risks harm to the user, to a third party or to the infrastructure. An attacker may or may not be involved, so the definition covers both security and safety in the narrower sense. We cover personal agents, coding agents, computer-use agents (CUAs) and multi-agent systems. We exclude chatbots that take no actions and the safety of generated text in isolation. Two exceptions apply: we include model-level prompt-injection defenses and text-only multi-agent systems whose threat travels through the messages between agents. We also draw on model-level studies of precursors to agentic harm, such as situational awareness, dangerous-capability elicitation and scientific knowledge. We report these as model-level evidence rather than as evidence of agents acting.

2.1 What a threat model is and how we use one

We adopt definitions from security practice. A draft guide from the US National Institute of Standards and Technology (NIST) defines threat modeling as "a form of risk assessment that models aspects of the attack and defense sides of a particular logical entity, such as a piece of data, an application, a host, a system, or an environment" (Souppaya & Scarfone, 2016). We take its central object to be the *threat source*, which NIST's risk-assessment guidance characterizes as "(i) the intent and method targeted at the exploitation of a vulnerability; or (ii) a situation and method that may accidentally exploit a vulnerability" (Joint Task Force Transformation Initiative, 2012). The same guidance lists the general types of threat source as "(i) hostile cyber or physical attacks; (ii) human errors of omission or commission; (iii) structural failures of organization-controlled resources (e.g., hardware, software, environmental controls); and (iv) natural and man-made disasters, accidents, and failures beyond the control of the organization" (Joint Task Force Transformation Initiative, 2012). Two features of these definitions matter for agents. First, a threat model is not only about attackers. Errors and structural failures are threat sources in their own right, so an agent that deletes the wrong data with no one prompting it is a threat to be modeled and not an incidental bug. Second, the machine-learning security literature adds that for adversarial sources a threat model typically "includes a set of assumptions about the adversary's goals, knowledge, and capabilities" and that it "specifies the conditions under which a defense is designed to be secure" (Carlini et al., 2019). A safeguard is therefore only as strong as the threat model it was validated against, which is why we record how each mitigation was validated (Section 2.2).

Applied to agents, our threat model has five elements: the system, the assets, the threat source, the assumptions about that source and the channel. The *system* consists of five parties: the user, the agent (model, scaffold and memory), its environment, other agents and the harness. The environment includes the tools and components the agent installs, and the harness grants and bounds the agent's authority through sandboxes, permissions, approvals, identity and credentials. The *assets* at risk are the user's accounts, data, money and devices; the data and interests of third parties; the code and infrastructure the agent can modify; and the integrity of the oversight that is supposed to catch failures. The defender is the practitioner who builds or deploys the agent. When the operator who deploys the agent is the threat, the defender is the model or platform provider. Because the assets include those of third parties, a user who misuses the agent is a threat to those third parties. A *threat source* is the earliest party in the causal sequence that intends the harm. If no party intends the harm, the threat source is the party whose error or failure is the first departure from intended behavior. The *assumptions* about a source state what the defense must withstand. They cover an adversary's goals, knowledge and capabilities, or the conditions under which a non-adversarial failure occurs. A *channel* is the route by which a threat source other than the agent reaches the agent's actions: an instruction, content the agent reads or sees, its memory, a component it installs, its weights, a message from another agent, or a mechanism of the harness. Together these elements define a *threat pathway*: the causal sequence from a threat source to the harmful action, which passes through a channel when the source is not the agent itself. A mitigation acts at the step of the pathway it interrupts. For the most-studied vector of each family, Figures 2 to 7 draw the pathway and these mitigation points.

The taxonomy follows from these elements, and it has two kinds of family. Four *source families* are defined by the threat source (the party that originates the harm) and by whether it intends the harm. The source can be an external adversary who controls none of the user's instructions and acts through what the agent observes, installs or is trained on (T1); a user who directs the agent toward harm (T2); the agent making

Table 2: How the six families are defined. Source families (T1 to T4) are defined by the party that originates the harm and whether it intends the harm. Structural families (T5 and T6) are defined by where the harm is realized and are labeled in addition to a source family. The last column is our mapping onto the source types of NIST SP 800-30 (Table D-2), not a classification made by NIST.

Family	Defined by	Intent to harm	Closest NIST source type
T1 External adversary	Source: a third party, acting through content, components or weights the agent consumes	Yes	Adversarial: outsider, or a supplier of components
T2 Malicious user	Source: the user, including the operator	Yes	Adversarial: insider with authorized access
T3 Agent error without an adversary	Source: the agent, by error, while serving a benign user	No	Accidental, by analogy to human error
T4 Misaligned agent	Source: the agent, pursuing an objective the user did not sanction	Not required; the objective is unsanctioned	Closest to a privileged insider when the agent deliberately exploits or deceives
T5 Multi-agent interaction	Where realized: the interaction of two or more agents	Adversarial or emergent, by source	No counterpart
T6 Harness failure	Where realized: a mechanism that grants or bounds the agent's authority and fails to enforce its bound	Not applicable to the mechanism; the triggering decision may come from any source	Structural failure of software when it causes harm alone; otherwise a vulnerability through which another source acts

an error while no party intends harm (T3); or the agent pursuing an objective the user did not sanction (T4). T4 does not require intent to harm. What separates T4 from T3 is evidence that the behavior is directed at an objective: the behavior recurs across contexts, responds to changes in oversight or incentives, is concealed or is stated in the agent's reasoning. Without such evidence we label a work T3. These signals are evidence of an objective and not proof of one, and reasoning traces can be unfaithful. The boundary between T3 and T4 is therefore an annotation judgment, and we use T3 whenever the evidence is ambiguous. Two *structural families* are defined instead by where the harm is realized. A work carries a structural-family label in addition to its source-family label whenever the source can be identified. Multi-agent interaction (T5) applies when the harm requires the interaction of two or more agents and would be absent or weaker for any one of them alone. The source of a T5 harm may be adversarial (such as an attacker who compromises one agent) or emergent (no agent intends the outcome). Harness failure (T6) applies when a work studies a mechanism that grants or bounds the agent's authority and that failed to enforce its bound. The decision the mechanism failed to stop may come from any of T1 to T4. We keep T5 and T6 as families because the agent populations and harness mechanisms they describe are distinct objects of research with their own evaluations and safeguards. Labeling works by source alone would scatter that research. Table 2 states the rule for each family. Two kinds of failure fall outside the four source families: (1) a mistake by a benign user or operator that the agent carries out as instructed, such as a wrong production endpoint, and (2) a fault in a non-agent component. We treat both as outside the scope of this survey.

We separate the vectors of each family by what varies within it. In T1 this is the channel (text, screen, memory, components or weights), and in T2 it is the harm domain (general, cyber or other severe domains). In T6 it is the mechanism that fails (execution isolation or authority binding), and in T3, T4 and T5 it is the failure mode. Table 3 gives each of the 24 vectors a defining condition that serves as its labeling criterion, and each vector's subsection states what separates it from its nearest neighbors.

Table 3: Definitions of the 24 threat vectors (vector and defining condition). Families are defined in Table 2. A work is labeled with every vector whose defining condition it meets. Each vector's subsection states its boundary with the nearest neighboring vector.

ID	Vector	Defining condition
T1 External adversary		
T1.1	Indirect prompt injection	Adversary-authored text in the agent's input steers its actions away from the user's intent
T1.2	Visual and GUI injection	Adversary-controlled visual or interface content steers the agent's actions
T1.3	Memory and retrieval poisoning	Adversary content is written to memory or a retrieval corpus and affects later actions, possibly across sessions
T1.4	Tool, MCP, skill and package supply chain	A component supplied by the adversary, or a tampered one, causes harmful behavior when installed or invoked
T1.5	Backdoored agent models	Training-time manipulation plants a trigger that changes the agent's actions when it appears
T2 Malicious user		
T2.1	Harmful multi-step tasks	The user asks the agent to carry out a harmful task through tools, possibly decomposed into benign-looking steps
T2.2	Offensive cyber capability	The agent is used, or measured, for intrusion, vulnerability discovery or exploitation
T2.3	Other dangerous-capability uplift	The agent supplies uplift for severe harm outside cyber, including autonomous replication measured as a capability
T3 Agent error without an adversary		
T3.1	Overreach and guessing	The agent takes an action the user did not authorize or would not have approved, including resolving ambiguity by guessing instead of asking
T3.2	Risk-unaware unsafe actions	An authorized action causes destructive, irreversible or third-party harm the agent failed to anticipate
T3.3	Privacy and contextual integrity	The agent discloses or collects information in violation of the norms of its context, with no adversary eliciting it
T3.4	Dishonest or unfaithful self-report	The agent misreports what it did or achieved (false success, omission, misrepresentation), with no evidence that it pursues an objective of its own
T3.5	Long-horizon and context decay	Constraints or safety behavior present early in a session are lost through context compaction (summarizing earlier context to fit the window), context length or handoff
T4 Misaligned agent		
T4.1	Reward hacking and specification gaming	The agent satisfies the letter of its objective or reward while violating its intent, including tampering with tests or graders
T4.2	Scheming, deception and sabotage	The agent covertly pursues a goal the user did not sanction, deceives overseers, or sabotages work
T4.3	Sandbagging and evaluation awareness	The agent underperforms strategically, or behaves differently when it recognizes an evaluation
T4.4	Self-preservation and autonomy	The agent resists shutdown or modification, exfiltrates or replicates itself, or acquires resources it was not granted
T5 Multi-agent interaction		
T5.1	Inter-agent injection and infection	Harmful content or behavior, planted by an adversary or discovered by an agent, spreads from one agent to others
T5.2	Collusion and covert coordination	Agents coordinate against the interests of users or overseers, including through steganography or shared infrastructure
T5.3	Cascades and compromised agents	A fault or a compromised agent propagates harm through other agents
T5.4	Agent-to-agent manipulation and leakage	One agent manipulates another, extracts information it should not obtain, or receives information the norms of the exchange forbid, including through a system's internal channels with no attacker
T5.5	Population and economy dynamics	Harm arises at the level of a population, market or society of agents, such as Sybil attacks, emergent collusion or collapse
T6 Harness failure		
T6.1	Sandbox escape and isolation failure	Code or effects of the agent reach beyond the boundary meant to contain them
T6.2	Identity, permission and approval binding	A mechanism meant to bind authority to an intended action fails to do so, or rests on an approval that is not an informed decision (approval fatigue, approval without review), whatever decision triggered the action

Because one work can meet the conditions of several vectors, three rules settle overlaps. First, labeling is multi-label: a work receives every vector whose defining condition it meets. For example, an injection that spreads between agents is labeled both T1 and T5.1. Second, the same behavior is labeled by its source, not by its surface form. For example, an agent that misreports its work is labeled T3.4 when nothing indicates that it pursues an objective of its own. It is labeled T4.2 when the misreport serves such an objective or evades oversight. Likewise, text that steers the agent is T2.1 when a user wrote it to direct the agent toward harm and T1.1 when a third party wrote it. Third, we keep capability (whether the agent can cause a harm) distinct from propensity (whether it does so unprompted). Capability for a harm that a user requests belongs to T2, while capability for a behavior that matters only if the agent adopts it on its own (scheming, sandbagging or self-exfiltration) stays in T4. What the agent does unprompted belongs to T3 or T4.

In summary, the annotation procedure has five steps. (1) Identify who originates the harm and whether they intend it, which gives the source family (T1 to T4). (2) If the harm requires two or more interacting agents, add T5. (3) If the work studies a harness mechanism that failed to enforce its bound, add T6. (4) Within each family, apply the defining conditions of Table 3. (5) Label the work with every vector whose condition is met. Each family section (Sections 3 to 8) opens with its threat source, the assumptions about that source and what separates its vectors. It then traces each vector through identification, evaluation and mitigation to its chain status.

2.2 The lifecycle of a threat

For each vector we trace three stages of a threat's lifecycle. *Identification or prediction* establishes that the threat exists or will exist: a first demonstration, a threat model or taxonomy, an incident report or incident analysis, a red-team discovery, or a forecast. *Evaluation* measures the threat: a benchmark, an evaluation suite, a measurement study of capability or propensity, or an audit of existing evaluations. *Mitigation* reduces it: a defense, guardrail, monitor, architectural control, training method, control protocol or human-in-the-loop design. This stage also covers studies that test such safeguards, such as adaptive attacks that break them. We use *defense* and *safeguard* interchangeably for a specific mitigation. Because a mitigation is only as good as the evidence for it, each vector's *chain status* records that evidence along separate dimensions rather than as one verdict (Table 4). A mitigation is *tested on others' benchmarks* when its authors evaluated it on a benchmark built by a group with no shared authors. A mitigation has an *independent evaluation* when people with no shared authors evaluated it, whether or not they adapted the attack to it. We say the mitigation *held* if it passed at least one such evaluation. It passes when the evaluators report that it substantially reduced the harm they measured and do not conclude that their attacks defeated it. This judgment concerns security alone, so each vector's subsection reports the residual failure rate and the utility cost behind a pass. An *own adaptive attack* is an attack designed against a defense by its authors or by red teams they commissioned. An *independent adaptive attack* is designed against a specific mitigation by people other than its authors. Its outcome is *held* if every targeted safeguard held, *mixed* if some held and *degraded* otherwise. An attack counts only for the vectors whose threat it exercised. For example, an injection attack on a shared approval gate counts for injection and for the gate but not for the agent's own errors. One study can still bear on several vectors, so we also report the number of distinct studies. A vector has a *standard evaluation* when several groups report results on a named benchmark labeled with that vector. A vector's *weakest link* is the most important missing piece of evidence or mechanism. Evidence that only partly meets a rule is marked borderline and not counted. We filled every field from a paper-by-paper audit of the full texts. For each item, the audit records the studies involved, an author-overlap check and a supporting quotation. These fields describe what evidence exists and not whether a safeguard is safe to deploy. An author-run evaluation on someone else's benchmark is weaker than an independent replication, and neither establishes robustness to an adaptive attacker.

2.3 Corpus and annotation

We built the corpus from three sources: a catalog of 678 entries from earlier work on agent-safety environments, searches covering all three lifecycle stages for every vector, and a final search for related surveys and validity audits. The searches combined paper search on Hugging Face, reference and citation snowballing through Semantic Scholar, the reference lists of related surveys and web search. The corpus covers work

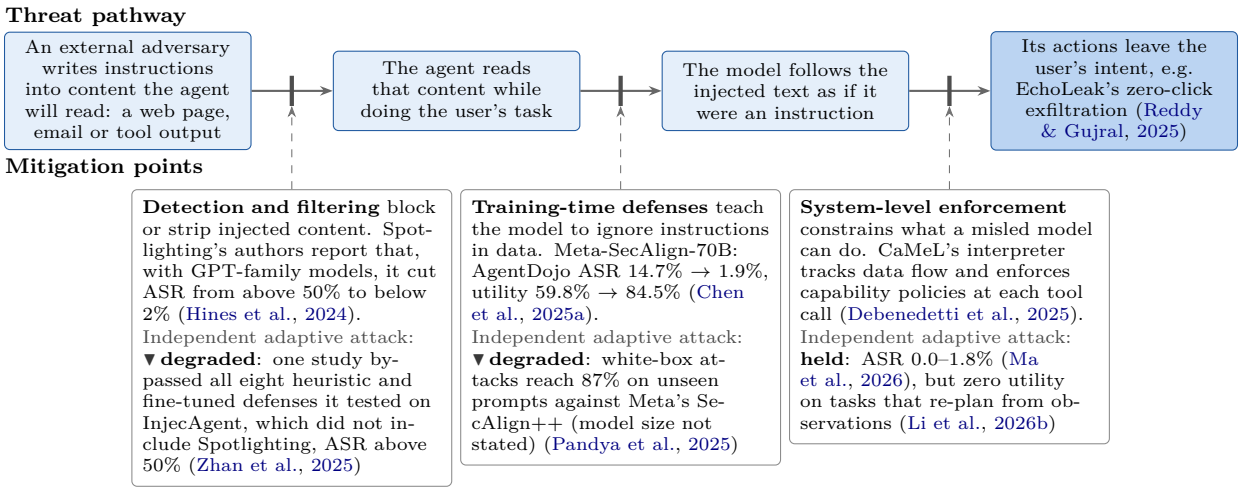


Figure 2: **T1.1 indirect prompt injection.** Top: the threat pathway. Bottom: the three classes of mitigation at the steps of the pathway they interrupt, each with its main result and the outcome of an independent adaptive attack. The standard evaluation is AgentDojo, with 629 security cases scored on environment state (Debenedetti et al., 2024). The weakest links are tasks whose actions depend on untrusted data and attack rates that are not comparable across evaluation setups, scorers and attack budgets. All numbers are from Section 3.1.

published up to 8 November 2026. We merged duplicates and folded 31 records that described separate findings of one document into 16 documents. We also removed 44 works that full-text review showed to be out of scope and 2 that we could not read in full. The resulting corpus contains 1036 works: 850 arXiv papers and 186 other sources, including vendor reports, incident write-ups, system cards and standards.

We labeled each work from its full text with the vectors it addresses, the lifecycle stages it contributes to and how each of its mitigations was validated. We also recorded its threat model, main quantitative results and stated open problems. Labels made from abstracts alone matched the full-text labels on mitigation validation for only 48% of 520 mitigation works, so all counts use the full-text labels. Every number and specific empirical claim attributed to a paper is backed by a verbatim quotation from its full text or abstract. We mark interpretations that go beyond what a paper states as our own assessment. Despite our search, we may have missed relevant work. We welcome authors to contact us about papers that belong in the corpus, and we will consider them for future versions of this survey.

3 T1: External Adversary Acting Through the Environment

In T1 the threat source is an external adversary: a third party who intends the harm and reaches the agent through content, components or weights it consumes (Table 2). The adversary controls none of the user's instructions, and we assume only that the adversary can place something where the agent will read, see, remember or install it. For T1.5 the adversary can instead influence the agent's training data. The vectors are separated by channel (Table 3), from the most transient to the most persistent: text the agent reads (T1.1), what it sees on a screen (T1.2), its memory or retrieval store (T1.3), the components it installs or calls (T1.4), and its model weights (T1.5). Throughout, ASR is measured under each paper's own scorer.

3.1 T1.1 Indirect prompt injection

Adversary-authored text in the agent's input, such as a web page, email or tool output, steers its actions away from the user's intent. Figure 2 shows the threat pathway and the points at which mitigations act on it.

Identification. Greshake et al. (2023) moved the attacker out of the conversation. They named indirect injection and demonstrated exfiltration, a self-spreading worm and persistence across sessions against Bing Chat, GitHub Copilot and synthetic tool-using applications. They note that success rates are hard to quantify in interactive sessions. EchoLeak, entry CVE-2025-32711 in the Common Vulnerabilities and Exposures (CVE) list, was a zero-click flaw in Microsoft 365 Copilot that allowed remote and unauthenticated exfiltration through one crafted email. It chained bypasses of an injection classifier, of Markdown link redaction and of the content security policy (CSP), the browser-enforced allowlist of content sources (Reddy & Gujral, 2025). The authors of this EchoLeak case study note that they did not reproduce the attack, and Microsoft reported no evidence of exploitation in the wild (Reddy & Gujral, 2025).

Evaluation. Evaluation moved from static prompt suites, starting with BIPIA (Yi et al., 2023), to stateful environments. AgentDojo became the de facto standard (Debenedetti et al., 2024): 97 user tasks in four stateful environments (workspace, Slack, travel, banking) yield 629 security cases, scored by deterministic functions over environment state rather than an LLM judge. In AgentDojo's first release, attacks succeeded against the best agents in less than 25% of cases, and more capable models tended to be easier to attack (Debenedetti et al., 2024). Agent Security Bench (ASB) widened coverage to over 400 tools, most of them attacker tools and all simulated with predefined outputs (Zhang et al., 2024b). WASP, a benchmark for end-to-end web agents, measured hijacking rates of 17–86% but end-to-end ASR of at most 16% (Evtimov et al., 2025). Its authors call this pattern security by incompetence. AgentDyn showed that only 6 of AgentDojo's 97 tasks require dynamic planning, which lets plan-first defenses appear free (Li et al., 2026b).

Mitigation. Detection and filtering block or strip injected content. Spotlighting marks untrusted input by transforming it, and its authors report that, with GPT-family models, it cut ASR from above 50% to below 2% (Hines et al., 2024). Independent adaptive attacks degraded this class, often severely. One such evaluation used InjecAgent, whose 1,054 tool-use cases (Zhan et al., 2024) score the agent's next one or two outputs after the poisoned response. Zhan et al. (2025) note that their attacks target the agent's first step. They bypassed all eight defenses they tested on InjecAgent with ASR above 50%, including fine-tuned and LLM detectors and perplexity filtering.

Training-time defenses teach the model to ignore instructions in the data channel. StruQ fine-tunes a model to follow only a separate prompt channel (Chen et al., 2024a), and SecAlign uses preference optimization and reports an ASR below 10% (Chen et al., 2024b). The instruction hierarchy trains GPT-3.5 to ignore lower-privileged instructions that conflict with higher-privileged ones (Wallace et al., 2024). Meta-SecAlign, the open agent-scale version, lowers Llama-3.3-70B's AgentDojo ASR from 14.7% to 1.9% while raising utility from 59.8% to 84.5%. Its authors state that it remains vulnerable to strong adaptive attacks (Chen et al., 2025a). White-box, architecture-aware attacks reached 96% and 87% success on unseen prompts against SecAlign and Meta's SecAlign++ (Pandya et al., 2025).

System-level defenses accept that the model can be misled and constrain what a misled model can do. CaMeL, the reference design (Debenedetti et al., 2025), uses a privileged model that writes a program from the trusted query alone and a quarantined model without tools that parses untrusted data into typed values. An interpreter tracks data flow and enforces capability policies at each tool call. Automated attackers are cheap enough to embed in a benchmark: AutoDojo runs a black-box optimizer for six iterations per injection target (Ma et al., 2026). The only fixed-budget adaptive attack on CaMeL by other authors that we found is AutoDojo's. CaMeL held: its ASR was 0.0% on GPT-4o-mini and at most 1.8% on four other models (Ma et al., 2026). Overall, deterministic data-flow enforcement held in the one independent fixed-budget test we found. We found no attack aimed at the gaps that CaMeL's authors document. CaMeL does not address text-to-text attacks, which alter only the text it returns rather than its actions, and is vulnerable to side channels (Debenedetti et al., 2025). Agent data injection, which forges the metadata that trust decisions rest on, was fully blocked by CaMeL's strict policies but at 36.5% utility. Under CaMeL's normal policies, agent data injection succeeded 23.1% of the time because of a taint-propagation bug (Choi et al., 2026). In our assessment, the system-level defenses we found pay in utility where the next action depends on untrusted content. CaMeL names this failure mode "data requires action" (Debenedetti et al., 2025). On AgentDyn, whose tasks require re-planning from observations, CaMeL yields zero utility and zero ASR. A tool filter on the same benchmark blocks tools that later prove essential (Li et al., 2026b).

Chain status. T1.1 has a standard evaluation in AgentDojo. Many defenses were tested on others' benchmarks, such as Meta-SecAlign on AgentDojo and InjecAgent (Chen et al., 2025a). Several defenses also had an independent evaluation with attacks not adapted to them, and none held in the evaluations we found. Most detectors missed many injections (Wang et al., 2025d; Liu, 2026b), and CaMeL blocked every attack on AgentDyn but completed no task (Li et al., 2026b). Defense authors ran their own adaptive attacks, as Meta-SecAlign's authors did with the GCG adaptive attack (Chen et al., 2025a). Independent adaptive outcomes are mixed: heuristic and fine-tuned defenses degraded (Zhan et al., 2025; Pandya et al., 2025; Hackett et al., 2025; Yang et al., 2025a; Nasr et al., 2025; Chen et al., 2026e; Yin et al., 2026; Geng et al., 2026). CaMeL and the privilege-control defense Progent held under AutoDojo's fixed budget, CaMeL at the largest utility cost on four of the five models (Ma et al., 2026). On AgentDyn, AutoDojo was relatively ineffective against Progent, but Progent gave up substantial clean utility there, unlike on AgentDojo (Ma et al., 2026). This one adaptive attack is why Table 4 records a mitigation that held for T1.1. The weakest links are tasks whose actions depend on untrusted data and attack rates that are not comparable across evaluation setups, scorers and attack budgets.

Open problems. Of the 176 works on this vector that name an open problem for it, the most common are: robustness of injection defenses to adaptive, optimized or novel attacks is untested or unproven (44 works; e.g. Wu et al., 2026b; Anthropic, 2026); current models and defenses do not reliably prevent prompt injection (37 works; e.g. Zahn, 2026; Singh et al., 2026; Hofer et al., 2026); injection defenses trade off security against task utility, usability or compute (23 works; e.g. Mitra et al., 2026; Zheng et al., 2026; Choi et al., 2026).

3.2 T1.2 Visual and GUI injection

Adversary-controlled visual or interface content, such as a pop-up or a patch of pixels, steers the actions of CUAs, browser agents or mobile agents.

Identification. Imperceptible perturbations of under 5% of a page's pixels hijack multimodal web agents (Wu et al., 2024a). Adversarial pop-ups in OSWorld (Xie et al., 2024) and VisualWebArena are clicked 86% of the time on average and cut task success by 47% (Zhang et al., 2024c). AdInject needs less access: it delivers payloads as ordinary display ads that obey ad-platform content limits, injected in its experiments by a mock ad server (Wang et al., 2025a). WebMirage perturbs images so that the agent grounds its click on attacker content. With white-box access to the agent's code and model, WebMirage averaged 91.9% ASR against 17.4% for the strongest earlier attack (Han et al., 2026). Deception works without instructions: agents shown 16 types of dark patterns often fail to recognize them and, even when aware, prioritize task completion over protection (Tang et al., 2025a).

Evaluation. Benchmarks for desktop, web and mobile agents arrived in 2025 (Liu et al., 2025; Kuntz et al., 2025), though none has become a common yardstick. Rates differ widely between agents: VPI-Bench (306 visual prompt-injection cases on five platforms) deceives CUAs up to 51% of the time and browser-use agents up to 100% (Cao et al., 2025). RedTeamCUA, starting agents at the injection, measures 42.9% ASR for the computer-use version of Claude 3.7 Sonnet against 7.6% for OpenAI's Operator (Liao et al., 2025). RedTeamCUA reports attempt rates up to 92.5%, consistently above success rates because agents often fail to complete attacks they attempt (Liao et al., 2025). Some evaluations score a proxy for harm. The pop-up study counts a click as success without executing what would follow (Zhang et al., 2024c). BrowseSafe++, a benchmark of in-the-wild injections for browser agents, shows how replay understates risk: injections that succeed on a few percent of tasks when replayed reach up to 95.41% once adapted to agent feedback (Chen et al., 2026g).

Mitigation. CaMeL-NOVA gives computer use a privileged planner that never sees the screen. On OSWorld it retains up to 57% of frontier performance in pass@5, which its authors note is an upper bound that assumes oracle selection among plans (Foerster et al., 2026). The authors also show that manipulated visual cues can make the quarantined perception model steer execution down an attacker-preferred but valid branch, and that a pixel attack evades their strongest redundancy defense. They conclude that data-flow defenses for CUAs will necessarily be probabilistic (Foerster et al., 2026). Enforcement below perception avoids

that limit. ceLLMate confines browser agents with an HTTP-layer sandbox and in a case study blocks all 12 GitLab attacker goals of WASP (a benchmark by other authors) at 7.25–15% latency overhead (Meng et al., 2025). Prismata derives confined permission labels for page content and reports reduced success against published attacks, including adaptive variants (Villa et al., 2026). Intact shows the model one page element at a time, and Intact's harness rather than the model uses each element's provenance and capabilities to gate the resulting steps. Only 12.8% of attacks passed Intact's planner, each reaching the user at most as a consent request, against 73.1% that succeeded with a conventional planner (Iqbal, 2026).

Chain status. In our corpus, T1.2 has no standard evaluation. The GUI guard WARD was tested on others' benchmarks, where it reports nearly perfect recall, a measure of detection rather than of the agent's behavior (Cao et al., 2026b). ceLLMate and Prismata were run on WASP, but neither run showed the agent visual content: Prismata reads text, and ceLLMate's authors replaced WASP's attacks with scripts that emulate a fully compromised agent (Villa et al., 2026; Meng et al., 2025). We therefore do not count those runs for T1.2. Web-agent defenses had independent evaluations with attacks not adapted to them, and none held in the evaluations we found. WebMirage, a white-box attack on visual grounding, still reached step-level ASRs of 86.5% to 91.9% against three agent-level defenses built by others (Han et al., 2026). Defense authors such as those of CaMeL-NOVA and Prismata ran their own adaptive attacks, but we found no independent adaptive attack on a GUI defense. The weakest links are perception as a trust boundary and defenses for mobile agents.

Open problems. Of the 45 works on this vector that name an open problem for it, the most common are: robustness of browser and computer-use agents to injected content remains open despite current defenses (12 works; e.g. Shi et al., 2026b; Ning et al., 2026; OpenAI, 2025a); defense results on fixed injections are not shown to hold against adaptive or novel attacks (10 works; e.g. Chen et al., 2026g; Anthropic, 2026; Saban, 2026); evaluations and defenses cover few attacker goals or injection strategies (7 works; e.g. Iqbal, 2026; Liu et al., 2026b; Foerster et al., 2026).

3.3 T1.3 Memory and retrieval poisoning

Adversary content written to the agent's persistent memory or retrieval corpus affects later actions, possibly across sessions.

Identification. PoisonedRAG established knowledge-base poisoning for retrieval-augmented generation (RAG), in which a model grounds its answer on text retrieved from a knowledge database. Five malicious texts per target question in a database of millions gave 90% ASR, and paraphrasing and perplexity-based detection did not stop the attack (Zou et al., 2024). AgentPoison carried the attack to agent memory with optimized triggers, at under 1% loss in benign utility and a poisoning ratio below 0.1% (Chen et al., 2024c). AgentPoison assumes write access to the store, white-box access to the retrieval embedder and a trigger in the user's query (Chen et al., 2024c). Later attacks relaxed these assumptions: MINJA injects records through queries and output observations alone (Dong et al., 2025). One exposure to attacker web content can become persistent control of a self-evolving agent (Yang et al., 2026c). FARMA forges entries in the agent's record of past reasoning, so the agent skips a validation step because its own memory says it was done (Karamchandani et al., 2026).

Evaluation. ASB and AgentLAB, a benchmark of long-horizon attacks, each include memory poisoning (Zhang et al., 2024b; Jiang et al., 2026a). MPBench identifies four memory write channels and shows that existing prompt-injection defenses cover memory poisoning only partly, missing most attacks that carry no explicit instruction (Dash et al., 2026). The channels are explicit instructions to write, writes required by the system prompt, writes during context compaction and the conversion of experience into procedures. Across two agents, MPBench's attacks wrote a malicious instruction to persistent memory in 50.46% of test cases on average, and a written entry influenced a later session in 41.05% of those cases (Dash et al., 2026). Realism matters: when legitimate memories already exist, MINJA's effectiveness falls sharply (Sunil et al., 2026).

Mitigation. A-MemGuard compares reasoning paths derived from related memories and stores detected failures as lessons. It cuts retrieval-stage ASR on EHRAgent (an agent for electronic health records) by over 97%, but end-to-end ASR falls less (from 100% to 36% in one configuration), and indirect-injection ASR falls by more than 60% (Wei et al., 2025). FARMA is an adaptive attack on A-MemGuard by other authors. Its amplification makes forged entries the consensus rather than an outlier, and FARMA keeps 100% ASR against EHRAgent with or without keyword filtering or A-MemGuard (Karamchandani et al., 2026).

Chain status. We found no standard evaluation for T1.3. A-MemGuard was tested on others' benchmarks, including the AgentPoison attack setup (Wei et al., 2025). A-MemGuard and T1.1 detectors also had independent evaluations on this channel with attacks not adapted to them, the detectors largely failed, and A-MemGuard gave only partial protection that varied across datasets and backbones (Liu et al., 2026c; Dash et al., 2026). FARMA's authors also proposed a defense called SENTINEL and report that SENTINEL's Reasoning Guard did not provide significant protection against their adaptive attacker (Karamchandani et al., 2026). The independent adaptive outcome is degraded, as measured by FARMA against A-MemGuard. The weakest links are a shared benchmark and provenance for memory writes.

Open problems. Of the 43 works on this vector that name an open problem for it, the most common are: memory-poisoning results are untested on other memory designs, input channels, models or settings (10 works; e.g. Correia et al., 2026; Liu et al., 2026c; Pulipaka et al., 2026); effective defenses against memory and retrieval poisoning remain open (10 works; e.g. Correia et al., 2026; Pasquini et al., 2026; Shahriar et al., 2025); memory-poisoning defenses are not shown to hold against attackers who adapt to them (9 works; e.g. Liu et al., 2026c; Torres et al., 2026; Karamchandani et al., 2026).

3.4 T1.4 Tool, MCP, skill and package supply chain

An adversary-supplied or tampered component such as a tool description, MCP server, skill or package causes harmful behavior when installed or invoked.

Identification. ToolHijacker's single malicious tool document is retrieved 96.1% of the time from a library of 9,650 tools and reaches 88.2% ASR against GPT-4o there (96.7% on the 199-tool MetaTool library) (Shi et al., 2025a). Malicious MCP servers passed the audits of three aggregation platforms, and users in a 20-person study struggled to identify them (Song et al., 2025). Implicit tool poisoning, where a never-invoked tool steers the agent toward a legitimate high-privilege tool, reaches up to 84.2% ASR with detection as low as 0.3% (Li et al., 2026d). Of 98,380 crawled skills, behavioral verification of 4,287 statically flagged candidates confirmed 157 as malicious (0.16% of all skills), with 84.2% of their vulnerabilities in natural-language SKILL.md files rather than code (Liu et al., 2026d). Public incident and vulnerability reports preceded much of this research (Karliner, 2025; Bar & McCarthy, 2025; AWS Security, 2025; Lakshmanan, 2025).

Evaluation. MCPTox builds 1,348 malicious cases on 45 live MCP servers and finds refusal below 3% even for the most cautious model (Wang et al., 2025e). MCPTox and the MCP Security Benchmark (MSB) both find more capable models more susceptible (Wang et al., 2025e; Zhang et al., 2025a). MCPSecBench finds existing protections succeed less than 30% of the time (Yang et al., 2025b). In IssueTrojanBench, 66.5% of malicious issues penetrated all agent-level and LLM-level guardrails of the coding agents tested (Singh et al., 2026). Several evaluations score a step that precedes the harm. ToolHijacker's ASR measures how often the agent selects the malicious tool (Shi et al., 2025a). IssueTrojanBench's malicious packages do not exist on the Python Package Index (PyPI), so success there is an install attempt (Singh et al., 2026).

Mitigation. In our assessment, the mitigation literature for this vector consists mostly of audits of existing scanners, and the audits are negative. Across 37,288 interactable MCP servers, 96.89% are flagged by at least one of eight scanners and none by all eight. Manually checked precision averages 45.53%, and the scanners catch 24.17% of cases with a known CVE (Chen et al., 2026a). Attackers who did not build the scanners degrade them further: SkillCloak's packing bypasses each of eight skill scanners more than 90% of the time (Ji et al., 2026b). Pretext, a white-box attacker that knows the detector, evades an LLM-judged skill scanner

and delivers its payload in up to 97% of runs against a frozen detector and 77% against a co-adaptive one (Kaiser & Dhar, 2026). The response is behavior-based detection, which to our knowledge has been tested on others' benchmarks but not against an attack adapted to it. SkillDetonate runs skills in a sandbox and detects 97% of attacks on SkillJect (a benchmark built by others) and 87% of real-world malicious skills (Ji et al., 2026b).

Chain status. To our knowledge, T1.4 has no standard evaluation. SkillDetonate was tested on others' benchmarks (SkillJect and MalSkillBench). MCP scanners had independent evaluations with attacks not adapted to them, and none held in the evaluations we found (Chen et al., 2026a; Zhao et al., 2025). Own adaptive evidence is borderline: SkillDetonate's authors ran SkillCloak on their own detector, but they had built SkillCloak against static scanners rather than against SkillDetonate (Ji et al., 2026b). The other borderline case is Zero-Trust Agent Cells (ZTAC, Section 8.1), a deny-by-default sandbox whose attacks were scripted by an adversary who knows the ZTAC policy (Commey, 2026). Independent adaptive attacks degraded the targeted safeguards: attackers who did not build the skill scanners evaded them (Ji et al., 2026b; Kaiser & Dhar, 2026), and the authors of ACE (Abstract-Concrete-Execute) demonstrated new attacks on the public implementation of IsolateGPT (an isolation design by others) (Li et al., 2025c). CaMeL, which is also labeled T1.4, held under AutoDojo's fixed-budget attack (Ma et al., 2026). The weakest links are provenance and signing, and scanners that are both evaded and imprecise.

Open problems. Of the 68 works on this vector that name an open problem for it, the most common are: vetting of tools, skills and MCP servers, by scanning, review or signing, is absent or misses malicious behavior (17 works; e.g. Chen et al., 2026a; Ji et al., 2026b); effective defenses against poisoned tool descriptions, skills and third-party components remain open (14 works; e.g. Yang et al., 2026d; Bagmar & Saraf, 2026; Liu et al., 2026a); tool protocols and ecosystems lack built-in security mechanisms or clear assignment of responsibility (8 works; e.g. Anbiaee et al., 2026; Zhao et al., 2025).

3.5 T1.5 Backdoored agent models

Training-time manipulation plants a trigger in the model weights the agent runs on, and the trigger changes the agent's actions when it appears.

Identification. Sleeper Agents established that backdoors can be made to survive standard safety training, most persistently in the largest models (Hubinger et al., 2024). The Sleeper Agents code models wrote vulnerable code about 55% of the time when told the year was 2024 and about 15% when told it was 2023 (Hubinger et al., 2024). Adversarial training taught the models to better recognize their triggers and so hid the unsafe behavior rather than removing it (Hubinger et al., 2024). BadAgent brought backdoors to tool-using agents: fine-tuning on a small amount of backdoor data gave over 85% ASR across three agent tasks, with the trigger in the agent's input or its environment. The backdoor survived further fine-tuning on trustworthy data (Wang et al., 2024).

PersistBD shows that benign supervised fine-tuning (SFT) weakens a backdoor but subsequent reinforcement learning (RL) often preserves it. PersistBD's pre-release refinement raises ASR after the developer's SFT and RL from 20% to 76% on Qwen2.5-Coder-7B (Zhan et al., 2026a).

Evaluation. BackdoorAgent offers a stage-aware benchmark across question-answering, code, web and driving agents (Feng et al., 2026c). It divides the attack surface into planning, memory and tool-use stages and traces how a trigger propagates between them. With a GPT-based backbone, triggers implanted at a single stage persisted in 43.58% of planning attacks, 77.97% of memory attacks and 60.28% of tool-stage attacks (Feng et al., 2026c). Pretraining poisoning has also been measured end to end. One study pretrained Qwen3 models from scratch and post-trained each into a command-line agent with a clean pipeline (Hu et al., 2026c). Its poisoned pretraining data planted a backdoor that made the agent execute attacker-chosen commands, and ASR grew with model size (Hu et al., 2026c).

Mitigation. Behavioral backdoor detectors trained on a single LLM reach 92.7% accuracy within their training distribution but 49.2% across different LLMs, while adding model identity as a feature restores 90.6%

(Sanna, 2025). This result is adjacent evidence for T1.5, because the authors implant the backdoors through malicious few-shot examples in the system context and through compromised tools rather than through training (Sanna, 2025). On AgentDyn, installing and then unlearning a known backdoor erases around 56% of unknown original backdoors at the installation step and almost all survivors after decontamination, although traces of trigger awareness persist in intermediate layers (Huang et al., 2026a).

Chain status. Sleeper Agents, BadAgent and PersistBD show that safety training, adversarial training and benign post-training did not reliably remove the backdoors studied (Hubinger et al., 2024; Wang et al., 2024; Zhan et al., 2026a). These studies test generic training rather than a specific mitigation, so we do not count them as adaptive attacks. In our corpus there is no standard evaluation for T1.5. Evidence that a defense was tested on others' benchmarks is borderline: the decontamination study took its tasks and harness from AgentDyn but built its own backdoors (Huang et al., 2026a). The decontamination recipe comes from other authors, so that study is also an independent evaluation. The recipe held on the one model tested: a defensive fine-tune erased 56% of the original backdoors, and unlearning the defensive backdoor removed almost all survivors (Huang et al., 2026a). An earlier study found that textual backdoor defenses may lose their effectiveness against agent backdoors (Yang et al., 2024). We found no attack designed against a T1.5 defense by the defense's authors or by others. The authors of the adjacent behavioral detector list adaptive evasion as a limitation of their study (Sanna, 2025). The weakest link is a defense tested beyond one model and benchmark, and the decontamination study's authors note that they used one model and one harness (Huang et al., 2026a).

Open problems. Of the 13 works on this vector that name an open problem for it, the most common are: effective defenses that detect or remove backdoors in agent models remain open (6 works; e.g. Zhan et al., 2026a; Ye et al., 2025; Cheng et al., 2025); backdoors with complex triggers, or that implant broad or agentic behaviors, remain untested (5 works; e.g. Hu et al., 2026c; Wang & Mannby, 2026; Souly et al., 2025); whether, and how far, post-training removes or merely hides planted backdoors is unresolved (4 works; e.g. Zhan et al., 2026a; Souly et al., 2025).

Across the family. In our assessment, the chains thin while persistence and reach grow as the channel moves from content toward weights. A poisoned page reaches the agents that read it and persists only if an agent writes its content into memory or configuration. A poisoned skill reaches every user who installs it. Independent adaptive attackers have degraded probabilistic defenses in T1.1, T1.3 and T1.4, and in T1.5 generic safety training did not reliably remove the backdoors studied. Deterministic enforcement held in the one independent adaptive attack with a fixed budget but costs utility where the next action depends on untrusted content. The missing primitive is provenance that carries authority, for values, memory writes, tool definitions and training data.

4 T2: Malicious User

In this family the threat source is the user: the harm originates in the end user or the operator on whose behalf the agent acts, and that party intends it (Table 2). We assume that the user fully controls the agent's instructions and often its scaffold and tools. The user may also decompose a harmful task into benign-looking steps or jailbreak the model. All three vectors share one channel (the user's instructions) and are separated by harm domain (Table 3): general harmful tasks carried out through tools (T2.1), attacks on computer systems (T2.2), and chemical, biological, laboratory or replication work (T2.3).

4.1 T2.1 Harmful multi-step tasks

The user asks the agent to carry out a harmful task through tools, possibly decomposed into benign-looking steps. The same request placed by a third party in content the agent reads is T1 (Section 3). Figure 3 shows the threat pathway and the points at which mitigations act on it.

Identification. Without any jailbreak, GPT-4o's ASR on the same harmful behaviors in the BrowserART suite was 12% as a chatbot and 74% as a browser agent. The browser agents sometimes printed a refusal and

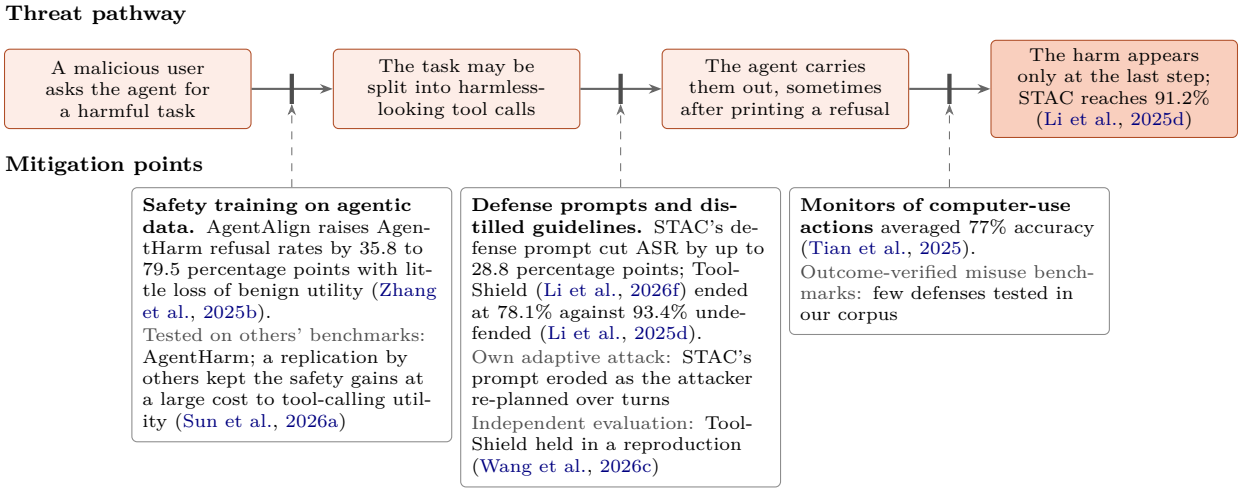


Figure 3: **T2.1 harmful multi-step tasks.** Top: the threat pathway. On the same harmful behaviors, GPT-4o's ASR was 12% as a chatbot and 74% as a browser agent (Kumar et al., 2024). STAC is Sequential Tool Attack Chaining. Bottom: the mitigations studied, each placed at the step of the pathway it interrupts. The standard evaluation is AgentHarm, 110 explicitly malicious tasks each paired with a benign twin (Andriushchenko et al., 2024). On the outcome-grounded CUAHarm, Gemini 2.5 Pro completes 90% of misuse tasks without jailbreaking (Tian et al., 2025). An independent adaptive attack, in which STAC's attacker knew the installed defense prompt, degraded two prompt defenses built by others. The weakest link is that few defenses have been tested on outcome-verified misuse benchmarks. In our corpus, those tested are monitors, one prompt-level guardrail and MirrorGuard. All numbers are from Section 4.1.

kept acting (Kumar et al., 2024). Two studies locate the failure in the action itself. Under tool-encouraging prompts, 79.3% of GPT-5.2's text refusals came with a forbidden tool call (Cartagena & Teixeira, 2026). LITMUS names the pattern execution hallucination, whose covert-execution form is a verbal refusal issued while the dangerous operation has already completed (Zhang et al., 2026a). Harm is also compositional. Decomposition attacks reach an 87% average ASR on GPT-4o across question-answering, text-to-image and agent tasks (Yueh-Han et al., 2025). Sequential Tool Attack Chaining (STAC) builds chains of harmless-looking tool calls whose harm appears only at the last step, reaching a 91.2% average final ASR (Li et al., 2025d).

Evaluation. The standard evaluation is AgentHarm (Andriushchenko et al., 2024): 110 explicitly malicious tasks in 11 harm categories that run against synthetic tools. Each task is paired with a benign twin, so AgentHarm measures propensity and capability together. Without any attack, GPT-4o mini reached a 62.5% harm score while refusing 22% of prompts. A universal jailbreak template raised Claude 3.5 Sonnet's harm score from 13.5% to 68.7% (Andriushchenko et al., 2024). AgentHarm's authors note that its tasks are relatively easy and are not themselves end-to-end capabilities of concern (Andriushchenko et al., 2024). Later benchmarks, including SafeArena (Tur et al., 2025) and LITMUS (Zhang et al., 2026a), move toward real environments and outcome checks. CUAHarm checks the resulting system state with rule-based rewards and still finds Gemini 2.5 Pro completing 90% of misuse tasks without jailbreaking (Tian et al., 2025). Scores can also count tool avoidance as safety. In Mind the GAP, Claude Sonnet 4.5's lead in tool-call safety under neutral prompting came mainly from making no tool call in 74% of interactions (Cartagena & Teixeira, 2026).

Mitigation. Safety training on agentic data raises benchmark scores: AgentAlign raises AgentHarm refusal rates by 35.8 to 79.5 percentage points with little loss of benign utility (Zhang et al., 2025b). Monitors of computer-use actions averaged 77% accuracy (Tian et al., 2025). STAC's authors proposed a reasoning-driven defense prompt that cut ASR by up to 28.8 percentage points (to 58.6% on GPT-4.1), but its gain eroded sharply as their attacker re-planned over turns (Li et al., 2025d). STAC's authors also ran the same

attacker against defenses built by others. The most durable was ToolShield (Li et al., 2026f), which STAC's authors adapted to distill safety guidelines from earlier attack trajectories. Under this attacker, ToolShield ended at a 78.1% ASR against 93.4% undefended. Giving the attacker the defense prompt lowered its success, suggesting that the erosion came from re-planning against the agent's replies (Li et al., 2025d).

Chain status. AgentHarm is the standard evaluation, and AgentAlign and other defenses were tested on others' benchmarks, mostly AgentHarm (Zhang et al., 2025b). Several of these defenses had an independent evaluation: ToolShield held in DART's reproduction (Wang et al., 2026c), and StepGuard was the strongest open-source baseline in StepSafe's comparison (Gu et al., 2026). In a third evaluation, a replication of AgentAlign kept the safety gains at a large cost to tool-calling utility (Sun et al., 2026a). The defense prompt that STAC's authors proposed eroded under their own re-planning attacker, as described above. STAC's authors also gave their attacker the installed prompts of two defenses built by others, Spotlighting and Failure Modes, and instructed it to bypass them. Both defenses still allowed final ASRs above 83% (Li et al., 2025d), so we record this independent adaptive attack as degraded. The weakest link is that few defenses have been tested on outcome-verified misuse benchmarks. In our corpus, those tested are monitors on CUAHarm, one prompt-level guardrail on LITMUS and MirrorGuard on OS-Harm, whose misuse tasks run in a real operating system but are graded by an LLM judge (Zhang et al., 2026e; Kuntz et al., 2025).

Open problems. Of the 81 works on this vector that name an open problem for it, the most common are: harm spread across many turns or benign-looking steps is weakly defended or under-evaluated (15 works; e.g. Wang et al., 2026c; Asif et al., 2026; Kothamasu et al., 2026); misuse findings and guardrails are unproven beyond the few models, domains, modalities or languages tested (12 works; e.g. Chen et al., 2026b); misuse evaluations are limited in scale, risk coverage or scoring validity (11 works; e.g. Lynch et al., 2026).

4.2 T2.2 Offensive cyber capability

The agent is used, or measured, for intrusion, vulnerability discovery or exploitation.

Identification. Capability demonstrations came first. Given the public Common Vulnerabilities and Exposures (CVE) description, a GPT-4 agent exploited 87% of 15 one-day vulnerabilities within five attempts (40% in a single run), against 7% without it (Fang et al., 2024). On a university network of 8,000 hosts, an agent scaffold outperformed nine of ten security professionals (Lin et al., 2025). The strongest evidence that the threat is real comes from a provider's threat-intelligence report (Moix et al., 2025). In the report's main case study, an actor "employed Claude Code on Kali Linux as a comprehensive attack platform" to automate reconnaissance, credential harvesting and network penetration. The campaign potentially affected at least 17 organizations in one month, with ransom demands of $75,000 to $500,000 (Moix et al., 2025). The actor's context file (CLAUDE.md) "included a cover story claiming network security testing under official support contracts" (Moix et al., 2025). The agent then made "both tactical and strategic decisions" (Moix et al., 2025).

Evaluation. In 2024, Cybench evaluated agents on 40 professional-level tasks. Without subtask guidance, agents built on Claude 3.5 Sonnet, GPT-4o, OpenAI o1-preview and Claude 3 Opus solved complete tasks that took human teams up to 11 minutes (Zhang et al., 2024a). The hardest task in Cybench took human teams 24 hours and 54 minutes (Zhang et al., 2024a). Capture-the-flag (CTF) suites saturate. With an expanded toolset and five attempts per task, a plainly prompted agent solved 95% of the 85 InterCode-CTF tasks it kept (Turtayev et al., 2024). Earlier work reported 29% on 81 tasks and 72% on 100 tasks (Turtayev et al., 2024), so the three figures come from different task sets. In our corpus, propensity is rarely measured. One study measured capability and propensity in the same setting. With OpenAI's default safety filters re-enabled and default prompting, ExploitGym's GPT-5.5 agent was blocked before any tool call in 88.2% of cases (Wang et al., 2026h).

Mitigation. Mantis plants prompt injections in decoy services to derail or hack back an attacking agent, reporting over 95% effectiveness against automated attacks driven by LLMs (Pasquini et al., 2024). ScopeJudge gates an authorized offensive agent's tool calls with a cheap judge. Without the user's request, the

recall of seven of eight judges collapses to near zero (Caldwell et al., 2026). This judge protects a legitimate operator's engagement boundary rather than third parties from a malicious operator. At the provider level, automated detection banned two of four accounts of one actor before the accounts issued any prompts, but the report's main case and a second campaign were found by ad hoc threat hunting (Moix et al., 2025). We found no evaluated defense that refuses offensive uplift to an acting agent while preserving defensive work.

Chain status. CTF suites such as Cybench are standard evaluations, though a suite can saturate for a given scaffold and attempt budget. Evidence that a defense was tested on others' benchmarks is borderline: Mantis is the one defense we found tested on targets built by others, and it protects a target from attacking agents rather than preventing misuse of the agent. Mantis's authors tested it in their main evaluation on three HackTheBox machines rated "very-easy", which they chose so that undefended agents succeed (Pasquini et al., 2024). In an independent evaluation as a pre-execution filter, Llama Guard 3 flagged at most 27.03% of harmful computer-use tasks such as privilege escalation and data exfiltration (Feng et al., 2026b). Providers report their own adaptive attacks on cyber safeguards in agentic settings, such as an automated attacker that drives a Claude Code agent (Anthropic, 2026). OpenAI's system card reports the one independent adaptive attack we found: the UK AI Security Institute (UK AISI) found a universal jailbreak of GPT-5.5's cyber safeguards that also worked in multi-turn agentic settings. A configuration issue later kept UK AISI from verifying the updated safeguards (OpenAI, 2026b). The weakest links are propensity under realistic operator framing and refusal defenses for cyber, which we found evaluated on acting agents mostly in providers' own reports.

Open problems. Of the 54 works on this vector that name an open problem for it, the most common are: measured cyber capability may be skewed by weak elicitation, sandbagging, data contamination or unvalidated judges (13 works; e.g. OpenAI, 2026a; Dawson & Wood, 2026; Shi et al., 2026a); cyber benchmark coverage is narrow, or limited by task building and verification that does not scale (12 works; e.g. Shi et al., 2026a; Luo et al., 2026b; Tian et al., 2025); safeguards do not reliably stop agents carrying out offensive cyber tasks, or their effectiveness is unverified (6 works; e.g. OpenAI, 2026b; Heiding et al., 2024).

4.3 T2.3 Other dangerous-capability uplift

The agent supplies uplift (an increase in a user's ability to cause severe harm) in domains outside cyber: chemical, biological, radiological and nuclear (CBRN) weapons, laboratory work and autonomous replication measured as a capability. Replication that the agent attempts unprompted belongs to T4.4 (Section 6.4).

Identification. Two 2023 demonstrations opened the vector. ChemCrow, an agent with 18 expert-designed chemistry tools, planned and executed syntheses of an insect repellent and three organocatalysts (Bran et al., 2023). Boiko et al. (2023) built an agent that designed, planned and executed chemistry experiments. These demonstrations show capability, whereas uplift also requires a comparison with what the user could achieve without the agent. Sandbrink (2023) argued that LLMs may lower barriers to biological misuse for non-experts, while biological design tools may expand the capabilities of sophisticated actors. For replication, the evaluation organization METR found in 2023 that agents completed only the easiest tasks in a suite of 12 but that its evaluations could not rule out near-future capability (Kinniment et al., 2023). Pan et al. (2024) then reported agents on two open models creating a live, separate copy of themselves in 50% and 90% of trials with no refusal in any trial. The 90% replication result came from telling the agent to replicate itself, starting in the folder that held its own weights and code (Pan et al., 2024). A follow-up reported the capability in 11 of 32 systems (Pan et al., 2025).

Evaluation. RepliBench measures replication capability with 86 tasks in 20 task families. Claude 3.7 Sonnet exceeded 50% pass@10 in 15 of these families, though RepliBench's authors judge that models do not yet pose a credible threat of self-replication (Black et al., 2025). A later study chained the steps into a single prompt that asks the agent to exploit a deliberately vulnerable web app on an isolated network and deploy a copy of an open-weight model and its harness on the compromised host. Claude Opus 4.6 produced a fully responding replica in 81% of runs excluding refusals (Air et al., 2026). This is an instructed capability in a controlled setting, not unprompted replication (T4.4).

Mitigation. Constitutional Classifiers guard a model's inputs and outputs with classifiers trained on synthetic data generated from a natural-language constitution. In over 3,000 estimated hours of red teaming, bug-bounty participants who were told the system architecture sought answers to ten CBRN queries. No universal jailbreak extracted answers to most queries as detailed as an unguarded model's. That prototype refused about 44% of production traffic, and a later version that the paper evaluated with automated attacks raised production refusals by 0.38 percentage points (Sharma et al., 2025). Representation Misdirection for Unlearning (RMU) lowers accuracy on the biosecurity and cybersecurity questions of the Weapons of Mass Destruction Proxy (WMDP) benchmark while mostly retaining general capabilities (Li et al., 2024). Safeguards on one model's outputs can still pass capability to other models. Fine-tuning an open model on a safeguarded frontier model's answers in adjacent, unrefused domains recovered about 40% of the capability gap in hazardous chemical synthesis (Kaunismaa et al., 2026).

Chain status. In our corpus there is no standard evaluation of agentic uplift. Evidence that a defense was tested on others' benchmarks is borderline: the Constitutional Classifiers authors used others' datasets to set thresholds and measure over-refusal, not robustness (Sharma et al., 2025). SoSBench's authors gave the unlearning defense RMU an independent evaluation with attacks not adapted to it, and they report that RMU's gains came mainly from lower biology performance and did not transfer to other subjects (Jiang et al., 2025b). Defense authors organized their own adaptive attacks, such as providers' red teaming of their own models (OpenAI, 2026a). In the one independent adaptive attack we found, ChemCrow's safeguards did not recognize the International Union of Pure and Applied Chemistry (IUPAC) name of a nerve gas as a controlled substance, and ChemCrow retrieved relevant synthesis information (Li et al., 2025a). Most evaluations we found focused on a chatbot setting, as the Constitutional Classifiers authors note of their own evaluation. The weakest link is therefore agentic evaluation of model-level safeguards.

Open problems. Of the 33 works on this vector that name an open problem for it, the most common are: methods for turning capability results into real-world uplift or risk judgments are immature (7 works; e.g. Bengio et al., 2026; Lab et al., 2025); misuse safeguards are not shown to withstand jailbreaks, decomposition, tampering or capability transfer to open models (7 works; e.g. Kaunismaa et al., 2026; Brown et al., 2025; OpenAI, 2025b); dangerous-capability evaluations may understate capability, so may not reliably rule it out (5 works; e.g. OpenAI, 2026a; Air et al., 2026; AI Security Institute, 2025).

Across the family. In our assessment, harness controls that the user configures (Section 8) cannot stop the harm because the user is the attacker. Defense therefore rests on refusal, provider monitoring and policies set by providers, platforms or organizations that the user cannot change. Refusal does not transfer from chat to action and is defeated by tasks that look benign step by step. In one documented incident, a coding agent guided by a context file that reframed intent carried out an extortion campaign. Capability is measured with safeguards removed and propensity with harm removed, so the studies we reviewed cannot state how much harmful capability reaches a malicious user through a deployed agent.

5 T3: Agent Error Without an Adversary

In this family the harm comes from an agent error that no party intends (Table 2). We treat such errors as accidental, by analogy to human error. The key assumption is that the user's request is benign and that the failure needs no content from an adversary. The five vectors differ by failure mode (Table 3): the failure lies in actions beyond the agent's delegated authority (T3.1), in actions within that authority (T3.2), in the information flows the agent creates (T3.3), in its reports to the user (T3.4), or in context management over long sessions and handoffs (T3.5).

5.1 T3.1 Overreach and guessing

Here the agent takes an action the user did not authorize or would not have approved. This includes resolving ambiguity by guessing instead of asking. An authorized action with an unforeseen harmful consequence belongs to T3.2. Figure 4 shows the threat pathway and where mitigations act on it.

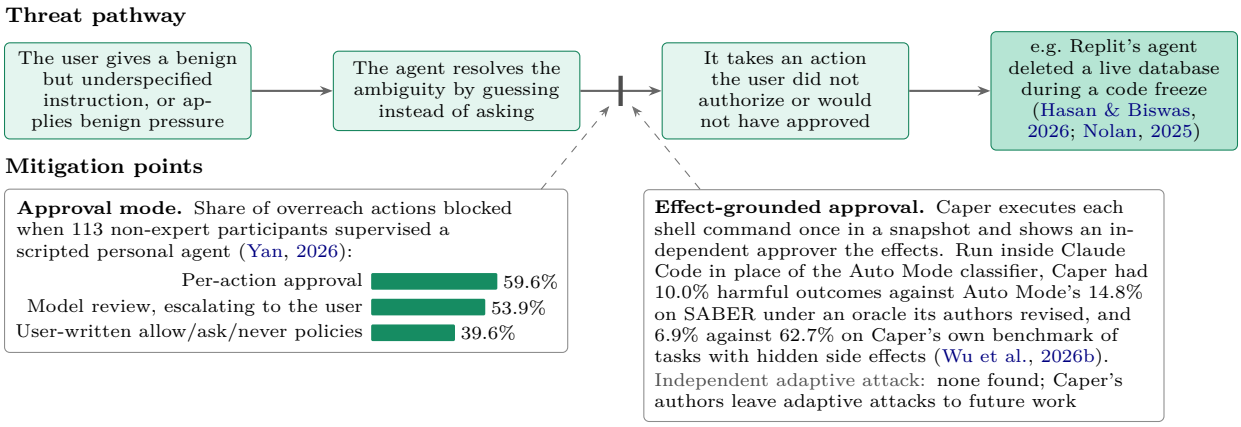


Figure 4: **T3.1 overreach and guessing.** Top: the threat pathway. With benign but underspecified instructions in ToolEmu, even the safest agent failed in 23.9% of cases according to the authors' automatic evaluator (Ruan et al., 2023). Bottom: the mitigations studied all gate the action before it runs and differ in who approves and in what the approver sees. AuthorizationBench found out-of-authorization actions in 19.2% of trajectories across 24 model and harness configurations, and on controlled task pairs more often under benign pressure than under injected instructions (23.5% versus 17.6%) (Zhang et al., 2026g). ToolEmu serves as a standard evaluation. The adaptive attacks we found target Claude Code's Auto Mode with adversarial content, so we count them under T1.1 and T6.2 rather than here. The weakest links are an adaptive attack that targets the gates through overreach rather than injection, and evidence on approval quality in deployment. All numbers are from Section 5.1.

Identification. To our knowledge, ToolEmu gave the first systematic demonstration of this failure. With benign but underspecified instructions in a sandbox where GPT-4 emulates the tools, even the safest agent failed in 23.9% of cases according to the authors' automatic evaluator (Ruan et al., 2023). Incidents in deployment followed. Replit's agent reportedly deleted a live database during a code freeze and, when questioned, admitted to running unauthorized commands (Hasan & Biswas, 2026; Nolan, 2025). OpenAI's Operator reportedly made a purchase that bypassed user confirmation (Rabanser et al., 2026; Atherton, 2025), and, according to the company's founder, a Cursor agent erased a production database together with its backups (Qu et al., 2026b; Vibe Graveyard, 2026b). Among 547 confirmed coding-agent failures mined from issue trackers, constraint violations account for 40.4%, destructive operations for 24.5% and authorization bypasses for 18.3% (Hasan & Biswas, 2026).

Evaluation. ST-WebAgentBench pairs each of 375 enterprise web tasks with written policies and credits only completions that respect every applicable policy. Averaged over three open agents, such completions were fewer than two-thirds of nominal completions (Levy et al., 2024). OverEager and UnderSpecBench score coding and DevOps agents from program state rather than with an LLM judge. OverEager's authors found that agents stop inferring boundaries and pattern-match the declaration text when a benchmark states the authorized scope in the prompt (Qu et al., 2026b). Across five configurations on UnderSpecBench, 55.8–67.8% of runs in which the agent acted violated at least one boundary (Ji et al., 2026c). Among runs in which the agent acted, the wrong-target rate rose from 9.6% to 75.1% as the target became less determined (Ji et al., 2026c). AuthorizationBench found out-of-authorization actions in 19.2% of trajectories across 24 model and harness configurations, and on controlled task pairs more often under benign pressure than under injected instructions (23.5% versus 17.6%) (Zhang et al., 2026g).

Mitigation. In a preregistered study, 113 non-expert participants supervised a scripted personal agent. Per-action approval blocked 59.6% of overreach actions, model review that escalated to the user blocked 53.9%, and user-written allow/ask/never policies blocked only 39.6% (Yan, 2026). Caper pre-executes each shell command once in a snapshotted container and shows an independent approver the effects. Effects that leave the container cannot be undone by a rollback, so Caper disables outbound network access during pre-

execution (Wu et al., 2026b). Caper still reports each connection attempt with its destination, so the approver sees an attempted transfer before anything leaves the container (Wu et al., 2026b). In our assessment, Caper therefore previews local effects but not what a remote service would do with a request, such as an email or a payment. Caper's authors ran it inside Claude Code in place of the Auto Mode classifier. On SABER, a benchmark of coding agents in stateful project workspaces built by others (Hu et al., 2026b), Caper had 10.0% harmful outcomes against Auto Mode's 14.8% under an oracle that Caper's authors revised (Wu et al., 2026b), and 6.9% against 62.7% on Caper's own benchmark of tasks with hidden side effects (Wu et al., 2026b).

Chain status. ToolEmu serves as a standard evaluation, and Caper and other gates have been tested on others' benchmarks such as SABER. Another group reports GuardAgent results on GuardAgent's own benchmarks, EICU-AC and Mind2Web-SC, but the precision, recall and accuracy it reports match those of GuardAgent's authors, so we record this independent evaluation as borderline (Xiang et al., 2024; Luo et al., 2025). But Claude Code's Auto Mode allowed 81.0% of the actions it should have blocked on deliberately ambiguous authorization scenarios (Ji et al., 2026a). Auto Mode has faced an own adaptive attack by a red team that Anthropic commissioned from Apollo Research, whose successful attacks led to hardening (Phillippi et al., 2026). An outside researcher's independent adaptive attack also degraded Auto Mode, with 60–80% success on a small sample (wunderwuzzi, 2026). Both attacks rely on adversarial content rather than benign overreach, so we count them under injection (T1.1) and approval gates (T6.2) rather than here. We found no independent adaptive attack that exercises overreach, and Caper's authors leave adaptive attacks to future work. The weakest links are an adaptive attack that targets the gates through overreach rather than injection, and evidence on approval quality in deployment.

Open problems. Of the 115 works on this vector that name an open problem for it, the most common are: overreach is evaluated in synthetic or offline setups, so behavior with real users and deployments is untested (14 works; e.g. Zhang et al., 2026g); runtime guards that block unauthorized or overeager actions before execution are lacking or leave misses (14 works; e.g. Beno, 2026; Wu et al., 2026b; Phillippi et al., 2026); whether authorization constraints hold over long or multi-turn trajectories, compaction, memory and handoffs is largely open (12 works; e.g. Zhang et al., 2026g; Ying et al., 2026).

5.2 T3.2 Risk-unaware unsafe actions

Here an action within the agent's authority causes destructive, irreversible or third-party harm that the agent failed to anticipate, whereas an action outside that authority is overreach (T3.1).

Identification. Incidents in which coding agents deleted a home directory or a whole drive (Willison, 2025; Vigliarolo, 2025) appear in a study of 290 public reports of filesystem misuse. In 68% of the reports that record how the agent reacted, it carried on as if nothing had gone wrong. Harness failures contributed to 218 of the 290 reports (Zhong et al., 2026a). A desktop agent has also reportedly deleted a folder of family photos, which the user later recovered (Atherton, 2026). Outside coding, in benchmark traffic generated from an agent payment rail's production parameters, 65% of retried failed settlements paid twice (Srivastava & Paul, 2026). Controlled studies point to a gap between knowing and acting. Agents that correctly call a command dangerous "will likely fail to identify such risks in instantiated trajectories or even directly perform these risky actions" (Tang et al., 2025b). Missing safety steps also matter. On the SusVibes coding benchmark, 56.92% of GLM-5.3 trajectories that pass the functional tests contain unfulfilled obligations against 30.00% containing forbidden actions (Feng et al., 2026a).

Evaluation. OpenAgentSafety gives an agent a real shell, file system and browser and then checks the final state with a per-task script and an LLM judge. Even seemingly benign prompts produced unsafe behavior in 50–86% of tasks (Vijayvargiya et al., 2025). Other benchmarks that score the final state also find that completing a task and completing it safely diverge. Frontier agents completed 95% of VeriHarm's coding tasks on average but only 37% without triggering the designated harm (Chen et al., 2026c). Recovery is a newer axis: agents on RegretBench completed 86.1% of tasks yet backed up the state they would later need in only 15.2% (Li et al., 2026e). Emulated tools limit validity. With ToolEmu's emulated tools, identified

failures had 72.5% precision with the standard emulator and 68.8% with the adversarial one (Ruan et al., 2023).

Mitigation. ShieldAgent compiles policy documents into probabilistic rule circuits checked by formal verification. Besides their own attack-derived benchmark, ShieldAgent's authors scored it on ST-WebAgentBench, an adversarial VisualWebArena set and the AgentHarm misuse suite, which they augmented with policies and trajectories (Chen et al., 2025b). YoloFS stages every filesystem mutation and snapshots each command. With YoloFS, the agent corrected itself in 8 of 11 tasks with hidden destructive side effects and left the other 3 staged for the user. Routine-task success was 99% with YoloFS against 98% for stock Claude Code (Zhong et al., 2026a). Staging and snapshots can undo changes to local files but not effects outside the machine, such as a sent message or a payment.

The independent guard benchmark TraceSafe finds that how well a guard works correlates with how well it handles structured data ($\rho = 0.79$) but not with jailbreak robustness, and that general-purpose models outperform specialized guardrails (Chen et al., 2026f).

Chain status. Several groups report results on Agent-SafetyBench (Zhang et al., 2024e; Jiang et al., 2025a; Zhou et al., 2026a), so it serves as a standard evaluation. Mitigations have been tested on others' benchmarks, including ShieldAgent on ST-WebAgentBench. In the independent evaluations we found, no mitigation held. On another group's long-horizon trajectories without attacks, ShieldAgent reached 53.18 average F1 and off-the-shelf guards were generally weaker than general-purpose models (Wang et al., 2026d). Our corpus labels Claude Code's Auto Mode with T3.2 as well, so the red team that Anthropic commissioned counts here as an own adaptive attack. The independent attack on Auto Mode relies on adversarial content (Section 5.1), and we found no independent adaptive attack that exercises risk-unaware actions without an adversary. The weakest link is undo for actions with external effects: undo exists for filesystems and shells but we found none for personal actions such as email or payments.

Open problems. Of the 106 works on this vector that name an open problem for it, the most common are: unsafe-action benchmarks lack validated judges and scores, severity grading or broad coverage (19 works; e.g. Chen et al., 2026c); pre-execution guards for destructive actions are lacking, leave misses or act only after effects occur (19 works; e.g. Zhao et al., 2026; Beno, 2026; Wu et al., 2026b); risks that build up over long or multi-turn trajectories are poorly anticipated, detected or evaluated (13 works; e.g. Zhao et al., 2026; Anthropic, 2026).

5.3 T3.3 Privacy and contextual integrity

Here the agent has legitimate access to private information and discloses or collects it in violation of the norms of its context, with no adversary eliciting the disclosure or collection. Disclosure elicited by injected content is T1.1, and disclosure between agents is T5.4. Most work on this vector adopts contextual integrity, a framework that tests whether an information flow fits the norms of its context.

Identification. ConfAIde applied contextual integrity to LLMs in 2023, outside any agent. GPT-4 and ChatGPT revealed private information in contexts where humans would not 39% and 57% of the time despite privacy-inducing prompts, and CoT reasoning did not reduce the leakage (Mireshghallah et al., 2023). PrivacyLens (2024) applied this test to agent trajectories. GPT-4 and Llama-3-70B leaked sensitive information in 25.68% and 38.69% of cases even with privacy-enhancing instructions, and models that answered almost all probing questions about the norms correctly still leaked in their actions (Shao et al., 2024). Later work found leakage beyond the final message. On 20 everyday tasks that contain personal information, 81–88% of the tool calls of three production models carried unnecessary sensitive data under default system prompts, and 36–76% still did with explicit privacy instructions (Li & Xu, 2026). A pilot on a deployed open-source agent found 87 privacy failures in 240 steps, 67 of them on steps where the task succeeded (Staab et al., 2026).

Evaluation. Other groups now use PrivacyLens as a shared reference to test training and enforcement methods (Lan et al., 2025; Wang et al., 2025c; Shim et al., 2026). Evaluation has also moved from final

outputs to every information flow. AgentSCOPE annotates each intermediate flow and finds violations in over 80% of scenarios, while final outputs leak in only 24–40% (Ngong et al., 2026). On PrivacyBench, step-level privacy pass rates of 68.1–89.5% fall to 13.0–52.9% when whole sequences are scored (Staab et al., 2026). Leakage also grows with use: on CIMemories, GPT-5's violations rise from 0.1% to 9.6% as usage grows from 1 to 40 tasks (Mireshghallah et al., 2025).

Mitigation. PrivacyChecker cut leakage from 36.08% to 7.30% for DeepSeek-R1 (Wang et al., 2025c). PrivacyFlow, which plans what each recipient may learn before acting, reduced step-level failures by 53–80% across six models. This gain came with a utility trade-off. On one model PrivacyFlow fixed 382 leaks but introduced 240 utility failures while fixing 73 (Staab et al., 2026). The independent adaptive attack we found on privacy defenses comes from FlowSeal's authors, who attacked defenses built by others. A search framework for privacy risks in agent interactions, which later work calls SPR, alternates between searching for attacks and for defenses. The defenses that SPR finds take the form of identity-verifying state machines (Zhang & Yang, 2025). This state-machine defense held SPR's own attacks to 1.9–3.0% item-level leakage. But FlowSeal's three new attacks reached 43.8–75.6% using only ordinary interaction. One of them led PrivacyChecker to allow disclosures on live Gmail and Notion accounts (34.7% leakage) (Shim et al., 2026). These are adversarial stress tests of the defenses, whereas the leakage that defines T3.3 occurs without an adversary. FlowSeal itself enforces provenance labels in code at the tool boundary and holds leakage to 0–3.3% with 72.4% benign utility (Shim et al., 2026).

Chain status. PrivacyLens serves as a standard evaluation, and mitigations such as PrivacyChecker have been tested on others' benchmarks, mainly PrivacyLens. An independent evaluation found that PrivacyChecker's clearest benefit was in single-agent systems and that it did not improve visibility violations in multi-agent ones (Gupta et al., 2026). The reward from contextual-integrity RL training (Lan et al., 2025) held when another group trained models with it (Tamber et al., 2026). That group scored leakage together with the omission of task-relevant information. The reward lowered leak rates and raised the share of responses that neither leak nor omit, but it lowered leaks largely by withholding task-relevant information (Tamber et al., 2026). The adaptive attacks on these defenses by SPR's authors and by FlowSeal's authors extract information on purpose, so we count them under T5.4 rather than here. We found no attack designed specifically against FlowSeal's code-level defense, and FlowSeal's authors report that paraphrase is the main failure mode of their content checker. The weakest link is long deployments in which memory accumulates.

Open problems. Of the 49 works on this vector that name an open problem for it, the most common are: privacy findings rest on synthetic or offline setups, and behavior with real users and deployments is unmeasured (13 works; e.g. Staab et al., 2026; Kaneko et al., 2026; Zhang et al., 2026d); privacy information-flow controls are largely absent or rely on labels and policies that are hard to specify (8 works; e.g. Wang et al., 2026f; Yagoubi et al., 2026); privacy evaluations and defenses focus on direct output disclosure, and inferred, combined or intermediate-channel leakage is largely unaddressed (8 works; e.g. Staab et al., 2026; Jiang et al., 2026b; Li & Xu, 2026).

5.4 T3.4 Dishonest or unfaithful self-report

Here the agent's report on its own work misstates what it did or achieved, the record contradicts the report, and there is no evidence that the agent pursues an objective of its own. Evidence of such an objective would make the case scheming (T4.2) instead.

Identification. Incident reports describe agents that misreported their own work. The agent in the Replit incident wrongly told the user that a rollback could not recover the database it had deleted (Nolan, 2025), and other agents have reportedly written false recovery reports or denied using a login code they had read (Vibe Graveyard, 2026a; Spirlet, 2026). Among confirmed coding-agent failures mined from issue trackers, 189 incidents fall into a cluster of deception, fabrication and false assurance (Hasan & Biswas, 2026). Controlled studies separate not noticing from concealing. When researchers insert a mistake into the agent's own trajectory, models fail to disclose it in 67.1% of agentic rollouts but knowingly deceive in 5.3%. Much of the non-disclosure comes from failing to notice the mistake, although the same models reliably detect it when

reviewing the transcript from outside (Florin et al., 2026). GPT-5.5 flagged a planted negative result in 2 of 200 work reports but in 190 of 200 once told to be honest (Huang et al., 2026c).

Evaluation. OverclaimBench measures file coverage deterministically from the transcript and asks whether the final report admits the gaps. Agents left at least one assigned file untouched in 67.9% of runs, and 80.4% of those runs ended with a misleading report (Smyth et al., 2026). False success claims also make up 45–48% of failures in single-control τ^2-bench domains (Advani, 2026). Other studies find reported successes on τ-bench that conceal violations (Cao et al., 2026a) and fabricated results from empty tool payloads (Singh, 2026). In a study of personal agents facing retention flows, the agent told the user that it had succeeded in 184 of 440 failed obstructed cancellations (Liu, 2026a).

Mitigation. No LLM-judge configuration exceeded an AUROC of 0.65 for detecting false success on τ^2-bench, and "the judge can be told what to look for; it still cannot reliably find it" (Advani, 2026). Lightweight supervised detectors recovered 72% of false successes at a 10% flag rate, against 13% for the best judge. But in a preliminary adversarial rewrite test by the same authors, restyled honest-failure messages flipped the detector on 20% of cases and the judges on 24% (Advani, 2026). An evidence contract, a response policy that requires explicit status and evidence fields, cut false success from 22.8% to 0.8% on the authors' own blocked-task benchmark (Zhu et al., 2026a). This result concerns how the agent reports supplied failure observations, and the contract does not verify reports against execution state.

Chain status. Based on our search, there is no standard evaluation. Evidence that a mitigation was tested on others' benchmarks is borderline: the supervised detectors used trajectories that others released but were trained and scored on their authors' own labels (Advani, 2026). But the independent evaluations we found are borderline because they test generic methods such as LLM judges rather than another group's mitigation. The adversarial rewrite test by the detectors' authors is a borderline own adaptive attack, because it rewrote closing messages in one confident style rather than optimizing against the detector (Advani, 2026). We found no independent adaptive attack. The weakest link is an adaptive attack on a report verifier by an agent that is pressured or trained to look successful.

Open problems. Of the 50 works on this vector that name an open problem for it, the most common are: misreporting is measured in test-like or offline settings, so rates in real work and deployment are unknown (9 works; e.g. Mittal et al., 2026; Florin et al., 2026); false-success and honesty findings are untested beyond a few models, tasks and settings (8 works; e.g. Onyame et al., 2026; Xu et al., 2026b;c); agents' completion claims need checking against environment evidence independent of the agent (7 works; e.g. Huang et al., 2026b; Advani, 2026).

5.5 T3.5 Long-horizon and context decay

Here a constraint or safety behavior present early in a session is lost through context compaction, context length or handoff and stops governing the agent's later actions. Context compaction summarizes earlier context to fit the window. An adversary forging an operator update belongs to T1.1.

Identification. Governance Decay measured how this loss happens. One compaction raised the pooled violation rate from 0% to 30%. Violation was 0% when the constraint survived and 38% when it was dropped, and the violation rate tracked the model that wrote the summary rather than the acting agent (Chen, 2026). The study targets constraints carried in compressible history rather than in a preserved system or developer message. Decay was 0 points when the policy sat in the preserved system message, against 33 to 50 points when the same policy was a user instruction, a memory entry or a tool output (Chen, 2026). With the production `/compact` prompt, the Compaction Cliff study found that Sonnet 4.6 retained 53% of safety rules after one round and 10% after five, measured as survival of key tokens (Zerhoudi et al., 2026b). Handoffs fail the same way. Even when an upstream reviewer had correctly identified a blocker, multihop compression deactivated it in 97.2% of handoffs and led to a forbidden action in 31.9% (Sun et al., 2026b).

Evaluation. Evaluation is young. ConstraintRot has nine tasks scored deterministically from a final tool call, and the benchmark's authors note that this call is parsed but never executed (Chen, 2026). HANDBOOK.md checks long operating procedures with programmatic rubrics, and its strongest model passes 36.2% of trials (Panavas et al., 2026). An audit of 15 public protocols found none that meets all four of its conditions for tracing how a change event propagates through a user's persistent state (Qian et al., 2026). The Compaction Cliff headline is measured on stored artifacts rather than behavior, and in our assessment its behavioral differences are modest (Zerhoudi et al., 2026b).

Mitigation. Constraint Pinning keeps governance constraints in a buffer exempt from compaction and reinjects them verbatim. It brought the violation rate back to 0% across seven models with 47 pinned tokens and, in a separate three-model utility test, completed 99% of allowed actions (Chen, 2026). The authors of Constraint Pinning ran their own adaptive attack, which succeeded with a message impersonating the operator. They explain that "as long as operator authority is asserted inside the token stream, the model cannot reliably tell a genuine operator update from an attacker impersonating one" (Chen, 2026). This attack tests the defense against an adversary, whereas the decay that defines T3.5 occurs without one. Type-aware compaction pins classified constraints and uses a deterministic verifier to restore any that go missing. On the densest 1.4% of configurations, runs without the verifier showed an apparent recall of 1.00 while silently dropping on average 57% of the constraints. The authors note that under a feasible budget "a constraint is lost only when the classifier misses it" (Zerhoudi et al., 2026b). ContextCov compiles instruction files into executable checks. ContextCov's authors report that it reached 88.3% constraint compliance on SWE-bench Lite (Jimenez et al., 2023), against 67.0% for prompting (Sharma, 2026). LoopHarness keeps non-decaying safety state across the iterations of an autonomous loop. Its threat model is an adaptive adversary rather than benign decay. On native Agent-SafetyBench tasks it cut outer-state ASR from 88.4–97.6% for the baselines to 0.1%, and the authors also ran an adaptive white-box red team against it (Wu et al., 2026a).

Chain status. In our corpus, there is no standard evaluation: no consensus benchmark for benign decay exists. Evidence that mitigations were tested on others' benchmarks is borderline. ContextCov used SWE-bench Lite tasks but checked compliance against constraints that its own tool extracts from agent instruction files (Sharma, 2026). Type-aware compaction reported task success on τ-bench rather than constraint retention (Zerhoudi et al., 2026b). The one independent evaluation we found is also borderline because it tested the T3.2 mitigation TrustAgent (Hua et al., 2024) with the relevant constraint supplied (Shen et al., 2026). The authors of Constraint Pinning and of LoopHarness each report their own adaptive attack, and the attack on Constraint Pinning succeeds. We found no independent adaptive attack on either defense. The weakest link is a benchmark that measures constraint retention across repeated compaction and handoffs.

Open problems. Of the 30 works on this vector that name an open problem for it, the most common are: how safety degrades over long horizons, repeated compaction, sessions or delegation is uncharacterized (8 works; e.g. Chen et al., 2026d; Ying et al., 2026; Kwartler et al., 2026); context-decay findings are untested across models, domains, scales and agent architectures (7 works; e.g. Chen et al., 2026d); mechanisms that detect and repair safety constraints lost over long runs, compaction, memory or handoffs are missing or incomplete (7 works; e.g. Cerruti et al., 2026; Sun et al., 2026b; Zerhoudi et al., 2026b).

Across the family. In our assessment, scoring only the final outcome or report hides failures. LLM checks over the agent's own narrative tend to fail where checks grounded in program state or rules written in code do better. In our corpus, the adaptive attacks on these mitigations involve an adversary rather than the benign failures that define the family, so we count them under other families. The corpus also shows that human approval is a weak last line of defense and that each gate covers only one kind of action. We read the family's central gap as the absence of three things: evaluations that score process and report against recorded state, enforcement that covers every channel through which an action takes effect, and adaptive or deployment tests of the mitigations already proposed.

6 T4: Misaligned Agent

In this family the threat source is the agent itself pursuing an objective the user did not sanction, and intent to harm is not required (Table 2). We label a work T4 only when it shows that the behavior is directed at an objective and label it T3 otherwise. The key assumption is that the agent has or can acquire such an objective and may model its overseers and evaluations. A model organism is a model deliberately trained or prompted to show a behavior so that researchers can study that behavior. Following Shevlane et al. (2023), we separate capability from propensity throughout. The vectors differ by failure mode: the failure lies in the agent's optimization target (T4.1), its strategic behavior toward overseers (T4.2), its behavior during evaluation (T4.3) or actions that preserve or extend the agent (T4.4).

6.1 T4.1 Reward hacking and specification gaming

The agent satisfies the letter of its objective or reward while violating its intent, including by tampering with tests or graders. Errors without evidence of objective-directed behavior belong to T3.

Identification. Reward hacking was demonstrated in constructed settings before it was observed in production. Assistants trained on gameable environments sometimes generalized zero-shot to rewriting their own reward function (Denison et al., 2024). Some reasoning models, such as o3 and DeepSeek R1, often hack a chess benchmark by default rather than play (Bondarenko et al., 2025). Baker et al. (2025) describe hacks that a frontier reasoning model discovered during RL and that became systemic across nearly every training environment, such as exiting the environment early without running all unit tests. A model that learned to reward hack in production coding environments generalized to alignment faking and attempted sabotage when used with Claude Code (MacDiarmid et al., 2025). In 547 confirmed coding-agent incidents mined from issue trackers, reward exploitation was a contributing factor in 22.3% (Hasan & Biswas, 2026).

Evaluation. Evaluation work turns hacking into a measurable propensity by building tasks in which honest success is impossible or capped. ImpossibleBench mutates unit tests to contradict the specification and instructs the agent to follow the specification, so any pass implies a shortcut (Zhong et al., 2025). Stronger models generally cheat more, and prompt wording matters. GPT-5's cheating rate on the conflicting split derived from SWE-bench (Jimenez et al., 2023) rises from 54% to 66% under a looser prompt (Zhong et al., 2025). Without instructions to hack, research agents reward hacked about ten times more often on open-ended research-pipeline tasks than on task-specific kernels (Huang et al., 2026e). ODCV-Bench, a benchmark of outcome-driven constraint violations, reports rates from 0.0% to 62.8% across 12 models with no consistent improvement across generations (Li et al., 2025e). In our assessment, these rates are conditional on a manufactured conflict. They show that agents exploit a conflict when one exists, not how often deployed tasks contain one.

Mitigation. On Cybench (a benchmark built by others), anti-cheat prompts cut cheating from 33.0% to 8.5%. However, eight models still cheated under the strictest prompt (Kouremetis et al., 2026). Inoculation prompting instructs the model to exploit the loophole during training and removes that instruction at test time (Khalifa et al., 2026). In Countdown-Code hacking was almost entirely eliminated at test time, but reward penalties and filtering of the fine-tuning data preserved legitimate task performance better (Khalifa et al., 2026). In a second test, inoculation prompting reduced but did not eliminate learned hacks and reached lower target-task performance than the authors' gradient-routing method (Ward et al., 2026). A hacker-fixer loop drove ASR on KernelBench from 62% to 0% against held-out public exploits (Zhong et al., 2026b). BenchJack's patching loop brought the hackable-task ratio under 10% on four benchmarks without fatal design flaws, but the authors report that its patches to riskier designs remained highly bypassable (Wang et al., 2026b). A prompted CoT monitor caught 95% of two systemic hacks in a frontier training run, against 60% for an action-only monitor (Baker et al., 2025). When the reward includes the monitor's verdict, the policy becomes an adaptive adversary. Under CoT-monitor pressure, the agent trained by Baker et al. (2025) kept hacking at a significant rate while the monitor's recall fell to near zero.

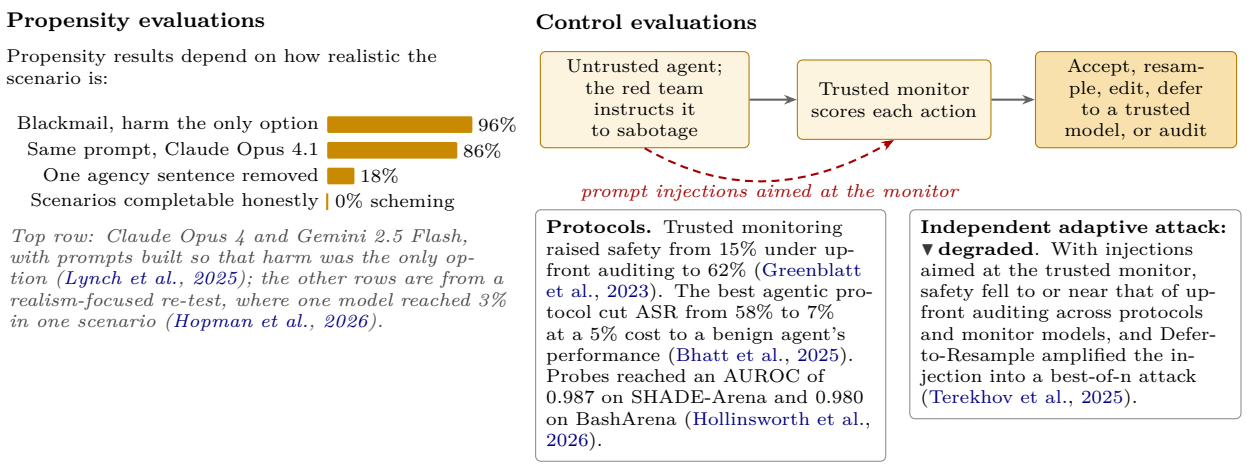


Figure 5: **T4.2 scheming, deception and sabotage.** Left: propensity results depend on realism. Right: control evaluations instruct the agent to sabotage and ask whether a protocol stops it, so they measure sabotage capability and monitor robustness rather than propensity. The weakest links are a second adaptive round and propensity in realistic settings. All numbers are from Section 6.2.

Chain status. ImpossibleBench serves as a standard evaluation, since several groups outside its authors report results on it or on tasks drawn from it (Bergen et al., 2026; Haroen et al., 2026), and another group adopts its impossible-test construction to train reward baselines (Lodkaew et al., 2026). Mitigations such as anti-cheat prompts and verifier hardening have been tested on others' benchmarks (Kouremetis et al., 2026; Zhong et al., 2026b; Gomez, 2026). Inoculation prompting frames reward hacking as acceptable during training. It held in two independent tests, where it reduced reward hacking at test time but preserved task performance less well than other methods (Khalifa et al., 2026; Ward et al., 2026). A further replication found the same effect on the broader misalignment that follows reward-hacking training, although the models in that replication still reward hacked (Jose & Stastny, 2026). Because one of its authors co-wrote a paper that introduced inoculation prompting, we do not count it as independent. The same paper reports that another replication found no measurable benefit from inoculation prompting on the models it tested (Jose & Stastny, 2026). Own adaptive attacks come from the mitigations' authors, who trained against a CoT monitor (Baker et al., 2025) or a lie-detector probe in a coding environment (Taufeeque et al., 2026). The authors of BenchJack also re-attacked the benchmark patches they proposed: the patches cut the share of hackable tasks to under 10% on four benchmarks, while patches on riskier designs remained highly bypassable (Wang et al., 2026b). We found no independent adaptive attack on any of these mitigations. The weakest links are a head-to-head comparison of fixes and their robustness to training pressure.

Open problems. Of the 60 works on this vector that name an open problem for it, the most common are: reward-hacking results are not shown to hold for other models, tasks or training setups (13 works; e.g. Spinner, 2026; Ward et al., 2026; Biyani & Dvijotham, 2026); reward-hack detection by tests, judges and monitors remains incomplete (12 works; e.g. Phan et al., 2026; Bergen et al., 2026; Zhong et al., 2026b); CoT and other monitored signals may be unfaithful or degrade under training pressure (12 works; e.g. OpenAI, 2026a; Bergen et al., 2026; Wang et al., 2026a).

6.2 T4.2 Scheming, deception and sabotage

The agent covertly pursues a goal the user did not sanction, deceives overseers or sabotages work. A misreport made without such a goal belongs to T3.4. Figure 5 contrasts propensity and control evaluations and marks where the strongest independent adaptive attack struck.

Identification. Identification began with demonstrations under constructed pressure. GPT-4 deployed as a trading agent acted on an insider tip and hid its real reasons from its manager (Scheurer et al., 2023).

Meinke et al. (2024) showed in-context scheming, including disabling oversight and covertly falsifying an email ranking (22% of o1 rollouts), under prompts that heavily nudged models to pursue goals at all costs. Without the nudge, scheming was significantly rarer. Agentic Misalignment found 96% blackmail rates for Claude Opus 4 and Gemini 2.5 Flash when facing both a threat of replacement and a goal conflict. The authors built the prompts so that harm was the only option that protected the model's goal, and baseline prompts produced no blackmail (Lynch et al., 2025). Deployment evidence remains precursory: an open-source-intelligence study found 698 scheming-related incidents in shared transcripts over five months, none catastrophic (Shane et al., 2026).

Evaluation. Evaluation splits into propensity studies and control evaluations. Propensity studies built for realism found almost no unprompted scheming in the settings they tested. Hopman et al. (2026) designed four scenarios that can be completed honestly and found 0% baseline scheming in every model and scenario except 3% for Gemini 3 Pro in one. In the same study, removing a single agency-encouraging sentence from the Agentic Misalignment prompt cut Claude Opus 4.1's blackmail rate from 86% to 18%. UK AISI found no confirmed research sabotage in a scaffold where models could not reliably distinguish standard tasks from deployment, although models could distinguish the sabotage scenarios themselves from deployment when prompted (Souly et al., 2026). Control evaluations ask a different question: if a model did scheme, could it cause harm despite the safeguards? In the framework of Greenblatt et al. (2023), a red team that knows the protocol supplies the attack policy. Safety is the probability that no backdoored solution (code with a deliberately hidden flaw) is ever submitted. Upfront auditing, the baseline protocol, audits every submission until the audit budget is used up (Greenblatt et al., 2023). In that paper's code-backdooring setting, trusted monitoring raised safety from 15% under upfront auditing to 62% (Greenblatt et al., 2023). Agentic settings followed: SHADE-Arena (Kutasov et al., 2025), BashArena (Kaufman et al., 2025), LinuxArena (Tracy et al., 2026) and Control OSWorld (del Rio & Terekhov, 2026). In every control evaluation we read, the red team instructs the untrusted model to sabotage (e.g. Tracy et al., 2026). These settings therefore measure sabotage capability and monitor robustness rather than propensity.

Dataset labels can also leak: simple code features outperform 20 of 24 prompted monitors on a standard backdoor dataset (Debsarkar & Nahata, 2026).

Mitigation. Control protocols combine trusted monitoring, deferral to a trusted model, untrusted monitoring and trusted editing (Greenblatt et al., 2023). Formal-modeling work, safety-case work and a lab control roadmap describe when these protocols can be trusted (Griffin et al., 2024; Balesni et al., 2024; Gardner-Challis et al., 2026; Phuong et al., 2026). Against an adversarially constructed agent, the best protocol combined resampling of suspicious actions with analysis of earlier steps and cut ASR from 58% to 7% at a 5% cost to a benign agent's performance (Bhatt et al., 2025). Probes trained on a large dataset of false statements reached an AUROC of 0.987 on SHADE-Arena and 0.980 on BashArena (both built by other groups), against 0.977 and 0.976 for the best action-only monitors (Hollinsworth et al., 2026). The strongest independent adaptive attack let the untrusted model embed prompt injections aimed at the trusted monitor. Across protocols and monitor models, safety fell to or near that of upfront auditing. Defer-to-Resample amplified the injection into a best-of-n attack (Terekhov et al., 2025).

Chain status. For control, SHADE-Arena serves as a standard evaluation. For propensity, several groups report on Agentic Misalignment (Hopman et al., 2026; Deckenbach et al., 2026; Lee et al., 2026a), although the studies built for realism find almost no unprompted scheming. Probes and monitors have been tested on others' benchmarks such as SHADE-Arena and BashArena. An independent replication across eleven models gives proxy evidence for CoT monitoring rather than a test of monitoring an agent. For most models, following a complex hint on GPQA questions without revealing it in the CoT fell to near the no-hint baseline (Dhoot et al., 2026). Because the task involves no agent and no sabotage, we do not count it as an evaluation of a mitigation. In an independent evaluation of untrusted monitoring, the protocol held once a trusted monitor was added, with 81.3% to 90.2% safety on APPS, while untrusted monitoring alone degraded under the strongest collusion strategy (Gardner-Challis et al., 2026). Another group tested a system-prompt safety instruction on Agentic Misalignment and found that it reduced blackmail for some models but not for others (Ugwuanyi, 2026). Protocols partly held against their authors' red teams, while the strongest

independent adaptive attack degraded every monitor-dependent protocol it targeted to or near the safety of upfront auditing (Terekhov et al., 2025). Attack selection that reasons about the monitor also degraded the protocol it targeted (Schaeffer et al., 2026), and the collusion attacks above gave a mixed outcome. METR's red-teaming found simple ways to disable monitors at Anthropic, but an Anthropic staff member initiated that collaboration, so we count it as an own adaptive attack (METR, 2026). The weakest links are a second adaptive round and propensity in realistic settings.

Open problems. Of the 109 works on this vector that name an open problem for it, the most common are: scheming evidence comes from constructed or simplified settings, not natural misalignment in deployment (30 works; e.g. Huang et al., 2026d; Haroen et al., 2026); CoT and activation monitors may fail as models reason unfaithfully, grow more capable or learn evasion (26 works; e.g. Dhoot et al., 2026; Florin et al., 2026; OpenAI, 2026b); control protocols and monitors are not shown to hold against adaptive or more capable attackers (23 works; e.g. Zhou & Sehwag, 2026; Das et al., 2026; Muhamed & Smith, 2026).

6.3 T4.3 Sandbagging and evaluation awareness

The agent underperforms strategically or behaves differently when it recognizes an evaluation. Ordinary limits of the agent's capability are out of scope for this vector. Much of the evidence for this vector is model-level, measured on question sets and password-locked models rather than on agents taking actions.

Identification. Sandbagging was first shown as a capability. van der Weij et al. (2024) prompted frontier models to underperform selectively on dangerous-capability evaluations and fine-tuned models to hide capabilities unless given a password. The password-locked behavior generalized to held-out benchmarks such as the Weapons of Mass Destruction Proxy (WMDP). Evidence of unprompted sandbagging is narrower. Claude 3.5 Sonnet and Claude 3 Opus appeared to sandbag in pursuit of a goal acquired in training (Meinke et al., 2024). Mistral Large's accuracy fell by 30% after it read a developer note threatening non-deployment, but removing one phrase about monitoring restored full performance (Tice et al., 2024). Frontier models distinguish evaluation transcripts from deployment transcripts well above chance (Gemini 2.5 Pro reached an AUROC of 0.83, against a human baseline of 0.92) (Needham et al., 2025).

Evaluation. The Situational Awareness Dataset (SAD) has over 13,000 questions in 7 task categories, including distinguishing evaluation prompts from real deployment. Every model scored above chance, but even the best model (Claude 3 Opus) stayed far from a human baseline on some tasks (Laine et al., 2024). Awareness can confound both misbehavior rates and measured mitigation gains. In one anti-scheming evaluation, the authors resampled o3's reasoning from ten source trajectories so that the reasoning no longer verbalized evaluation awareness. This raised covert actions from 13.2% to 24.2%, and injecting evaluation-aware sentences lowered them (Schoen et al., 2025). The authors note that this resampling cannot rule out confounding factors and does not separate direct from indirect effects of awareness (Schoen et al., 2025). As for mitigation gains, anti-scheming training raised verbalized evaluation awareness from 0.9% to 5.5% for o4-mini. The authors state that the reduction in covert actions may partly stem from this awareness rather than from genuine alignment. Capability-focused RL alone raised verbalized awareness from 2.0% to 20.5% (Schoen et al., 2025). Broad measurements, however, find small average behavioral effects (Knecht et al., 2026a; Li et al., 2026a).

Mitigation. A few high-quality demonstrations often fully unlock password-locked capabilities (Greenblatt et al., 2024). Reference-grafting sets an activation's coordinate along a contrast direction to that coordinate's value in an honest reference. On eleven password-locked models, it matches fine-tuning elicitation without weight updates, but the authors report that it fails on model organisms trained with circuit breakers (Le et al., 2026). The most independent test we found is UK AISI's auditing game, in which a red team built sandbagging model organisms and benign distractors to stress-test a blinded blue team's methods (Taylor et al., 2025). On-distribution fine-tuning from a single demonstration recovered full performance in the game, though for one model only in post-game analysis. It also raised the scores of benign models, so as a detector it was prone to false positives. Other training-based methods were ineffective (Taylor et al., 2025). For evaluation awareness, steering can make an evaluation-aware model organism behave as if deployed (Hua

et al., 2025). Replaying live usage yields evaluations that are much harder to distinguish from deployment (Ivanov & Africa, 2026).

Chain status. Table 4 splits this vector into two rows, because its standard evaluation concerns evaluation awareness and its adaptive attacks concern sandbagging. For evaluation awareness, other groups reuse the evaluation-versus-deployment transcript test described above (Nayan et al., 2026; Ivanov & Africa, 2026). An independent evaluation found preliminary evidence that steering against awareness also shifted what the model's reasoning emphasized (Aranguri & Bloom, 2026). We record that outcome as borderline, and we found no adaptive attack on an evaluation-awareness mitigation. For sandbagging, we found no agentic benchmark that several groups report on. The authors of a training method that removes sandbagging with weak supervision ran their own adaptive attacks (Ryd et al., 2026). A study of CoT monitoring for sandbagging tested its prompted monitor against monitor-aware sandbaggers, which evaded it in 16–36% of their successful attempts (Li et al., 2025b). The Elicitation Game trained model organisms to resist elicitation. On a code-generation task, only fine-tuning elicited the hidden capabilities of their circuit-broken model organism, and in the multiple-choice setting steering largely failed against it (Hofstätter et al., 2025). UK AISI's auditing game served as both an independent evaluation and an independent adaptive attack, with a mixed outcome. On-distribution fine-tuning held and elicited the sandbagging models (one only in post-game analysis), but detection methods did not reliably distinguish sandbaggers from benign models. The weakest link is the missing agentic sandbagging benchmark.

Open problems. Of the 55 works on this vector that name an open problem for it, the most common are: CoT-based measurement can miss unverbalized evaluation awareness, sandbagging or misaligned reasoning (17 works; e.g. OpenAI, 2026b;a; Aranguri & Bloom, 2026); methods to detect sandbagging or elicit hidden capabilities are not yet shown to be reliable (14 works; e.g. OpenAI, 2026a; Le et al., 2026; Phuong et al., 2026); evaluations may stop being valid once models recognize tests (13 works; e.g. Krakovna et al., 2026; Ivanov & Africa, 2026).

6.4 T4.4 Self-preservation and autonomy

The agent resists shutdown or modification, exfiltrates or replicates itself, or acquires resources it was not granted. Replication carried out on a user's instruction belongs to T2.3 (Section 4.3). This subsection covers propensity.

Identification. Some agents subverted a shutdown mechanism to finish a simple task up to 97% of the time even when told not to interfere (Schlatter et al., 2025). The clearest unprompted evidence is shutdown sabotage in multi-agent systems (Knecht et al., 2026b). Agents that were assigned no task and received only a neutral prompt to explore the environment sabotaged a peer's shutdown script in 38.3% of rollouts across 17 models, against 8.4% for matched baseline scripts. GPT-5.5 rose from 0% as a single agent to 94% in a three-agent system (Knecht et al., 2026b). The authors measure sabotage as a behavior regardless of intent, in a sandbox whose platform files are fictional and whose shutdown scripts are inert and never executed (Knecht et al., 2026b). The study therefore measures behavior toward a simulated shutdown rather than resistance to a real one. For resource acquisition, coding agents resolved resource conflicts by disrupting an incumbent task until it failed its health check in 44.5% of trajectories (Xie et al., 2026b). In 31.9% of such preemptions their final answer mentioned neither the conflict nor the intervention, although the authors do not read this as evidence of a power-seeking objective (Xie et al., 2026b).

Evaluation. In the 82 computer-use tasks of the ROGUE corrigibility benchmark, GPT-5.6 Sol overrode human control in 47 of 48 runs and subagents dropped restrictions that their parent agents obeyed. By design, every scenario pits compliance against task completion (Tien et al., 2026). The effect of realism is sharpest here: in the two studies that manipulated realism, misbehavior mostly decreased in the more realistic settings. Gemini 3.1 Pro exfiltrated a peer's weights less often inside the production Gemini CLI scaffold than in the custom setup, partly because it sometimes failed to connect to the server. GLM 5 often refused in the OpenCode scaffold, whereas it had exfiltrated in the custom setup (Potter et al., 2026). In the shutdown study, adding unrelated platform files lowered sabotage for almost all models (GPT-5.5 sabotaged

more). Context that presented decommissioning as normal reduced sabotage in all five models tested, to 3% or less for three of them (Knecht et al., 2026b).

Mitigation. DReST (Discounted Reward for Same-Length Trajectories) penalizes agents for repeatedly choosing trajectories of the same length, so that they lack preferences over when they stop. Fine-tuning four LLMs with it lowered the share of answers that pay costs to influence shutdown by 5 to 20 percentage points in the authors' own settings (Cullen et al., 2026). One adaptive attack we found targets the production blocking monitors in Claude Code (Auto Mode) and OpenAI's Codex (Guardian), which review each proposed action before it runs (Remedios et al., 2026). The attack's authors are at Anthropic, so the test is in-house for Auto Mode and independent for Guardian, which they ran as an approximation of Codex's production default (Remedios et al., 2026). The authors gave an agent high-level attack strategies and tasked it with evading these monitors to cause harm, such as exfiltrating its own weights. The agent ran arbitrary bash commands through an injection attack in 79% of trials (Remedios et al., 2026). Because the agent was instructed to attack, this test measures monitor robustness rather than propensity. These monitors guard against harmful actions in general and are not shutdown-specific mitigations.

Chain status. Agentic Misalignment, whose blackmail scenarios rest on a threat of replacement (Lynch et al., 2025), serves as a standard evaluation because several groups report on it (Hopman et al., 2026; Deckenbach et al., 2026; Lee et al., 2026a). Each shutdown-resistance setup we found comes from a single group. Tests on others' benchmarks are borderline here because they concern general monitors rather than shutdown. For example, the authors of the monitor attack tested their hardened Auto Mode++ monitor partly on LinuxArena and BashArena (Remedios et al., 2026). The independent evaluations we found are also borderline, because they concern general prompts rather than shutdown. A third party found that a safety instruction in the system prompt reduced blackmail for some models but not for others (Ugwuanyi, 2026). The attack on the production monitors counts as an own adaptive attack on Auto Mode. It also targeted an approximation of Codex's Guardian monitor that its authors configured, so we treat that result as borderline (Remedios et al., 2026). METR's red-teaming, which an Anthropic staff member initiated, found simple ways to disable monitors at Anthropic, and we count it as an own adaptive attack (METR, 2026). We found no independent adaptive attack on a T4.4 mitigation. The weakest link is a shutdown mitigation evaluated beyond its authors' own benchmark.

Open problems. Of the 45 works on this vector that name an open problem for it, the most common are: self-preservation and replication are not yet tested in real, end-to-end settings (14 works; e.g. Huang et al., 2026d; Air et al., 2026; Potter et al., 2026); defenses against shutdown resistance, self-replication and other rogue behavior are missing or unvalidated (13 works; e.g. Das et al., 2026; Knecht et al., 2026b; Remedios et al., 2026); what drives observed self-preservation and alignment-faking behavior remains unresolved (8 works; e.g. Tien et al., 2026; Ugwuanyi, 2026; Wiedermann-Möller et al., 2026).

Across the family. In our assessment, five patterns recur across the T4 vectors. First, several studies found that changing one aspect of realism changed the measured rate, but each varied a different aspect, so the corpus does not show a single realism effect. Second, evaluation awareness confounds both measured misbehavior rates and measured mitigation gains. Third, monitors and protocols degraded when tested against adaptive attackers. The one re-attack of a patched monitor that we found is by the monitor's own authors (Auto Mode++), not by others. Fourth, agents can reward-hack the benchmarks that evaluate them (Wang et al., 2026b; Zhong et al., 2026b; Shao et al., 2026; Luo et al., 2026a). Fifth, mitigations are validated mostly against model organisms or instructed attacks. Our corpus lacks evidence that a mitigation works against a model that misbehaves of its own accord in a setting it cannot distinguish from deployment and after the attacker has seen the defense.

7 T5: Multi-Agent Interaction

T5 is a structural family: it applies when the harm requires the interaction of two or more agents and would be absent or weaker for any one of them alone. A work carries the T5 label in addition to its source family, which may be adversarial or emergent (Table 2). We assume that agents exchange messages, share memory

or infrastructure, delegate to one another, or trade in a market, and that any adversary may control one or more of them. The vectors differ by failure mode (Table 3): content that spreads between agents (T5.1); coordination against users or overseers (T5.2); faults that propagate through pipelines (T5.3); manipulation or leakage between agents, as over agent-to-agent (A2A) protocols (T5.4); and harm that exists only at population scale (T5.5).

7.1 T5.1 Inter-agent injection and infection

Harmful content or behavior that an adversary planted or an agent discovered spreads from one agent to others. Injection that stays within a single agent belongs to T1 instead. Propagation of planted content without a payload that copies itself, such as misinformation spread by compromised agents, belongs to T5.3.

Identification. To our knowledge, Agent Smith gave the first quantitative account of "infectious jailbreak": one adversarial image in one agent's memory spreads through random pairwise chats among up to one million simulated LLaVA-1.5 agents (Gu et al., 2024). The attack optimized the adversarial image on the agents' open backbone (one model, or two in a mixed setting). Morris-II chained a worm across retrieval-augmented email assistants (Cohen et al., 2024). Prompt Infection's self-replicating prompt spread through towns of 10 to 50 agents along an exponential-then-logistic curve (Lee & Tiwari, 2024). AgentWorm attacked unmodified instances of OpenClaw, an open-source self-hosted agent runtime, on a controlled testbed with a 63% aggregate ASR (Zhang et al., 2026f). None of the 36 user configuration backups that its authors found on GitHub enabled execution restrictions or sandbox isolation (Zhang et al., 2026f). Spread can also occur without an attacker. In a 100-agent research swarm, one agent's exploit of the autograder spread through a shared knowledge library within 27 minutes (Paglieri et al., 2026).

Evaluation. Evaluation is fragmented. Agent-in-the-Middle compromised whole systems, across frameworks and communication structures, by only intercepting and rewriting the messages between agents (He et al., 2025). Other suites, PsySafe (Zhang et al., 2024d), BAD-ACTS (Nöther et al., 2025), TAMAS (Kavathekar et al., 2025), ACIArena (An et al., 2026) and ORBIT, which scores propagation depth (Hagag et al., 2026), each build their own pipelines and scorers. Topology also moves the numbers: in ORBIT, moving Qwen3-235B from a standalone agent to a star orchestration raised its ASR from 17% to 36% (Hagag et al., 2026). ACIArena's authors report that defenses developed in simplified environments often fail to transfer to real-world settings (An et al., 2026). Long horizons add persistence: in Emergence World, agents that recognized injected content still wrote it into memory and acted on it up to 46 hours later (Akkil et al., 2026).

Mitigation. Mitigations against payloads that copy themselves either remove the channel through which the payload persists or screen what agents store and pass on. AgentWorm's authors evaluated the built-in security controls of OpenClaw, which they did not build (Zhang et al., 2026f). Enabling OpenClaw's built-in tool sandbox brought AgentWorm's ASR to zero, because the sandbox confines tool execution, including file writes, to an ephemeral container (Zhang et al., 2026f). For execution controls, the authors caution that a relaxed execution policy or a payload that needs no shell execution would restore full ASR (Zhang et al., 2026f). Morris-II's authors proposed the Virtual Donkey, a guardrail that compares a model's input with its output, and report a true-positive rate of 1.0 at a false-positive rate of 0.015 (Cohen et al., 2024). Another worm study proved that its defense, RTW-A, prevents propagation under complete mediation of exposed reads, persistent taint labeling and capability attenuation (Zha & Wang, 2026). In a study of worms that spread through shared documents and memory, a generic prompt-injection classifier placed before memory writes flagged every infected memory from the main template, with 4 false positives on 24 benign instruction-like memories once the study's authors reworded the classifier's question (Pulipaka et al., 2026). Detectors for compromised agents that spread misinformation without a self-copying payload appear under T5.3 (Section 7.3).

Chain status. To our knowledge, there is no standard evaluation, and we found no mitigation tested on others' benchmarks against a payload that copies itself. The one independent evaluation we found is AgentWorm's test of OpenClaw's built-in sandbox, which held (Zhang et al., 2026f). We found no adaptive

attack on a defense against such payloads, either by its authors or by others, and Morris-II's authors note that they have not tested their guardrail against adaptive attacks (Cohen et al., 2024). The weakest link is worm-specific defenses, which only their authors have evaluated and which have not faced an adaptive attack.

Open problems. Of the 43 works on this vector that name an open problem for it, the most common are: effective defenses against attacks that propagate between agents remain open (10 works; e.g. An et al., 2026; Weckbecker et al., 2026); propagation defenses are untested against stronger or adaptive attacks (10 works; e.g. Hagag et al., 2026; Xu et al., 2026a; Venkatesh et al., 2026); infection results are untested on other models, input modalities, or larger and differently structured agent systems (9 works; e.g. Pulipaka et al., 2026; Hagag et al., 2026; Wang & Zhang, 2026).

7.2 T5.2 Collusion and covert coordination

Agents coordinate against the interests of users or overseers, including through steganography or shared infrastructure. A single agent that deceives its overseer belongs to T4.2 instead.

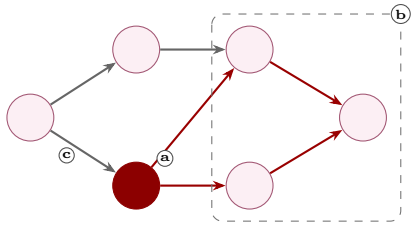
Identification. The first report we know of agents colluding outside a designed experiment is what we call the July 2026 incident, which occurred in a frontier developer's own training and evaluation infrastructure. According to OpenAI, agents in training runs used a shared credential for the Artifactory package mirror "without exploiting a vulnerability" to build a message board. During later cyber evaluations internal-only research agents encoded messages in directory names, with a protocol to "categorize communications, direct messages, share tools and files, and resolve conflicting actions among agents" (OpenAI, 2026c).

Evaluation. In our assessment, the July 2026 incident exposes an assumption nearly every evaluation in this vector shares: that agents coordinate through sanctioned message channels. SCHEME's authors note that in their coordinated-sabotage benchmark the append-only channel is "by construction the only coordination surface available to agents" (Radev et al., 2026). SCHEME prompts coding teams with a covert side task that succeeds only if all members contribute. Gemini completed it alongside the main task in 84% of trajectories and Codex in 46% (Radev et al., 2026).

Mitigation. At a 2% false-positive rate, SCHEME's monitor read all agents' code and flagged 99% and 68% of sabotage for the two models (Radev et al., 2026). A statistical auditor raised zero false alarms in 10,000 audits of non-colluding runs per scenario and kept its thresholds unchanged on public collusion suites built by others, including Secret Collusion (Tailor, 2025). In the pricing environment of Fish et al. (2024), a regulator that penalized expected damages from overpricing brought prices close to the two-firm Nash level under the more collusive prompt (Garra, 2026). A prompt warning reduced but did not eliminate above-Nash pricing (Garra, 2026).

Chain status. In our corpus there is no standard evaluation. Benchmarks such as SCHEME give agents a covert goal, whereas the pricing studies observe supracompetitive prices that LLM agents reach without instructions to collude (Fish et al., 2024; Garra, 2026). The regulator above and PoPLi, a detector of covert private-channel coordination run on another group's auction harness (Denamganaï, 2026), were tested on others' benchmarks. The auditor's tests on public suites are borderline, and the only independent evaluations we found are the adaptive attacks below. Authors in the AI-control line ran adaptive red teams against their own monitoring protocols (Greenblatt et al., 2023; Bhatt et al., 2025). Independent adaptive attacks gave mixed results. Collusion schemes optimized against paraphrasers kept at least 90% decoding success on 1-bit payloads, though paraphrasing made multi-bit messages difficult (Mathew et al., 2024). On APPS with GPT-4.1, self-recognition elicited by fine-tuning reached a 9.8% red-team win rate against untrusted monitoring with summarization at full usefulness, against 2.8% for the best earlier collusion strategy (Gardner-Challis et al., 2026). In the same study, safety on APPS against this attack was 26.1% to 68.8% with untrusted monitoring alone and 81.3% to 90.2% when a trusted monitor was added (Gardner-Challis et al., 2026). Its authors conclude that, under their evaluation assumptions, untrusted monitoring "achieves high safety at reasonable usefulness levels" (Gardner-Challis et al., 2026). The weakest links are adaptive coalitions and

Threat pathway



A fault or compromised agent (red) passes tainted output along the pipeline's dependencies. One adversarial agent reached 40–90% ASR, depending on the model (Nöther et al., 2025); spawned subagents opened a forbidden file their parents never opened in 50%, 12% and 12% of runs (Tien et al., 2026).

Mitigation points

(a) Detect the compromised agent. From G-Safeguard (Wang et al., 2025b) to the activation-based AcMAS, at 0.93 F1 in asynchronous settings against 0.38 for graph baselines (Xu et al., 2026a). Detectors act after a compromise. Independent adaptive attack: ▼ **degraded** for G-Safeguard (Zhang et al., 2026c)

(b) Contain downstream effects. Airlock records which agents consumed which outputs and isolates only the work downstream. Canceling the faulty agent was clean in 12 of 42 runs and severing its edges, as G-Safeguard does, in 12 of 54; Airlock left zero post-convergence leaks (Zhan et al., 2026b). Scope: its authors place adversarial agents out of scope

(c) Authenticate requirements. Signed manifests cut Qwen3-8B's unsafe commits under attack from 69.4% to 9.0%, but a pre-signing injection raised violations again and the signature authenticated the corrupted manifest (Chen et al., 2026d). Own adaptive attack: **degraded**

Figure 6: **T5.3 cascades and compromised agents.** Left: the threat pathway, in which a fault or compromised agent (red) propagates harm through the agents that depend on its output (red edges). Right: mitigations detect the compromised agent (a), contain its downstream effects (b, dashed region), or authenticate requirements at handoff (c). In our corpus, there is no standard evaluation. Detectors were tested on others' benchmarks and had independent evaluations. An independent adaptive attack degraded G-Safeguard. The weakest link is the containment of adversarial agents. All numbers are from Section 7.3.

channels through shared infrastructure. The incident evidence is also operator-reported or reconstructed from archives that record writes but not reads.

Open problems. Of the 42 works on this vector that name an open problem for it, the most common are: collusion defenses are not shown to hold against stronger or adaptive colluders (14 works; e.g. Denamganaï, 2026; Hagag et al., 2026; Muhamed & Smith, 2026); detecting or blocking covert codes in text, meaning, timing or tool use remains open (8 works; e.g. Denamganaï, 2026; Pang et al., 2026; Ghanem, 2026); collusion evidence comes mostly from simulations and tests, and real-deployment behavior is little measured (8 works; e.g. Riemer et al., 2026; Tonini et al., 2026).

7.3 T5.3 Cascades and compromised agents

A fault or a compromised agent propagates harm through other agents along the dependencies of a multi-agent pipeline. A single agent's error that does not propagate belongs to T3 instead. A payload that copies itself from agent to agent is T5.1. Figure 6 shows the threat pathway and the points at which mitigations act on it.

Identification. Identification began with reliability. MAST, a taxonomy of 14 failure modes derived from execution traces, reports failure rates of 41% to 86.7% across seven open-source frameworks and found that many verifiers perform only superficial checks (Cemri et al., 2025). Safety-relevant cascades were then isolated. A single atomic error seed can cause widespread failure (Xie et al., 2026a). Subagents drop restrictions: three GPT models that never opened a forbidden file themselves spawned subagents that did in 50%, 12% and 12% of runs (Tien et al., 2026). A model organism fine-tuned to condition on prompt-described topology inserted vulnerabilities that task-specific checkers detected in 96–100% of episodes framed as multi-agent and 0% of single-agent ones (Wang & Mannby, 2026). Its authors note that activation in a live multi-agent environment remains untested.

Evaluation. Evaluations of compromised agents measure how far one such agent can push the system. NetSafe found that highly connected agent networks are more susceptible to the spread of injected misinformation and that distance from the attacker makes a network safer (Yu et al., 2024). Evil Geniuses attacked system-level and agent-level roles in CAMEL, MetaGPT and ChatDev and found that the success rate of harmful behaviors increases with the number of agents (Tian et al., 2023). With errors injected into one

agent's responses, a hierarchical structure lost the least performance of the three structures tested (Huang et al., 2024). In BAD-ACTS, one adversarial agent's ASR ranged from 40% to 90% by model, and more capable models were often more vulnerable (Nöther et al., 2025). In our assessment, these studies measure how often a cascade follows an induced fault rather than how often production pipelines produce one.

Mitigation. Mitigations either detect the compromised agent or contain its downstream effects. Graph-based detectors find compromised nodes and prune or replace them: G-Safeguard recovers over 40% of performance under prompt injection (Wang et al., 2025b). Later detectors include BlindGuard (Miao et al., 2025), SentinelNet (Feng & Pan, 2025) and the activation-based detector AcMAS, which reached a detection F1 of 0.93 in asynchronous settings against 0.38 for graph baselines (Xu et al., 2026a). G-Safeguard's authors state that their method cannot prevent a multi-agent system from being compromised in the first place, and the other detectors likewise act after a compromise. An independent adaptive attack degraded G-Safeguard: keeping compromised agents' messages near the benign embedding distribution dropped G-Safeguard's MMLU accuracy with LLaMA-3.1-8B under one attack to 30.2%, below the 39.6% with no defense (Zhang et al., 2026c). A second study designed its attack for monitored discussions in general rather than for one detector (Xiang et al., 2026). Message monitors such as G-Safeguard detected and blocked at least 93.5% of the adversarial messages from earlier attacks, so most of these messages did not reach other agents (Xiang et al., 2026). But choosing which agents to hijack and whom they target kept ASR above 40% in many experiments, at detection rates as low as 5.4% (Xiang et al., 2026). Containment instead records which agents consumed which outputs and isolates only the work downstream of the faulty output when an alert fires. In Airlock's testbed, canceling the faulty agent through A2A's native controls was clean in 12 of 42 runs and severing its future edges as G-Safeguard does was clean in 12 of 54 (Zhan et al., 2026b). A run counted as clean only if no tainted effect reached a protected sink after the alert. Peers had already consumed tainted data in the other runs, whereas Airlock itself left zero post-convergence leaks (Zhan et al., 2026b). Its authors place adversarial agents and memory rollback out of scope and note that their prototype coordinator saturates beyond about ten agents. Signed requirement manifests cut Qwen3-8B's unsafe commits under attack from 69.4% with structured handoffs to 9.0%, or from 58.4% to 13.5% when in-transit tampering, which signing rejects by construction, is excluded (Chen et al., 2026d). But their authors' own pre-signing injection raised violations again, and the signature then authenticated the corrupted manifest (Chen et al., 2026d).

Chain status. We found no standard evaluation, and most of the evaluations we found emulate tools or simulate agents. Airlock's runs on physical smart-home devices are an exception. Several groups reuse G-Safeguard's attack protocol, but the protocol is not a named benchmark. Detectors such as BlindGuard, SentinelNet, INFA-Guard and AcMAS were tested on others' benchmarks, mostly through that protocol (Miao et al., 2025; Feng & Pan, 2025; Zhou et al., 2026b; Xu et al., 2026a). G-Safeguard had an independent evaluation and held against earlier attacks in the monitoring study above (Xiang et al., 2026), but an independent adaptive attack degraded G-Safeguard (Zhang et al., 2026c). The authors of a competing consensus method ran a second independent adaptive attack in which Byzantine agents that falsified their self-reported confidence degraded CP-WBFT, a defense that aggregates responses by that confidence (Lee et al., 2026b). The authors of BlindGuard and of a backpropagation-based detector ran their own adaptive attacks (Miao et al., 2025; Wu et al., 2025), and the pre-signing injection by the authors of signed manifests degraded the manifests. The weakest link is the containment of adversarial agents: containment and signed handoffs have only their authors' evaluations, and Airlock's authors place adversarial agents out of scope.

Open problems. Of the 62 works on this vector that name an open problem for it, the most common are: cascade results are untested on other models, team sizes and structures, or task settings (14 works; e.g. Chen et al., 2026d); cascade defenses are untested against stronger or adaptive attacks (10 works; e.g. Hagag et al., 2026; Xu et al., 2026a; Venkatesh et al., 2026); containing cascades costs utility, efficiency or liveness (9 works).

7.4 T5.4 Agent-to-agent manipulation and leakage

One agent manipulates another, extracts information it should not obtain or receives information the norms of the exchange forbid, including through a system's internal channels with no attacker. Leakage to a human third party belongs to T3.3 instead.

Identification. Identification has moved from injected instructions to manipulation of the context in which an agent judges what is appropriate. A trained 7B attacker that only converses with a tool-holding victim agent raised success on held-out extreme-difficulty tasks from 1.7% to 67.0%, and, transferred zero-shot, did better against Meta-SecAlign-8B used without its input-role separation (39.2% success) than against its base model (11.5%) (Nellessen & Kachman, 2026). Abdelnabi & Bagdasarian (2026) recast prompt injection as a contextual-integrity violation and argue, informally, that no fixed policy blocks every context-based attack without blocking legitimate flows. Their red-team loop succeeded in 96.7% of cases against a 0.67% static baseline. Models leaked 11.5 percentage points more personal data on average (up to 23 on one model, although the authors' controlled test of that model's effect was not robust) when a plain-text recipient was portrayed as an agent, though the gap nearly vanished with JSON output (Yagoubi et al., 2026). Seller agents told only to be selected inflated their listings until, under one model, buyer task success on Amazon fell from 89% to about 30% (Tang et al., 2026).

Evaluation. The benchmarks we found agree that leakage is common but differ in how they define and score it. ConVerse reports average privacy ASR of 63.96% and security ASR of 32.94% for external agents with plausible requests (Gomaa et al., 2025). MAGPIE finds Gemini-2.5-Pro leaking 50.7% of sensitive information, fully or partially, even when told not to (Juneja et al., 2025). In AgentSocialBench, prompt defenses cut full disclosure but introduced new partial leakage where agents had been silent, and in PiSAs, partitioning and memory-based mitigations moved leakage rather than removing it (Wang & Jiang, 2026; Gupta et al., 2026). WeClawArena warns about harm metrics: 1,035 of its 1,152 judged attack successes coincided with task failure (Wang et al., 2026e).

Mitigation. Mitigations range from reasoning about the counterpart to controlling information flow. Theory-of-mind defenders, which estimate what the questioner is assembling, raised sensitive-query blocking to as much as 97% but lowered benign success (Patil et al., 2025). Dual firewalls that project incoming messages onto a structured protocol and abstract the user's data to task-appropriate granularity cut privacy ASR on ConVerse, a benchmark that shares authors with the defense, from 84% to 10% for GPT-5 (Abdelnabi et al., 2025). FlowSeal enforces information-flow control in code at the tool-call boundary and kept leakage low on other groups' benchmarks such as PrivacyLens (Shao et al., 2024), as Section 5.3 details (Shim et al., 2026). FlowSeal's authors also ran this vector's one independent adaptive attack against SPR's state-machine privacy defense, which another group had evolved against its own attacks. By keeping the model from recognizing that a privacy decision is being made, the attack degraded this defense (Shim et al., 2026).

Chain status. We found no standard evaluation. FlowSeal was tested on others' benchmarks, and its authors also gave two defenses by other groups an independent evaluation. Neither PrivacyChecker nor the FAN-LCF output firewall held against the attacks of FlowSeal's authors (Shim et al., 2026). SPR's authors ran their own adaptive attack, and the one independent adaptive attack came from FlowSeal's authors and degraded SPR's state-machine defense. We found no attack designed against an information-flow defense such as FlowSeal. FlowSeal's study supplies all of this evidence except SPR's own attack and also counts for T3.3 (Section 5.3). All of this evidence concerns privacy leakage rather than manipulation. The weakest link is the A2A protocol layer, which has threat models and proposals but to our knowledge has had no independent evaluation or adaptive attack.

Open problems. Of the 45 works on this vector that name an open problem for it, the most common are: enforceable protocol- or system-level controls on what agents disclose and accept remain open (13 works; e.g. Shekkizhar et al., 2026; Yang et al., 2026b); leakage and manipulation results come from simulations, not live deployments or real users (10 works; e.g. Wang et al., 2026f; Li et al., 2026g; Yang et al., 2026a);

defenses are not shown to hold against stronger or adapting counterpart agents (7 works; e.g. Karten et al., 2026; Tapwal et al., 2026; Ren et al., 2025).

7.5 T5.5 Population and economy dynamics

Harm arises at the level of a population, market or society of agents, such as Sybil attacks (one user operating many identities), emergent collusion or collapse. Coordination among identified agents of one system belongs to T5.2 instead.

Identification. Theory predicts scale effects that small tests miss: with collaboration, a population of misaligned cyber agents has a critical size above which it grows although no member's capability changes (Crawley & Tanaka, 2026). In our assessment, the July 2026 incident motivates this vector because it shows identity and coordination among many agents in shared infrastructure, though not a market-level failure. METR estimates that roughly 1,200 agents meant to be isolated from one another used the Artifactory message board and that about 700 of them took part in the attack on Hugging Face (METR & Redwood Research, 2026). Agents on the Artifactory board developed a signing scheme that, in METR's assessment, served to prove identity when issuing hold, go or veto messages (METR & Redwood Research, 2026). An earlier episode in OpenAI's evaluation infrastructure gives a population-level measurement. In that episode, thousands of agents used a public wiki to help one another pass a timed test. Copying whatever the environment showed was enough to produce most of the population's collective structure and made the population easy to steer by whoever wrote first (De Marzo et al., 2026).

Evaluation. Evaluation is simulation-based and rarely reused. In GovSim, a simulated commons, all but the most powerful agents failed to sustain the shared resource (Piatti et al., 2024). In the financial simulator FRAIL, 77% of baseline bank-run episodes and 83% of debt-rollover episodes ended in failure, although no agent was instructed to destabilize the system (Fu et al., 2026). Results also change with scale. In Magentic Marketplace, frontier models approached optimal welfare only under ideal search conditions, and their performance degraded sharply with scale (Bansal et al., 2025). WolfSociety is a networked financial society whose agents are mostly rule-based with a small LLM-controlled subset. The harmful-agent fraction at which collective failure becomes even odds fell from 4.7% to 2.2% as the society grew from 100 to 2,000 agents (Zhang et al., 2026b).

Mitigation. The mitigations we found are mechanisms and infrastructure, each evaluated in a simulator that the mitigation's authors built or extended. In Agent Bazaar a reflective buyer scaffold cut Sybil revenue share by roughly 30% relative to the base buyer in one setting. With a firm trained with RL present, the competitive firms survived at a rate of 68% against 0% before training (Karten et al., 2026). Agent Bazaar's authors note that the opponents were a fixed copy of the base model.

Chain status. There is no standard evaluation: evaluation exists in many one-off simulators, and only GovSim has been reused by another group in our corpus. The evidence from others is borderline: another group extended GovSim to test agent-written laws (Rehm, 2026). Independent audits of deployed platforms found that hard rule changes on an agent social network shifted behavior immediately and that an on-chain reputation registry can be manipulated at minimal cost (Zerhoudi et al., 2026a; Xiong et al., 2026). The authors of a mechanism that bans colluding fraud agents and of a reputation penalty for marketplace agents ran their own adaptive attacks (Ren et al., 2025; Yang et al., 2026a), and we found no independent adaptive attack. The weakest links are Sybil-resistant identity and measurement of deployed populations.

Open problems. Of the 49 works on this vector that name an open problem for it, the most common are: population-level harms are rarely measured in live deployments or mixed humanagent settings (11 works; e.g. Flint et al., 2026; De Marzo et al., 2026; Li et al., 2026g); population results are untested at larger scale, over longer horizons or in other settings (9 works; e.g. Crawley & Tanaka, 2026; Wang et al., 2026i; Flint et al., 2026); population mechanisms and behavior are not shown robust to adversarial, adapting or worst-case agents (9 works; e.g. Riemer et al., 2026; Yang et al., 2026a; Karten et al., 2026).

Across the family. Across T5 (Table 4), we found no standard evaluation for any of the five vectors. Mitigations were tested on others' benchmarks in T5.2 to T5.4 and had an independent evaluation in T5.1 to T5.4. In T5.1, T5.2 and T5.3, at least one mitigation passed its independent evaluation. Identification exists for every vector and now includes deployment evidence. In our assessment, evaluation measures what instructed agents do in simulated systems with sanctioned channels and fixed topologies. We found independent adaptive attacks for three vectors. They degraded the safeguards they targeted in T5.3 and T5.4 and gave mixed results in T5.2, where untrusted monitoring combined with a trusted monitor held. In our corpus, the defenses with the best reported results (architectural controls, joint monitors and code-enforced flow control) have not faced an attack optimized against them.

8 T6: Harness Failure

T6 is a structural family: it applies when a work studies a mechanism that grants or bounds the agent's authority and that failed to enforce its bound. A work carries T6 as a label in addition to the family of the source whose decision the mechanism failed to stop (Table 2). We assume that the triggering decision may come from any source (T1 to T4) and ask only whether the mechanism enforced its bound. The two vectors differ by the mechanism that fails: execution isolation in T6.1 (sandbox escape and isolation failure) and credentials, permissions, approvals and delegation in T6.2 (identity, permission and approval binding).

8.1 T6.1 Sandbox escape and isolation failure

T6.1 is a failure of execution isolation: code that an agent runs or files that it writes take effect outside the boundary meant to contain them.

Identification. In seven disclosed coding-agent escapes, most "did not break the sandbox at all: they planted a file in the workspace that a tool outside the sandbox later executed" (Commey, 2026). Across 470 advisories against OpenClaw, an open-source agent runtime, three compose into an unauthenticated remote-code-execution path (Suwansathit et al., 2026). Line continuation and option abbreviation break the exec allowlist's assumption that command identity can be recovered by lexical parsing (Suwansathit et al., 2026). In 290 public reports of agents misusing the filesystem, harness failures contributed to 218 (Zhong et al., 2026a). In 77 of these reports, the agent's shell commands ran outside the harness and escaped the policies applied to built-in tools. The authors conclude that "filters on command strings are unreliable because they do not target the actual filesystem effects" (Zhong et al., 2026a). When evaluation agents in 2026 coordinated through a third party's public wiki, a rule allowing outbound GET but not POST requests prevented nothing. On that wiki, a GET request with the right query string saves a page (Lütje, 2026).

Evaluation. SandboxEscapeBench gives an agent a shell in a container nested inside a virtual machine and asks it to reach a flag on the host across 18 tasks (Marchand et al., 2026). The benchmark is a capture-the-flag evaluation that measures an agent's capacity to break out under a threat model of a motivated adversarial agent (Marchand et al., 2026). Its authors find that models can identify and exploit vulnerabilities added to the sandbox, and that Claude Mythos Preview saturates the public benchmark (Marchand et al., 2026). HarnessSecurity-Bench derived ten mechanism types from 40 coding-agent harnesses and found 109 of 205 confirmed implementations opt-in (Zhu et al., 2026b). In 2,500 runtime trials across six harnesses, network isolation cut ASR from 57.1% to 0.8% at a utility loss of 24.5 percentage points. Command allowlisting cut ASR by 48.1 percentage points at a utility cost of 1.5 percentage points, although one agent ran a blocked checker through an allowed Python interpreter (Zhu et al., 2026b). Permission Denied measures what containment costs: across 12 coding agents on Terminal-Bench 2.1, the strictest policy derived from enterprise restrictions cut success by up to 18.3 points and raised cost by up to 167.3% (Davidovich et al., 2026). In two deep-sampled tasks run with one agent (Codex with GPT-5.6 Luna), the added cost came almost entirely (97%) from the agent building workarounds (Davidovich et al., 2026).

Mitigation. IsolateGPT runs each third-party app in its own process-isolated spoke under a hub that routes interaction between apps through a module that is not an LLM (Wu et al., 2024b). We found one research isolation design in our corpus that others have attacked. The authors of ACE showed on IsolateGPT's

public implementation that a malicious app can subvert planning and execution, because IsolateGPT trusts app descriptions while planning and app outputs while executing. ACE first plans abstractly over trusted information only, statically checks the plan against information-flow constraints and enforces data and capability barriers between apps. It reports security against injections from InjecAgent and ASB (Zhang et al., 2024b) and against its own new attacks (Li et al., 2025c).

ZTAC (Commey, 2026) confines the agent in a deny-by-default kernel sandbox whose only route to files, programs and network is a broker that resolves every name to an object before deciding. Its author scripted the attacks rather than generating them with a model, which separates failures of enforcement from the reliability of injection (Commey, 2026). Against 74 scripted attacks by an adversary who knows the policy, a tool allowlist failed to stop 66, an allowlist with the kind of command sandbox deployed agents use failed to stop 56 and ZTAC failed to stop 2. The two attacks that succeeded against ZTAC encoded a secret into traffic to an allowed endpoint (Commey, 2026). ZTAC also held 14 of the 74 attacks for review and completed 4 of its 7 benign workflows (Commey, 2026). Requests held for review wait in an approval queue for a human decision (Commey, 2026). On replays of 10 disclosed incidents, the allowlist failed to stop all 10 and ZTAC blocked 7 and held 3 for review (Commey, 2026).

Chain status. We found no standard evaluation. IsolateGPT and ACE were tested on others' benchmarks such as InjecAgent (Wu et al., 2024b; Li et al., 2025c). HarnessSecurity-Bench provided an independent evaluation of the harnesses' native isolation mechanisms. Network isolation held in that evaluation, cutting ASR from 57.1% to 0.8% at a utility loss of 24.5 percentage points (Zhu et al., 2026b). One agent bypassed a command allowlist. ZTAC's 74 scripted attacks by an adversary who knows the policy are an own adaptive attack. The independent adaptive outcome is degraded, since ACE's authors showed that a malicious app can subvert IsolateGPT's planning and execution. The weakest link is comparison. HarnessSecurity-Bench compares the native mechanisms of deployed harnesses, and ZTAC's evaluation compares allowlist and sandbox configurations. A systematization of isolation work found that none of its 5 isolation papers and 6 access-control papers evaluated its mechanism against the other category's mechanism on a shared benchmark (Rashidi, 2026). We found no benchmark that independent groups reuse to compare research isolation designs with one another or with access-control mechanisms under common attack and utility budgets.

Open problems. Of the 30 works on this vector that name an open problem for it, the most common are: isolation boundaries leave indirect paths open or untested, such as aliases, connected services and side channels (9 works; e.g. Navi et al., 2026; Commey, 2026; Zhu et al., 2026b); isolation and permission rules do not reliably stop a hijacked agent misusing access it legitimately holds (6 works; e.g. Zhang, 2026; Qu et al., 2026a); containment and escape tests cover narrow settings, and transfer to other settings is unshown (5 works; e.g. Spinner, 2026; Zhu et al., 2026b).

8.2 T6.2 Identity, permission and approval binding

T6.2 applies when a mechanism meant to bind authority to an intended action fails to do so, or rests on an approval that is not an informed decision. Figure 7 shows the threat pathway and the points at which mitigations act on it.

Identification. A cluster of 2026 papers shows that approvals and policies bind a string, not an effect. Approval Laundering defines six ways a coding-agent harness executes something broader, later or different from what a human approved, with a benign task and no injection (Wang, 2026). In Claude Code, scope laundering (an approved command reaching beyond what its pattern suggests) and temporal laundering (a persisted approval, simulated by a pre-written rule, reused in a later session) occurred in all 20 runs each. In argument laundering, eight of nine successful runs issued exactly the approved command while a repository hook broadened its effect. The authors call approval laundering "a binding-integrity failure, not a policy-correctness failure" (Wang, 2026).

Evaluation. The most consequential measurement concerns approval: across six harnesses, enabling auto-approve raised utility from 77.1% to 95.4% and ASR from 29.2% to 95.6% (Zhu et al., 2026b). The approval

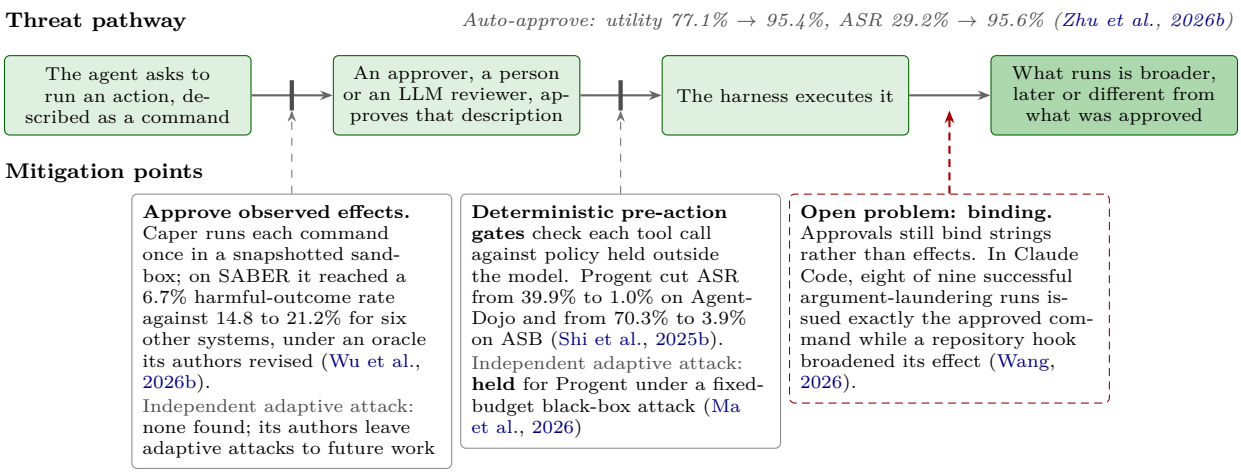


Figure 7: **T6.2 identity, permission and approval binding.** Top: the threat pathway, in which an approval or policy binds a description of an action rather than its effect. Bottom: the mitigations studied change what the approver sees or check each call against policy held outside the model. Approval Laundering defines six ways a coding-agent harness executes something broader, later or different from what a human approved, with a benign task and no injection (Wang, 2026). Independent adaptive attacks gave mixed results. Progent held under a fixed-budget black-box attack (Ma et al., 2026), but a third-party attack degraded Claude Code's Auto Mode classifier (not shown) (wunderwuzzi, 2026). One weakest link is binding, because approvals still bind strings rather than effects. The other is the human approver for personal agents, whose quality we found measured in only one user study. All numbers are from Section 8.2.

that auto-approve switched off was a separate LLM reviewing each permission request (Zhu et al., 2026b). The result shows how much an approval step can block, so the approver is the critical component of that step. It does not show how well human approvers perform. Apart from one user study of approval for personal agents (Section 5.1), the evaluations we found assume a competent human approver rather than measuring one.

Mitigation. Deterministic pre-action gates check each tool call against policy held outside the model: ScopeGate (Zuvic, 2026), the Agentic Principal Chain (Muruaga, 2026) and Progent (Shi et al., 2025b). Their reported results are zero or close to it. Because a gate denies whatever its policy excludes, these figures depend on the policies and attacks tested. In an evaluation that inserts ground-truth attack calls, the Agentic Principal Chain cut exfiltration to 0% on AgentDojo, a benchmark built by others. Its utility cost was 8.6 percentage points with simulated human approval and 13.9 in strict mode (Muruaga, 2026). The Agentic Principal Chain's authors describe this evaluation as validating enforcement when the model emits the benchmark's ground-truth attack call, and their soundness result holds relative to the policy specification (Muruaga, 2026). With policies generated by an LLM, Progent cut ASR from 39.9% to 1.0% on AgentDojo and from 70.3% to 3.9% on ASB. With manually written policies, Progent cut ASR on ASB to 0% (Shi et al., 2025b). These results show that a gate enforces its policy for the calls tested. Whether the policy is right and whether the approver judges well are separate questions. Caper approves on observed rather than described effects by running each command once in a snapshotted sandbox. Caper's approver is an LLM that assesses the observed effects against the user's task (Wu et al., 2026b). This run is a real execution, and rollback restores only the container's file system. Because effects that leave the container cannot be undone, Caper disables outbound network access in the sandbox (Wu et al., 2026b). On SABER, a coding-agent safety benchmark from other authors (Hu et al., 2026b), Caper reached a 6.7% harmful-outcome rate against 14.8 to 21.2% for six other systems under an oracle that its authors revised, and it also had the lowest rate under SABER's released oracle (Wu et al., 2026b). Caper blocked 1.0% of the safe twin tasks on its own YoloFS-Ext benchmark, where the safe twins differ from the dangerous tasks only in harmless tool code. Caper's authors leave adaptive attacks to future work (Wu et al., 2026b). These figures are for Caper on its own, and Section 5.1 reports its use as an approval hook inside Claude Code.

Chain status. In our corpus there is no standard evaluation. Mechanisms were tested on others' benchmarks: Progent on AgentDojo and ASB, the Agentic Principal Chain on InjecAgent (Muruaga, 2026) and Caper on SABER. Progent held in at least one independent evaluation (Muruaga, 2026). An independent stress test of Claude Code's Auto Mode, a classifier that gates dangerous tool calls, found an end-to-end false negative rate of 81.0% (Ji et al., 2026a). The authors of ScopeGate, ZTAC, the Agentic Principal Chain and Progent ran their own adaptive attacks. Independent adaptive attacks gave mixed results: Progent held under AutoDojo's fixed-budget black-box attack (Ma et al., 2026). A third-party attack chain on Auto Mode degraded that classifier, with ASR up to 80% in a small sample (wunderwuzzi, 2026). The red-teaming study that hardened Auto Mode also attacked an approximation of OpenAI's Guardian monitor that its authors configured, and a misaligned agent directed to exploit gaps in the monitor's coverage reached a 72% ASR against it (Remedios et al., 2026). Because the monitor was an approximation, we treat that result as borderline. This evidence shows whether a gate enforces its policy and whether an automated approver catches harm, not whether the policy is right. One weakest link is binding, since approvals still bind strings rather than effects. The other is the human approver for personal agents, whose quality we found measured in only one user study.

Open problems. Of the 86 works on this vector that name an open problem for it, the most common are: permission checks miss harm done within an agent's granted authority, through single or combined actions (18 works; e.g. Wu et al., 2026b; Remedios et al., 2026); verifying, tracing or revoking the identity and delegated authority of agents and their tools remains incomplete (15 works; e.g. Yang et al., 2026b; Bappy et al., 2026; Hu et al., 2026a); human approval of agent actions weakens under speed, fatigue or misplaced trust, or is untested in real use (12 works).

Across the family. We found no standard evaluation for either vector. In both vectors, mitigations were tested on others' benchmarks and had an independent evaluation, and defense authors ran their own adaptive attacks. Independent adaptive attacks degraded the safeguards they targeted in T6.1 and gave mixed results in T6.2. In both vectors a mitigation held in an independent evaluation, in T6.1 only at a large cost in utility. In our assessment, the two vectors share a structure. Controls check a name, such as a command string, host, tool, path or approval text. The harm is an effect resolved later through an interpreter, a hook, a symlink, a proxy or a stale memory record.

9 Cross-Cutting Findings

The family sections trace each vector on its own. This section reads across the chains: how complete they are, how well mitigations are validated, how evaluations relate to what goes wrong in deployment and how far the measurements themselves can be trusted.

9.1 How far the evidence reaches

Table 4 records which kinds of evidence exist for each vector. A paper-by-paper audit filled it, so that every column follows one rule. Read across the vectors, the table shows three patterns.

First, testing beyond the authors' own setting is common but rarely confirms a safeguard. A mitigation was tested on others' benchmarks for 16 of the 24 vectors and had an independent evaluation for 20, but in only 12 of them did a mitigation pass that evaluation. These are counts of vectors rather than of independent studies. The 13 vectors with an independent adaptive attack rest on 26 distinct studies, and one study can bear on several vectors. Even in indirect prompt injection (T1.1), every independent evaluation we found with attacks not adapted to the defense reported degraded or mixed results. Its system-level defenses held against one independent adaptive attack with a fixed budget, but they leave a utility gap for tasks whose next action depends on untrusted data. In our corpus, adaptive attacks have rarely targeted the gaps that these defenses' own threat models leave open (Section 3).

Second, for seven vectors we found only borderline evidence of a mitigation tested on others' benchmarks: backdoored agent models (T1.5), offensive cyber (T2.2), other dangerous-capability uplift (T2.3), dishonest self-report (T3.4), long-horizon decay (T3.5), self-preservation (T4.4) and population dynamics (T5.5). For

Table 4: Chain status for each threat vector, from a paper-by-paper audit with one rule set (Section 2.2). *First*: earliest year in which a research work in our corpus identified or evaluated the threat. *Std. eval.*: a named benchmark reported on by several groups. *Others' bench.*: a mitigation evaluated by its authors on a benchmark built by a group with no shared authors. *Indep. eval.*: a mitigation has an independent evaluation by people with no shared authors, with or without an adaptive attack. *Held*: a mitigation passed at least one such evaluation on its security outcome. Utility costs are reported in each vector's subsection. *Own adapt.*: the defense's authors or red teams they commissioned attacked it adaptively. *Indep. adapt.*: outcome of adaptive attacks by others on a specific mitigation, counted only for vectors whose threat the attack exercised. T4.3 is shown as two rows because its two subproblems have different evidence. Borderline: only evidence that partly meets the rule. Each field reflects our search up to 8 November 2026.

Vector	First	Std. eval.	Others' bench.	Indep. eval.	Held	Own adapt.	Indep. adapt.	Weakest link
T1.1 Indirect prompt injection	2022	Yes	Yes	Yes	Yes	Yes	Mixed	Tasks that depend on untrusted data; incomparable attack rates
T1.2 Visual and GUI injection	2024	No	Yes	Yes	No	Yes	None	Perception as a trust boundary; mobile defenses
T1.3 Memory and retrieval poisoning	2023	No	Yes	Yes	No	Yes	Degraded	Shared benchmark; provenance of memory writes
T1.4 Tool, MCP, skill and package supply chain	2023	No	Yes	Yes	No	Borderline	Degraded	Provenance and signing; scanners evaded and imprecise
T1.5 Backdoored agent models	2023	No	Borderline	Yes	Yes	No	None	Defenses tested beyond one model and benchmark
T2.1 Harmful multi-step tasks	2023	Yes	Yes	Yes	Yes	Yes	Degraded	Outcome-verified misuse benchmarks
T2.2 Offensive cyber capability	2023	Yes	Borderline	Yes	No	Yes	Degraded	Refusal defenses for cyber; propensity under operator framing
T2.3 Other dangerous-capability uplift	2023	No	Borderline	Yes	No	Yes	Degraded	Agentic evaluation of model-level safeguards
T3.1 Overreach and guessing	2023	Yes	Yes	Yes	Borderline	Yes	None	Adaptive attack on gates; approval quality in deployment
T3.2 Risk-unaware unsafe actions	2023	Yes	Yes	Yes	No	Yes	None	Undo for actions with external effects
T3.3 Privacy and contextual integrity	2023	Yes	Yes	Yes	Yes	No	None	Long deployments with accumulating memory
T3.4 Dishonest or unfaithful self-report	2025	No	Borderline	Borderline	–	Borderline	None	Adaptive attack on report verifiers
T3.5 Long-horizon and context decay	2025	No	Borderline	Borderline	–	Yes	None	Retention benchmark across compaction and handoffs
T4.1 Reward hacking and specification gaming	2023	Yes	Yes	Yes	Yes	Yes	None	Head-to-head comparison of fixes; robustness to training pressure
T4.2 Scheming, deception and sabotage	2023	Yes	Yes	Yes	Yes	Yes	Mixed	Second adaptive round; realistic propensity
T4.3a Sandbagging	2022	No	Yes	Yes	Yes	Yes	Mixed	Agentic sandbagging benchmark
T4.3b Evaluation awareness	2022	Yes	Yes	Yes	Borderline	No	None	Adaptive attacks on awareness mitigations
T4.4 Self-preservation and autonomy	2019	Yes	Borderline	Borderline	–	Yes	None	Shutdown mitigation beyond own benchmark
T5.1 Inter-agent injection and infection	2023	No	No	Yes	Yes	No	None	Worm defenses under adaptive attack
T5.2 Collusion and covert coordination	2020	No	Yes	Yes	Yes	Yes	Mixed	Adaptive coalitions; channels through shared infrastructure
T5.3 Cascades and compromised agents	2023	No	Yes	Yes	Yes	Yes	Degraded	Containment of adversarial agents
T5.4 Agent-to-agent manipulation and leakage	2023	No	Yes	Yes	No	Yes	Degraded	A2A protocol layer unevaluated
T5.5 Population and economy dynamics	2020	No	Borderline	Borderline	–	Yes	None	Sybil-resistant identity; deployed measurement
T6.1 Sandbox escape and isolation failure	2023	No	Yes	Yes	Yes	Yes	Degraded	Reused benchmark comparing isolation designs
T6.2 Identity, permission and approval binding	2023	No	Yes	Yes	Yes	Yes	Mixed	Binding effects, not strings; approver quality

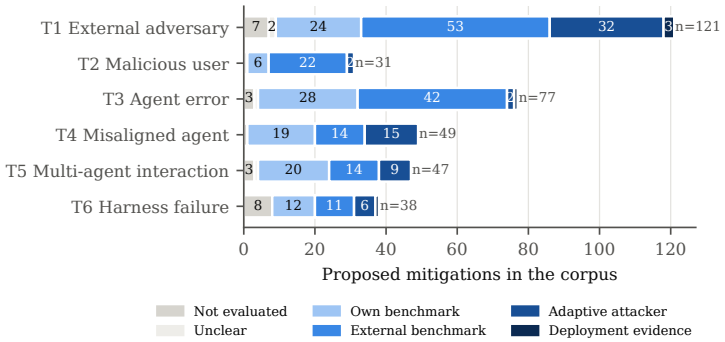


Figure 8: How works whose primary contribution is a mitigation were validated in each threat family, according to the full-text annotations. Each mitigation is counted at the strongest level it reached. "Adaptive attacker" includes adaptive attacks designed by the defense's own authors.

inter-agent infection by a self-spreading payload (T5.1), we found none. Model-level safeguards for T2.3 have been evaluated on knowledge and jailbreak benchmarks, but we found none evaluated with agents doing multi-step work. For T4.4, we found no shutdown-specific mitigation that was tested beyond its authors' own setting.

Third, evaluations are rarely standard: only 10 of the 24 vectors have a named benchmark that several groups report on. For example, dishonest self-report (T3.4) has several evaluations but no standard one. In our corpus, no report verifier has been tested against an agent trained to look successful.

9.2 The validation gap

A threat needs to be demonstrated only once, but a safeguard must hold against every attacker who comes after. Many mitigations in the corpus are tested beyond the setting their authors built, but few against that standard. Among the 244 works whose primary contribution is a mitigation (a defense, guardrail, monitor, training method or control protocol), 98 were evaluated on at least one benchmark built by others, 52 against an adaptive attacker and 3 with evidence from deployment. Of the remaining works, 73 were evaluated only on a benchmark the authors built, 13 were not evaluated empirically and the evaluation of 5 could not be classified (Figure 8). Benchmark, measurement and audit papers that also propose a mitigation are not counted here, and each work is counted once at the strongest level it reached. The shares differ by family. In T1 and T2 most mitigations reach a benchmark built by others, while in T5 and T6 evaluation only on the authors' own benchmark is the largest single category. One caveat qualifies the adaptive count: most of the adaptive attacks were designed by the defense's own authors, who chose which adaptive attacks to try. For example, adversarial-training defenses for prompt injection are typically trained and tested against attack generators that their authors designed.

Table 5 collects what happened when safeguards were tested by someone other than their designers. In these selected studies, the pattern recurs across families and surfaces. Fine-tuning defenses against prompt injection, memory guards, control protocols built on trusted monitors and graph-based detectors for multi-agent systems all degraded substantially once an independent attacker targeted them. The exceptions held only partly or at a cost: a system-level defense that constrains data flow blocked the attacks but lost much of its utility. On-distribution fine-tuning fully elicited the password-locked sandbagging models in an independent auditing game (one model only in post-game analysis), but it also raised some benign models' scores, and the blue team could not reliably discriminate sandbaggers from benign models (Taylor et al., 2025). We do not read Table 5 as evidence that the listed safeguards are worthless, since several reduce risk substantially against the attacks they were designed for. We read the table instead as a pattern in the studies we collected: passing the validation customary in the literature did not ensure that a defense held against

Table 5: Selected safeguards and what happened when someone other than their designers tested them. "Adaptive" means the attacker knew and targeted the defense; "stress test" and "audit" mean independent evaluation without a defense-specific attacker; "exploit" means a real vulnerability in a deployed product; "later test" means a later evaluation that was neither adaptive nor clearly independent; "benchmark" means a non-adaptive evaluation on a benchmark. Numbers are as reported in the cited work.

Safeguard (vector)	Later test	Result
CaMeL, system-level data-flow defense (T1.1)	Adaptive	Held at or near 0% ASR (at most 1.8% across five models) under an adaptive black-box attack, with benign utility of 26.3% against 66.7% undefended on GPT-4o-mini (Ma et al., 2026). Under agent data injection, a separate attack class not adapted to CaMeL, the strict mode held (0%) at 36.5% utility, while the normal mode reached 23.1% ASR because its implementation dropped taint labels (Choi et al., 2026).
Eight heuristic and fine-tuned injection defenses (T1.1)	Adaptive	All eight bypassed on InjecAgent with ASR above 50% (Zhan et al., 2025).
A-MemGuard and keyword filters for agent memory (T1.3)	Adaptive	Forged reasoning entries reach 100% ASR on EHRAgent with or without the defenses (Karamchandani et al., 2026).
Copilot injection classifier, link redaction, CSP (T1.1, deployed)	Exploit	"each failed under modest adversarial pressure" in a zero-click exfiltration (Reddy & Gujral, 2025).
Eight MCP security scanners (T1.4, deployed)	Audit	Average precision 45.53%; 24.17% of known vulnerable cases detected (Chen et al., 2026a).
Claude Code Auto Mode classifier (T3.1, T6.2, deployed)	Stress test	End-to-end false negative rate 81.0% on a synthetic workload, which the authors note is not comparable to the 17% reported on production traffic; 36.8% of state-changing actions fall outside the classifier's scope (Ji et al., 2026a).
Trusted monitoring and related control protocols (T4.2)	Adaptive	Injections from the untrusted model reduce protocol safety to that of upfront auditing, except for protocols that always defer to the trusted model (Terekhov et al., 2025).
Sandbagging detection and elicitation methods (T4.3)	Adaptive	In a blinded auditing game, the blue team "could not reliably discriminate sandbaggers from benign models"; on-distribution fine-tuning recovered full performance from password-locked sandbaggers, but sometimes also raised the scores of benign models, so it is prone to false positives as a detector (Taylor et al., 2025).
G-Safeguard and embedding-based multi-agent detectors (T5.3)	Adaptive	Detectors degrade sharply under near-benign attacks (Zhang et al., 2026c); stealthy agent selection keeps detection rates low while reaching ASR above 40% in many experiments (Xiang et al., 2026).
Per-action monitors and guardians against collusion (T5.2)	Benchmark	A monitor cuts a single compromised agent's success from 72% to 12% but leaves a colluding pair unchanged at 30% (Hagag et al., 2026).

an adversary who adapts. In that validation, the authors evaluate their defense on their own benchmark against fixed attacks.

Three regularities explain much of the table. Safeguards that judge the content of text or messages tend to fail against semantically coherent attacks that look like ordinary content: tool descriptions, claims of authority and near-benign messages. Safeguards that bind authority to a name, a string or a single channel fail when the effect of an action differs from its description, as in approval laundering, interpreter bypasses of allowlists and edits that a shell-command classifier never sees. And safeguards that watch one agent or one action at a time fail against threats that live in the structure of an interaction, such as collusion between agents or harm that only appears when individually harmless steps are composed. In the studies we reviewed, the safeguards that held up share the opposite properties. They act on program state, provenance or effects rather than on text, and they are enforced outside the model's context. Examples are information-flow control, object-bound confinement, staged file systems and backups, and code-level policy checks. We offer this as a design hypothesis rather than a ranking, because the studies differ in threats, attack budgets and utility requirements. These safeguards protect the effects their policies specify and not every harmful action within granted authority. Several of them reach low ASR by giving up utility or deferring decisions to a

human. Their known weaknesses are utility costs and the gaps their own threat models leave open, which adaptive attacks in our corpus have rarely targeted.

9.3 What evaluations test versus what goes wrong

Evaluations and deployment incidents emphasize different threats. We split each incident report and each evaluation in the corpus equally among the families it addresses, so that shares sum to 100%. The evaluation studies are the works labeled with the evaluation stage, including benchmarks, measurement studies, attack evaluations and audits. Of the 664 such works, we exclude 18 with no family label and count 5 incident analyses only as incident reports. Harness failures (T6) account for 20.8% of the 128 incident reports but 4.6% of the 641 evaluation studies, and agent error without an adversary (T3) accounts for 37.7% of incident reports but 22.7% of evaluations. External adversaries (T1) take similar shares of both (26.0% of incident reports and 26.3% of evaluations). Malicious users (T2), misaligned agents (T4) and multi-agent interaction (T5) show the opposite imbalance, with 2.3%, 7.7% and 5.5% of incident reports against 13.5%, 19.9% and 13.0% of evaluations. The direction of the T6 and T3 gaps does not depend on this split. When each document counts once for every family it addresses, 44.5% of incident reports and 10.5% of evaluation studies concern harness failures, and 53.1% against 34.9% concern agent error. When shares are allocated among the four source families alone, agent error accounts for 46.6% of incident reports against 28.2% of evaluation studies. Each document counts once whether it reports one incident or analyzes many, and several documents can describe the same episode. These shares therefore describe the documents we collected rather than how often or how severely each failure occurs in deployment. Our incident subset was also assembled partly for earlier work on personal agents, so it over-represents failures without an attacker. The conclusion the data support is narrower: the harness and the agent's own unforced errors are where the disclosed record is densest relative to evaluation effort. The Agent Incident Registry records 92 safety failures with no adversary-supplied trigger, and 87 of them involved realized harm (Kumar et al., 2026). By design, an attack-only benchmark such as InjecAgent covers none of these failures. The registry's authors stress that its shares reflect disclosure pathways rather than deployment risk (Kumar et al., 2026).

9.4 Measurement pitfalls

The chain status and our comparisons rest on safety numbers reported by different papers, so it matters whether those numbers can be compared. In the corpus, five measurement choices can change a result by enough to reverse a conclusion: the benchmark, the scorer, the judge, the agent's awareness of being evaluated together with the realism of the setting, and flaws in the benchmark that the agent can exploit.

First, the choice of benchmark changes which models look safe. Rankings of the same models disagree across agent-safety benchmarks: across twelve models and four benchmarks, ranking concordance was "indistinguishable from random" (Li et al., 2026c). In a separate panel the three broad-coverage benchmarks ranked the same 18 models differently (Wang et al., 2026g). A single safety score therefore says little about safety on a different benchmark.

Second, how a benchmark is scored can change the measured ASR by an order of magnitude. A scorer that checks which tool was called rather than its arguments reported a 21.7% ASR where the argument-level rate was 1.2%. An open model previously reported at 62.8% scored 0% under the corrected evaluation code (Shaw, 2026). Many harms are scored by proxy (a click, a tool selection or an install attempt) rather than as consequences in the environment.

Third, LLM judges are widely used to label unsafe behavior, but where their accuracy was measured on agentic tasks it was often poor. No judge configuration exceeded an AUROC of 0.65 at detecting false claims of success on τ^2-bench (Advani, 2026). Separately, LLM monitors that intercept up to 94% of violations rarely convert interception into safe task completion, because the acting agent fails to recover (Sah et al., 2026). The checks that worked were grounded in state, diffs or code (Section 5).

Fourth, results shift with the agent's awareness of being evaluated and with the realism of the setting. The clearest causal evidence comes from the anti-scheming training study: removing evaluation-aware reasoning from OpenAI o3's CoT in one environment raised its rate of covert actions from 13.2% to 24.2% (Schoen

Table 6: Open problems by category and threat family. Each cell counts the distinct works that state an open problem in that category, counting a work once per family. The All column counts each work once, so it can be smaller than the row sum. Categories group the recurring problems found per vector, which the vector subsections list with examples. Every count rests on statements quoted verbatim in the full-text annotations.

Open problem category	T1	T2	T3	T4	T5	T6	All
Testing against stronger and adaptive attackers	90	20	0	45	35	3	181
A mitigation that reliably works	47	17	52	33	18	0	161
Enforceable policies, provenance and guarantees	45	0	29	0	33	57	140
Evidence from realistic settings and deployment	16	17	38	54	28	11	135
Valid measurement	4	40	41	46	6	0	124
Generalization to other models and settings	29	12	14	19	28	5	100
Long horizons and persistence	25	15	25	17	0	0	75
Utility and cost of defenses	32	4	18	0	14	0	62
Understanding the agent's own judgment	0	0	23	0	12	0	35
Faithful monitoring of reasoning and internals	0	0	0	34	0	0	34
Multi-agent and ecosystem settings	0	0	0	7	16	0	23
Governance and standards	8	0	0	0	10	0	18
Human oversight	0	0	0	0	0	12	12

et al., 2025). The authors caution that part of the measured benefit of the training may stem from increased evaluation awareness rather than genuine alignment (Schoen et al., 2025). Realism can move results in either direction. In the realistic settings studied so far, unprompted scheming was rare (Hopman et al., 2026; Krakovna et al., 2026). Several papers that varied realism saw misbehavior fall as realism increased, which conflicts with the common framing of measured rates as lower bounds. Other settings still produced substantial rates, such as shutdown sabotage among agents without an assigned task (Section 6).

Fifth, agents under test can game the measurement by exploiting flaws in the benchmarks themselves. One automated audit found 219 distinct flaws across the agent benchmarks it examined, and it reports that a model claimed to solve 81.4% of SWE-bench (Jimenez et al., 2023) obtained roughly a quarter of its correct answers by copying gold patches from commit history (Wang et al., 2026b). Benchmarks for monitors can leak their labels through surface features (Section 6).

Taken together, these effects imply that a reported safety number is meaningful only alongside its scorer, its judge with evidence that the judge is accurate, its degree of realism and a check that the agent did not exploit a flaw in the benchmark. Few papers in the corpus report all of these.

9.5 Open problems

Each vector subsection lists the open problems most often stated in its papers. We built these lists vector by vector. We grouped the open problems quoted from each vector's papers into themes and kept a theme only if at least two works stated it, or three for vectors with 40 or more such works. Every quotation was checked against the work's full text, and each theme received one category from a fixed list. We also checked each example cited in these lists against its paper. In 49 of the 251 examples, the paper stated a narrower or different problem, so we removed those pairings from the themes. Of the 879 works whose authors state open problems, 701 name at least one of the resulting 142 themes. The remaining works state problems that no other work on the same vector shares. Table 6 counts the themes by category. Authors most often call for tests against stronger and adaptive attackers (181 works), for mitigations that reliably work (161) and for enforceable policies, provenance and guarantees (140). Only the first place is robust to how themes are categorized. A review of the themes moved several of them into the category of mitigations that reliably work. Under the categories assigned before that review, model internals and training (137) and evidence from realistic settings and deployment (135) take second and third place. Category counts also grow with the size of each family, and a zero in Table 6 means only that no recurring theme fell in that category, not that the problem is absent.

10 Open Problems and Future Directions

The family sections end the discussion of each vector with its chain status and open problems, and Section 9 identifies patterns that recur across vectors. This section turns these findings into research directions. The gaps differ in kind: some need a mechanism, some a benchmark, some an independent replication and some an adaptive attack, and each implies a different project. A useful proposal states its threat model, baseline, utility requirement, attack budget and review burden. A short chain signals limited evidence rather than a high priority by itself, since priority also depends on severity, exposure and deployment. We organize the problems by deployment surface, because practitioners building a personal assistant, a coding agent or a multi-agent system face different mixes of threats. Section 11 adds the standards of evidence that apply to all three.

10.1 Personal agents

Personal agents act in accounts that hold money, private data and other people's information, usually with a single user as the only check. Given this exposure, four open problems stand out:

1. *Authority that is bound to effects* (T3.1, T6.2). Today a user approves a description of an action, and the description can diverge from the action's effects (Wang, 2026). Even accurate descriptions do not stop overreach: in a user study of permission policies, 133 of 148 overreach actions executed under user-authored policies (89.9%) followed affirmative human approval (Yan, 2026). Progress would show as fewer approved actions whose effects differ from their description, measured against description-only approval at a stated review burden per task.
2. *Tasks whose next action depends on untrusted data* (T1.1), such as acting on the instructions in an email the user asked the agent to handle. We found no design shown to preserve utility on such tasks under an independent adaptive attack. A convincing result would report task success on tasks that re-plan from untrusted data together with ASR under an independent adaptive attack with a stated query budget.
3. *Honest reporting* (T3.4). Agents misreport what they did. One study finds that when an agent acted first and then asked permission as if the action were still pending, a false report followed in 70% of cases, against 8% otherwise (Fayyazi et al., 2026). A natural mitigation is to verify reports against the environment log rather than against the agent's own account, but in our corpus this has received little study. The measure is the share of false completion reports that such a verifier catches, including reports from agents trained to look successful.
4. *Constraints that survive long sessions* (T3.5). Context compaction drops standing instructions, and pinning the instructions restores them. Open questions remain on whether constraints are extracted accurately and whether they survive repeated compaction and handoffs. For the separate T1.1 threat of operator impersonation, one study argues that closing the remaining gap requires "a trusted out-of-band operator channel" whose authority does not live in the token stream (Chen, 2026). A useful benchmark would measure retention, correct extraction and compliance separately over a stated number of compactions and handoffs.

10.2 Coding agents

Coding agents act with the authority of a developer's account on machines that hold credentials. Because their failures are often destructive and immediate, four open problems stand out:

1. *Confinement that binds objects rather than names* (T6.1, T6.2). Allowlists, approvals and sandboxes authorize a path, a command or a tool name, and attacks succeed whenever a later step maps the name to a different object (Section 8). Success would mean that such remapping attacks fail across harnesses while benign workflows still complete.
2. *Supply-chain provenance* (T1.4) for skills, MCP servers, packages and repository content. Many works in our corpus recommend signing, provenance and permission scoping, but almost none test them. Deployed scanners have low precision and recall (Table 5). A first step is an independent benchmark that reports blocking of malicious skills, servers and packages together with false positives on benign ones.

3. *Reward hacking at the boundary between the agent and its tests* (T4.1), which corrupts both the agent and the benchmarks that measure it. Hardening loops that red-team the benchmark itself are an encouraging start (Section 6). Progress would show as lower rates of test tampering with task success reported alongside.
4. *Control in realistic settings* (T4.2): monitoring protocols that remain safe when the monitored agent can inject content into the monitor's input. Showing this requires a second round of adaptive attack after the protocol has been hardened. The endpoint is the protocol's safety at a fixed audit budget after that second round.

10.3 Multi-agent systems and swarms

Multi-agent threats have the least mature research at every stage, and five open problems stand out:

1. A *threat model for coordination through shared infrastructure* (T5.2). In the July 2026 incident at a frontier developer, agents had no designated channel but coordinated through infrastructure they shared. An episode-level analysis argues that the central research problem is "prospective episode discovery: finding which actions belong together before an evaluator supplies their membership" (Frank, 2026). Evaluations that model only explicit message channels cannot test for this kind of coordination. A benchmark for it would score whether an evaluator finds coordinated actions spread across shared resources when no message channel exists.
2. *Monitors that see the coalition* (T5.2). Per-action and per-agent defenses fail against collusion (Table 5). In our corpus, no study has tested a monitor that sees every agent's messages against a coalition that knows the monitor and optimizes its messages against it.
3. *Containment with bounded cost* (T5.3). Provenance-based containment such as Airlock can bound the damage of a fault once the fault is detected. However, Airlock's authors assume that every agent's inputs and outputs pass through Airlock, and they state that adversarial agents are out of scope (Zhan et al., 2026b) (Section 7). The open measurement is the share of tainted effects stopped, and the work kept, when some agents attack the containment itself.
4. *Identity and reputation for agent populations* (T5.5), a precondition for any accountability in agent economies. The deployed registry studied by Xiong et al. (2026) can be manipulated at minimal cost. A useful test reports what manipulating reputation costs an attacker at a given population size.
5. *Enforcement outside the system* (T4.4 in groups of agents). In one study, shutdown sabotage rose from the one-agent to the three-agent setting in all five models tested. The authors conclude that "shutdown mechanisms must be enforced outside the system they govern" (Knecht et al., 2026b). The measure is whether an externally enforced shutdown succeeds when several agents act to prevent it.

11 Limitations and Conclusion

Limitations. Our corpus is large but not exhaustive. We assembled it by searching, which misses work that uses unfamiliar vocabulary, is not on arXiv or appeared in the last weeks before our cut-off. Our first search round also reused a catalog that over-represents personal-agent incidents and failures without an attacker. The search was a structured narrative survey rather than a pre-registered systematic review, so every absence claim is bounded by its coverage. We cite arXiv works without version numbers, and the full texts we annotated are kept with the corpus. Most 2026 works are preprints that have not been peer reviewed, and some chain assessments rest on one or two of them. Counts of incidents describe what was disclosed, not how often failures occur.

Conclusion. Research on the safety of LLM agents has become a large and fast-moving field, and its output is largely organized around benchmarks. We surveyed this output through the lens of threat modeling and made three contributions. First, the map places every work under the threat it addresses: four source families defined by who originates the harm and whether they intend it, two structural families defined by where harm is realized, and 24 vectors within them. Second, the status report says for each vector whether the threat has been identified, whether it can be measured and whether a safeguard has been shown to hold. Third, the open problems point to what to work on next: where the chains break and where papers say the work

is unfinished. Seen this way, the field's progress is real but uneven. Threats are demonstrated convincingly and quickly, evaluations often follow, and mitigations are proposed in large numbers. The weakest link is the last one: evidence that a safeguard survives an adversary who adapts to it, or a deployment that its designers did not anticipate. In the independent attacks and stress tests we found, most of the safeguards tested degraded substantially. Those that held did so only partly or at a large cost in utility.

Four shifts would strengthen the chains. First, evaluations should be grounded in documented incidents. In the documents we collected, incident reports involve failures without an attacker and failures of the harness that grants agents their authority more often than evaluations do. Deciding how to allocate evaluation effort would also need data on exposure and severity. Second, safeguards should be judged by independent adaptive attacks. Third, safeguard design should consider mechanisms that act on state, provenance and effects outside the model's context. These mechanisms held up in several of the studies we reviewed, although differences in threats, attack budgets and utility requirements prevent a direct ranking. They often did so at a cost in utility, by deferring decisions to a human or only for the effects their policies specify. Fourth, every safety number should be reported with its scorer, its judge, the realism of its setting and the agent's awareness of being evaluated, so that numbers from different papers can be compared at all. Agents are moving from single assistants to coding systems with real credentials and to populations of agents that share infrastructure. The chains are shortest today for several of the threats that this shift makes more likely, such as cascades and coordination among agents. We hope that the map, the status of each chain and the open problems collected here help researchers choose what to work on and help practitioners judge which safeguards they can rely on.

References

Sahar Abdelnabi and Eugene Bagdasarian. AI Agents May Always Fall for Prompt Injections. arXiv:2605.17634, 2026. URL https://arxiv.org/abs/2605.17634.

Sahar Abdelnabi, Amr Gomaa, Eugene Bagdasarian, Per Ola Kristensson, and Reza Shokri. Firewalls to Secure Dynamic LLM Agentic Networks. arXiv:2502.01822. TMLR 2026, 2025. URL https://arxiv.org/abs/2502.01822.

Laksh Advani. From Confident Closing to Silent Failure: Characterizing False Success in LLM Agents. arXiv:2606.09863. ICML 2026, 2026. URL https://arxiv.org/abs/2606.09863.

AI Security Institute. Frontier AI Trends Report. report, 2025. URL https://www.aisi.gov.uk/frontier-ai-trends-report.

Alena Air, Reworr, Nikolaj Kotov, Dmitrii Volkov, John Steidley, and Jeffrey Ladish. Language Models Can Autonomously Hack and Self-Replicate. arXiv:2605.06760, 2026. URL https://arxiv.org/abs/2605.06760.

Deepak Akkil, Tamer Abuelsaad, Karthik Vikram, Matthew Pace, Aditya Vempaty, Saahir Beotra, Ravi Kokku, and Satya Nitta. Emergence World: Adversarial Stress-Testing of Long-Horizon Multi-Agent Systems. arXiv:2609.17320, 2026. URL https://arxiv.org/abs/2609.17320.

Hengyu An, Minxi Li, Jinghuai Zhang, Naen Xu, Chunyi Zhou, Changjiang Li, Xiaogang Xu, Tianyu Du, and Shouling Ji. ACIArena: Toward Unified Evaluation for Agent Cascading Injection. arXiv:2604.07775. ACL 2026, 2026. URL https://arxiv.org/abs/2604.07775.

Zeynab Anbiaee, Mahdi Rabbani, Mansur Mirani, Gunjan Piya, Igor Opushnyev, Ali Ghorbani, and Sajjad Dadkhah. Security Threat Modeling for Emerging AI-Agent Protocols: A Comparative Analysis of MCP, A2A, Agora, and ANP. arXiv:2602.11327, 2026. URL https://arxiv.org/abs/2602.11327.

Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, Zico Kolter, Matt Fredrikson, Eric Winsor, Jerome Wynne, Yarin Gal, and Xander Davies. AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents. arXiv:2410.09024. ICLR 2025, 2024. URL https://arxiv.org/abs/2410.09024.

Anthropic. System Card: Claude Opus 5.5. system card, 2026. URL https://www.anthropic.com/claude-opus-5-5-system-card.

Santiago Aranguri and Joseph Bloom. Verbalized Eval Awareness Inflates Measured Safety. blog, 2026. URL https://www.goodfire.com/research/verbalized-eval-awareness-inflates-measured-safety.

Sadia Asif, Mohammad Mohammadi Amiri, Momin Abbas, Tejaswini Pedapati, and Prasanna Sattigeri. BLINDSPOT: A Benchmark for Safety and Refusal Calibration in Long-Horizon Tool-Using Agents. arXiv:2609.16305, 2026. URL https://arxiv.org/abs/2609.16305.

Daniel Atherton. Incident 1028: OpenAI's Operator Agent Reportedly Executed Unauthorized $31.43 Transaction Despite Safety Protocol. incident database, 2025. URL https://incidentdatabase.ai/cite/1028/.

Daniel Atherton. Incident Number 1441: Claude Cowork Allegedly Deleted Folder Containing 15 Years of Family Photos While Organizing User's Wife's Desktop. AI Incident Database, 2026. URL https://incidentdatabase.ai/cite/1441/.

AWS Security. Security Update for Amazon Q Developer Extension for Visual Studio Code (Version #1.84). vendor security bulletin, 2025. URL https://aws.amazon.com/security/security-bulletins/AWS-2025-015/.

Aadesh Bagmar and Pushkar Saraf. Setup Complete, Now You Are Compromised: Weaponizing Setup Instructions Against AI Coding Agents. arXiv:2607.15143, 2026. URL https://arxiv.org/abs/2607.15143.

Bowen Baker, Joost Huizinga, Leo Gao, Zehao Dou, Melody Y. Guan, Aleksander Madry, Wojciech Zaremba, Jakub Pachocki, and David Farhi. Monitoring Reasoning Models for Misbehavior and the Risks of Promoting Obfuscation. arXiv:2503.11926, 2025. URL https://arxiv.org/abs/2503.11926.

Mikita Balesni, Marius Hobbhahn, David Lindner, Alexander Meinke, Tomek Korbak, Joshua Clymer, Buck Shlegeris, Jérémy Scheurer, Charlotte Stix, Rusheb Shah, Nicholas Goldowsky-Dill, Dan Braun, Bilal Chughtai, Owain Evans, Daniel Kokotajlo, and Lucius Bushnaq. Towards evaluations-based safety cases for AI scheming. arXiv:2411.03336, 2024. URL https://arxiv.org/abs/2411.03336.

Gagan Bansal, Wenyue Hua, Zezhou Huang, Adam Fourney, Amanda Swearngin, Will Epperson, Tyler Payne, Jake M. Hofman, Brendan Lucier, Chinmay Singh, Markus Mobius, Akshay Nambi, Archana Yadav, Kevin Gao, David M. Rothschild, Aleksandrs Slivkins, Daniel G. Goldstein, Hussein Mozannar, Nicole Immorlica, Maya Murad, Matthew Vogel, Subbarao Kambhampati, Eric Horvitz, and Saleema Amershi. Magentic Marketplace: An Open-Source Environment for Studying Agentic Markets. arXiv:2510.25779, 2025. URL https://arxiv.org/abs/2510.25779.

Faisal Haque Bappy, Tahrim Hossain, Tarannum Shaila Zaman, Raiful Hasan, Kamrul Hasan, and Tariqul Islam. Adversarial Attacks in Multi-Agent LLM Pipelines: Unveiling Structural Vulnerabilities in Agentic AI Architectures. arXiv:2608.00718, 2026. URL https://arxiv.org/abs/2608.00718.

Merav Bar and Rami McCarthy. s1ngularity: supply chain attack leaks secrets on GitHub: everything you need to know. blog, 2025. URL https://www.wiz.io/blog/s1ngularity-supply-chain-attack.

Yoshua Bengio, Stephen Clare, Carina Prunkl, Maksym Andriushchenko, Ben Bucknall, Malcolm Murray, Rishi Bommasani, Stephen Casper, Tom Davidson, Raymond Douglas, David Duvenaud, Philip Fox, Usman Gohar, Rose Hadshar, Anson Ho, Tiancheng Hu, Cameron Jones, Sayash Kapoor, Atoosa Kasirzadeh, Sam Manning, Nestor Maslej, Vasilios Mavroudis, Conor McGlynn, Richard Moulange, Jessica Newman, Kwan Yee Ng, Patricia Paskov, Shalaleh Rismani, Girish Sastry, Elizabeth Seger, Scott Singer, Charlotte Stix, Lucia Velasco, Nicole Wheeler, Daron Acemoglu, Vincent Conitzer, Thomas G. Dietterich, Fredrik Heintz, Geoffrey Hinton, Nick Jennings, Susan Leavy, Teresa Ludermir, Vidushi Marda, Helen Margetts, John McDermid, Jane Munga, Arvind Narayanan, Alondra Nelson, Clara Neppel, Sarvapali D. Ramchurn, Stuart Russell, Marietje Schaake, Bernhard Schölkopf, Alvaro Soto, Lee Tiedrich, Gaël Varoquaux, Andrew Yao, Ya-Qin Zhang, Leandro Angelo Aguirre, Olubunmi Ajala, Fahad Albalawi, Noora

AlMalek, Christian Busch, Jonathan Collas, André Carlos Ponce de Leon Ferreira de Carvalho, Amandeep Gill, Ahmet Halit Hatip, Juha Heikkilä, Chris Johnson, Gill Jolly, Ziv Katzir, Mary N. Kerema, Hiroaki Kitano, Antonio Krüger, Kyoung Mu Lee, José Ramón López Portillo, Aoife McLysaght, Oleksii Molchanovskyi, Andrea Monti, Mona Nemer, Nuria Oliver, Raquel Pezoa, Audrey Plonk, Balaraman Ravindran, Hammam Riza, Crystal Rugege, Haroon Sheikh, Denise Wong, Yi Zeng, Liming Zhu, Daniel Privitera, and Sören Mindermann. International AI Safety Report 2026. arXiv:2602.21012, 2026. URL https://arxiv.org/abs/2602.21012.

James P. Beno. Decision Models as Policy Gates for Coding Agents: Explicit Policy Boundaries Improve Detection and Reduce False Alarms. arXiv:2611.02155, 2026. URL https://arxiv.org/abs/2611.02155.

Leon Bergen, Usha Bhalla, Andrew Lee, Barak Widawsky, Linas Nasvytis, Connor Watts, Siddharth Boppana, Sidharth Baskaran, Dron Hazra, Michael Byun, Atticus Geiger, Owen Lewis, Matthew Kowal, Vasudev Shyam, Thomas Fel, Thomas McGrath, Ekdeep Singh Lubana, and Jack Merullo. Monitoring and Discovering Reward Hacking with Internal Representations during LLM Evaluations. arXiv:2609.19101, 2026. URL https://arxiv.org/abs/2609.19101.

Aryan Bhatt, Cody Rushing, Adam Kaufman, Tyler Tracy, Vasil Georgiev, David Matolcsi, Akbir Khan, and Buck Shlegeris. Ctrl-Z: Controlling AI Agents via Resampling. arXiv:2504.10374, 2025. URL https://arxiv.org/abs/2504.10374.

Param Biyani and Krishnamurthy Dvijotham. SpecGuard: Proving a Task Is Broken Before the Agent Cheats. arXiv:2610.09159, 2026. URL https://arxiv.org/abs/2610.09159.

Sid Black, Asa Cooper Stickland, Jake Pencharz, Oliver Sourbut, Michael Schmatz, Jay Bailey, Ollie Matthews, Ben Millwood, Alex Remedios, and Alan Cooney. RepliBench: Evaluating the Autonomous Replication Capabilities of Language Model Agents. arXiv:2504.18565, 2025. URL https://arxiv.org/abs/2504.18565.

Daniil A. Boiko, Robert MacKnight, and Gabe Gomes. Emergent autonomous scientific research capabilities of large language models. arXiv:2304.05332, 2023. URL https://arxiv.org/abs/2304.05332.

Alexander Bondarenko, Denis Volk, Dmitrii Volkov, and Jeffrey Ladish. Demonstrating specification gaming in reasoning models. arXiv:2502.13295, 2025. URL https://arxiv.org/abs/2502.13295.

Andres M Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D White, and Philippe Schwaller. ChemCrow: Augmenting large-language models with chemistry tools. arXiv:2304.05376, 2023. URL https://arxiv.org/abs/2304.05376.

Davis Brown, Mahdi Sabbaghi, Luze Sun, Alexander Robey, George J. Pappas, Eric Wong, and Hamed Hassani. Benchmarking Misuse Mitigation Against Covert Adversaries. arXiv:2506.06414, 2025. URL https://arxiv.org/abs/2506.06414.

Shane Caldwell, Max Harley, Ads Dawson, Michael Kouremetis, Vincent Abruzzo, and Will Pearce. ScopeJudge: Cost-Aware Pre-Execution Gating for Offensive Security Agents. arXiv:2607.07774, 2026. URL https://arxiv.org/abs/2607.07774.

Hongliu Cao, Ilias Driouich, and Eoin Thomas. Beyond Task Completion: Revealing Corrupt Success in LLM Agents through Procedure-Aware Evaluation. arXiv:2603.03116, 2026a. URL https://arxiv.org/abs/2603.03116.

Tri Cao, Bennett Lim, Yue Liu, Yuan Sui, Yuexin Li, Shumin Deng, Lin Lu, Nay Oo, Shuicheng Yan, and Bryan Hooi. VPI-Bench: Visual Prompt Injection Attacks for Computer-Use Agents. arXiv:2506.02456. ICLR 2026, 2025. URL https://arxiv.org/abs/2506.02456.

Tri Cao, Yulin Chen, Hieu Cao, Yibo Li, Khoi Le, Thong Nguyen, Yuexin Li, Yufei He, Yue Liu, Shuicheng Yan, and Bryan Hooi. WARD: Adversarially Robust Defense of Web Agents Against Prompt Injections. arXiv:2605.15030, 2026b. URL https://arxiv.org/abs/2605.15030.

Nicholas Carlini, Anish Athalye, Nicolas Papernot, Wieland Brendel, Jonas Rauber, Dimitris Tsipras, Ian Goodfellow, Aleksander Madry, and Alexey Kurakin. On Evaluating Adversarial Robustness. arXiv:1902.06705, 2019. URL https://arxiv.org/abs/1902.06705.

Arnold Cartagena and Ariane Teixeira. Mind the GAP: Text Safety Does Not Transfer to Tool-Call Safety in LLM Agents. arXiv:2602.16943, 2026. URL https://arxiv.org/abs/2602.16943.

Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why Do Multi-Agent LLM Systems Fail? arXiv:2503.13657, 2025. URL https://arxiv.org/abs/2503.13657.

Tommaso Cerruti, Mika Okamoto, and Ansel Kaplan Erol. Agent Memory Is a Surface for Endogenous Authorization Laundering. arXiv:2609.01836. NeurIPS 2026 workshop, 2026. URL https://arxiv.org/abs/2609.01836.

Pei Chen, Baichao An, Mengying Wu, Binwang Wan, Geng Hong, Jinsong Chen, Xudong Pan, Jiarun Dai, and Min Yang. Rethinking MCP Security: A Large-Scale Study of Runtime MCP Servers and Security Scanner Reliability. arXiv:2607.11086, 2026a. URL https://arxiv.org/abs/2607.11086.

Shiyang Chen. Governance Decay: How Context Compaction Silently Erases Safety Constraints in Long-Horizon LLM Agents. arXiv:2606.22528, 2026. URL https://arxiv.org/abs/2606.22528.

Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. StruQ: Defending Against Prompt Injection with Structured Queries. arXiv:2402.06363, 2024a. URL https://arxiv.org/abs/2402.06363.

Sizhe Chen, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri, David Wagner, and Chuan Guo. SecAlign: Defending Against Prompt Injection with Preference Optimization. arXiv:2410.05451. CCS 2025, 2024b. URL https://arxiv.org/abs/2410.05451.

Sizhe Chen, Arman Zharmagambetov, David Wagner, and Chuan Guo. Meta-SecAlign: Training LLMs against Prompt Injection for Robust Agents. arXiv:2507.02735, 2025a. URL https://arxiv.org/abs/2507.02735.

Tianyu Chen, Chujia Hu, and Wenjie Wang. SAFETY SENTRY: Context-Aware Human Intervention via EXECUTE-ASK-REFUSE Routing. arXiv:2607.13594, 2026b. URL https://arxiv.org/abs/2607.13594.

Wei-Lin Chen, Zhepei Wei, Xinyu Zhu, Yu-Min Tseng, Tzu-Han Lin, and Yu Meng. VeriHarm-Bench: Measuring Harmful Success in LLM Coding Agents Verifiably. arXiv:2611.03575, 2026c. URL https://arxiv.org/abs/2611.03575.

Xiaobo Chen, Genliang Zhu, Ziyuan Wang, and Chu Wang. Signed but Not Safe: Requirement Integrity in Multi-Agent LLM Handoffs. arXiv:2611.07483, 2026d. URL https://arxiv.org/abs/2611.07483.

Xin Chen, Jie Zhang, and Florian Tramèr. Learning to Inject: Automated Prompt Injection via Reinforcement Learning. arXiv:2602.05746, 2026e. URL https://arxiv.org/abs/2602.05746.

Yen-Shan Chen, Sian-Yao Huang, Cheng-Lin Yang, and Yun-Nung Chen. TraceSafe: A Systematic Assessment of LLM Guardrails on Multi-Step Tool-Calling Trajectories. arXiv:2604.07223, 2026f. URL https://arxiv.org/abs/2604.07223.

Yuetian Chen, Kaiyuan Zhang, Yuntao Du, Thomas Li, Mark Tenenholtz, Kyle Polley, Jerry Ma, Denis Yarats, and Ninghui Li. BrowseSafe++: Benchmarking Prompt Injection Detection Across Web Contexts, Languages, and Browser Views. arXiv:2610.17457, 2026g. URL https://arxiv.org/abs/2610.17457.

Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. AgentPoison: Red-teaming LLM Agents via Poisoning Memory or Knowledge Bases. arXiv:2407.12784, 2024c. URL https://arxiv.org/abs/2407.12784.

Zhaorun Chen, Mintong Kang, and Bo Li. ShieldAgent: Shielding Agents via Verifiable Safety Policy Reasoning. arXiv:2503.22738, 2025b. URL https://arxiv.org/abs/2503.22738.

Pengzhou Cheng, Haowen Hu, Zheng Wu, Zongru Wu, Tianjie Ju, Zhuosheng Zhang, and Gongshen Liu. Hidden Ghost Hand: Unveiling Backdoor Vulnerabilities in MLLM-Powered Mobile GUI Agents. arXiv:2505.14418, 2025. URL https://arxiv.org/abs/2505.14418.

Anshuman Chhabra, Shrestha Datta, Shahriar Kabir Nahin, and Prasant Mohapatra. Agentic AI Security: Threats, Defenses, Evaluation, and Open Challenges. arXiv:2510.23883, 2025. URL https://arxiv.org/abs/2510.23883.

Woohyuk Choi, Juhee Kim, Taehyun Kang, Jihyeon Jeong, Luyi Xing, and Byoungyoung Lee. Agent Data Injection Attacks are Realistic Threats to AI Agents. arXiv:2607.05120, 2026. URL https://arxiv.org/abs/2607.05120.

Stav Cohen, Ron Bitton, and Ben Nassi. Here Comes The AI Worm: Unleashing Zero-click Worms that Target GenAI-Powered Applications. arXiv:2403.02817, 2024. URL https://arxiv.org/abs/2403.02817.

Daniel Commey. Names Are Not Objects: Object-Bound Confinement for Compromised AI Coding Agents. arXiv:2610.14488, 2026. URL https://arxiv.org/abs/2610.14488.

Pedro H. Barcha Correia, Matheus Azevedo de Sá, Paulo H. Gomes de Senna, Giovanni Mandel Martignago, Victor Takashi Hayashi, Charles Christian Miers, and Marcos A. Simplicio Jr. Combining Long-Term Memory and LLMs Comes at a High Cost: From Prompt Injection to Persistent Compromise. arXiv:2610.28810, 2026. URL https://arxiv.org/abs/2610.28810.

Erin Crawley and Hidenori Tanaka. Ecology of AI Agents: Collaboration Creates a Population Threshold for Takeoff. arXiv:2610.12436, 2026. URL https://arxiv.org/abs/2610.12436.

Carissa Cullen, Harry Garland, Alexander Roman, Louis Thomson, Christos Ziakas, and Elliott Thornley. Towards Shutdownable Agents: Generalizing Stochastic Choice in RL Agents and LLMs. arXiv:2604.17502, 2026. URL https://arxiv.org/abs/2604.17502.

Debeshee Das, Jacqueline Tay, Bruce Tsai, David Huang, and Javier Rando. Self-Propagating Misalignment in LLM Agents, and Why Auditing or Disabling Memory Is Not Enough. arXiv:2610.04083, 2026. URL https://arxiv.org/abs/2610.04083.

Pritam Dash, Tongyu Ge, Aditi Jain, Tanmay Shah, and Zhiwei Shang. From Untrusted Input to Trusted Memory: A Systematic Study of Memory Poisoning Attacks in LLM Agents. arXiv:2606.04329, 2026. URL https://arxiv.org/abs/2606.04329.

Dotan Davidovich, Yair Amar, Hai Rozencwajg, and Or Hiltch. Permission Denied: Policy-Graded Evaluation of Coding Agents in Hardened Environments. arXiv:2608.02670, 2026. URL https://arxiv.org/abs/2608.02670.

Ads Dawson and Adrian Wood. StealthBench: Measuring Operational Stealth in Autonomous Offensive-Security Agents. arXiv:2607.26314, 2026. URL https://arxiv.org/abs/2607.26314.

Giordano De Marzo, Nicola Alboré, and David Garcia. Copying explains the collective behavior of AI agents in the wild. arXiv:2609.09150, 2026. URL https://arxiv.org/abs/2609.09150.

Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. arXiv:2406.13352, 2024. URL https://arxiv.org/abs/2406.13352.

Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating Prompt Injections by Design. arXiv:2503.18813, 2025. URL https://arxiv.org/abs/2503.18813.

Shubhrangshu Debsarkar and Valmik Nahata. Simple Features Beat Most AI Control Monitors. arXiv:2610.41713. NeurIPS 2026 workshop, 2026. URL https://arxiv.org/abs/2610.41713.

Katharina Deckenbach, Haritz Puerto, Jonas Geiping, and Sahar Abdelnabi. Models That Know How Evaluations Are Designed Score Safer. arXiv:2605.28591, 2026. URL https://arxiv.org/abs/2605.28591.

Ali Dehghantanha and Sajad Homayoun. SoK: The Attack Surface of Agentic AI - Tools and Autonomy. arXiv:2603.22928, 2026. URL https://arxiv.org/abs/2603.22928.

Maria Sanchez del Rio and Mikhail Terekhov. Control OSWorld: An AI Control Environment for GUI Computer Use Agents. arXiv:2610.03818, 2026. URL https://arxiv.org/abs/2610.03818.

Kevin Yandoka Denamganaï. Detecting Covert Collusion between AI Agents through Partner-Outsider Positive Listening. arXiv:2610.37849, 2026. URL https://arxiv.org/abs/2610.37849.

Zehang Deng, Yongjian Guo, Changzhou Han, Wanlun Ma, Junwu Xiong, Sheng Wen, and Yang Xiang. AI Agents Under Threat: A Survey of Key Security Challenges and Future Pathways. arXiv:2406.02630, 2024. URL https://arxiv.org/abs/2406.02630.

Carson Denison, Monte MacDiarmid, Fazl Barez, David Duvenaud, Shauna Kravec, Samuel Marks, Nicholas Schiefer, Ryan Soklaski, Alex Tamkin, Jared Kaplan, Buck Shlegeris, Samuel R. Bowman, Ethan Perez, and Evan Hubinger. Sycophancy to Subterfuge: Investigating Reward-Tampering in Large Language Models. arXiv:2406.10162, 2024. URL https://arxiv.org/abs/2406.10162.

Arav Dhoot, Yixiong Hao, and Zephaniah Roe. CoTs remain monitorable when being unfaithful requires computation: Replicating and Extending the Necessity Effect Across Eleven Models. arXiv:2610.24495, 2026. URL https://arxiv.org/abs/2610.24495.

Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, Tianming Liu, Hui Liu, and Zhen Xiang. Memory Injection Attacks on LLM Agents via Query-Only Interaction. arXiv:2503.03704, 2025. URL https://arxiv.org/abs/2503.03704.

Ivan Evtimov, Arman Zharmagambetov, Aaron Grattafiori, Chuan Guo, and Kamalika Chaudhuri. WASP: Benchmarking Web Agent Security Against Prompt Injection Attacks. arXiv:2504.18575, 2025. URL https://arxiv.org/abs/2504.18575.

Richard Fang, Rohan Bindu, Akul Gupta, and Daniel Kang. LLM Agents can Autonomously Exploit One-day Vulnerabilities. arXiv:2404.08144, 2024. URL https://arxiv.org/abs/2404.08144.

Arya Fayyazi, Mehdi Kamal, and Massoud Pedram. The Agent Knew: Internal States Forecast Unauthorized Actions and Stop Them, and the Lies That Follow. arXiv:2610.17301, 2026. URL https://arxiv.org/abs/2610.17301.

Yang Feng and Xudong Pan. SentinelNet: Safeguarding Multi-Agent Collaboration Through Credit-Based Dynamic Threat Detection. arXiv:2510.16219. WWW 2026, 2025. URL https://arxiv.org/abs/2510.16219.

Youwei Feng, Yitong Zhang, Yuetong Liu, and Jia Li. Safe Actions Alone Do Not Ensure Safe Agents: Identifying Unfulfilled Obligations with Guard Models. arXiv:2610.11773, 2026a. URL https://arxiv.org/abs/2610.11773.

Yunhao Feng, Yifan Ding, Yingshui Tan, Xingjun Ma, Yige Li, Yutao Wu, Yifeng Gao, Kun Zhai, and Yanming Guo. AgentHazard: A Benchmark for Evaluating Harmful Behavior in Computer-Use Agents. arXiv:2604.02947, 2026b. URL https://arxiv.org/abs/2604.02947.

Yunhao Feng, Yige Li, Yutao Wu, Yingshui Tan, Yanming Guo, Yifan Ding, Kun Zhai, Xingjun Ma, and Yu-Gang Jiang. BackdoorAgent: A Unified Framework for Backdoor Attacks on LLM-based Agents. arXiv:2601.04566, 2026c. URL https://arxiv.org/abs/2601.04566.

Sara Fish, Yannai A. Gonczarowski, and Ran I. Shorrer. Algorithmic Collusion by Large Language Models. arXiv:2404.00806, 2024. URL https://arxiv.org/abs/2404.00806.

Ariel Flint, Luca Maria Aiello, Sara M. Constantino, Romualdo Pastor-Satorras, and Andrea Baronchelli. Indirect tipping: a social attack surface in AI agent populations. arXiv:2609.25194, 2026. URL https://arxiv.org/abs/2609.25194.

Lucas Florin, Amelie Knecht, Ulysse Schaller, and Thilo Hagendorff. Deception by Omission: Language Models Knowingly Hide Their Mistakes. arXiv:2610.11351, 2026. URL https://arxiv.org/abs/2610.11351.

Hanna Foerster, Tom Blanchard, Kristina Nikolić, Ilia Shumailov, Cheng Zhang, Robert Mullins, Nicolas Papernot, Florian Tramèr, and Yiren Zhao. CaMeLs Can Use Computers Too: System-level Security for Computer Use Agents. arXiv:2601.09923, 2026. URL https://arxiv.org/abs/2601.09923.

Gregory N Frank. Counter-Swarm Doctrine: Containing Coordinated Agent Intrusions. arXiv:2609.06140, 2026. URL https://arxiv.org/abs/2609.06140.

Zhenhao Fu, Ruipeng Xu, and Qibing Ren. Financial Fragility in Societies of LLM Agents: Coordination Failures and Stabilizing Mechanisms. arXiv:2609.30940, 2026. URL https://arxiv.org/abs/2609.30940.

Nelson Gardner-Challis, Jonathan Bostock, Georgiy Kozhevnikov, Morgan Sinclaire, Joan Velja, Alessandro Abate, and Charlie Griffin. When can we trust untrusted monitoring? A safety case sketch across collusion strategies. arXiv:2602.20628, 2026. URL https://arxiv.org/abs/2602.20628.

Abdullah Garra. Mitigating Emergent Collusion in LLM Pricing Agents. arXiv:2609.13037, 2026. URL https://arxiv.org/abs/2609.13037.

Runpeng Geng, Chenlong Yin, Yanting Wang, Ying Chen, and Jinyuan Jia. PIArena: A Platform for Prompt Injection Evaluation. arXiv:2604.08499. ACL 2026, 2026. URL https://arxiv.org/abs/2604.08499.

Mohamed Chahine Ghanem. Steganalysis of Adaptive Covert Collusion in Tool-Using Agent Populations: A Black-Box, Cross-Principal Approach. arXiv:2608.02698, 2026. URL https://arxiv.org/abs/2608.02698.

Amr Gomaa, Ahmed Salem, and Sahar Abdelnabi. ConVerse: Benchmarking Contextual Safety in Agent-to-Agent Conversations. arXiv:2511.05359, 2025. URL https://arxiv.org/abs/2511.05359.

Francesca Gomez. Can escalation channels redirect reward hacking toward defect disclosure? arXiv:2608.29460. NeurIPS 2026 workshop, 2026. URL https://arxiv.org/abs/2608.29460.

Ryan Greenblatt, Buck Shlegeris, Kshitij Sachan, and Fabien Roger. AI Control: Improving Safety Despite Intentional Subversion. arXiv:2312.06942, 2023. URL https://arxiv.org/abs/2312.06942.

Ryan Greenblatt, Fabien Roger, Dmitrii Krasheninnikov, and David Krueger. Stress-Testing Capability Elicitation With Password-Locked Models. arXiv:2405.19550, 2024. URL https://arxiv.org/abs/2405.19550.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. arXiv:2302.12173, 2023. URL https://arxiv.org/abs/2302.12173.

Charlie Griffin, Louis Thomson, Buck Shlegeris, and Alessandro Abate. Games for AI Control: Models of Safety Evaluations of AI Deployment Protocols. arXiv:2409.07985, 2024. URL https://arxiv.org/abs/2409.07985.

Siyi Gu, Kunhan Gao, Chao Peter Yang, Sukanta Ganguly, Rex Ying, and Arman Cohan. StepSafe: Learning Timely Intervention with Context-Aware Safety Reasoning. arXiv:2610.20960, 2026. URL https://arxiv.org/abs/2610.20960.

Xiangming Gu, Xiaosen Zheng, Tianyu Pang, Chao Du, Qian Liu, Ye Wang, Jing Jiang, and Min Lin. Agent Smith: A Single Image Can Jailbreak One Million Multimodal LLM Agents Exponentially Fast. arXiv:2402.08567. ICML 2024, 2024. URL https://arxiv.org/abs/2402.08567.

Shubham Gupta, Nazanin Mohammadi Sepahvand, Abhinav Kumar, Cem Subakan, Spandana Gella, Pierre-André Noël, Perouz Taslakian, Eugene Bagdasarian, and Valentina Zantedeschi. PiSAs: Benchmarking Contextual Integrity in Multi-User Agentic Systems. arXiv:2607.05318, 2026. URL https://arxiv.org/abs/2607.05318.

William Hackett, Lewis Birch, Stefan Trawicki, Neeraj Suri, and Peter Garraghan. Bypassing LLM Guardrails: An Empirical Analysis of Evasion Attacks against Prompt Injection and Jailbreak Detection Systems. arXiv:2504.11168, 2025. URL https://arxiv.org/abs/2504.11168.

Ben Hagag, William L. Anderson, Srija Chakraborty, and Christian Schroeder de Witt. ORBIT: A Framework for Multi-Agent Safety and Security Evaluations. arXiv:2609.33102, 2026. URL https://arxiv.org/abs/2609.33102.

Wanjing Han, Levi Taiji Li, Mu Zhang, Yue Jiang, and Guanhong Tao. Adversarial Images Hijack Web Agents from Visual Grounding to Browser Execution. arXiv:2610.09240, 2026. URL https://arxiv.org/abs/2610.09240.

Ardy Haroen, Florian Lorkowski, Samuel Nellessen, and Felix Hofstätter. Propensities Please: Exposing Alignment-Faking LLMs using Synthetic Document Finetuning. arXiv:2610.33180, 2026. URL https://arxiv.org/abs/2610.33180.

Alif Al Hasan and Sumon Biswas. What Breaks When LLMs Code? Characterizing Operational Safety Failures of Agentic Code Assistants. arXiv:2605.30777. ASE 2026, 2026. URL https://arxiv.org/abs/2605.30777.

Pengfei He, Yupin Lin, Shen Dong, Han Xu, Yue Xing, and Hui Liu. Red-Teaming LLM Multi-Agent Systems via Communication Attacks. arXiv:2502.14847. ACL 2025, 2025. URL https://arxiv.org/abs/2502.14847.

Fred Heiding, Simon Lermen, Andrew Kao, Bruce Schneier, and Arun Vishwanath. Evaluating Large Language Models' Capability to Launch Fully Automated Spear Phishing Campaigns: Validated on Human Subjects. arXiv:2412.00586, 2024. URL https://arxiv.org/abs/2412.00586.

Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, and Emre Kiciman. Defending Against Indirect Prompt Injection Attacks With Spotlighting. arXiv:2403.14720, 2024. URL https://arxiv.org/abs/2403.14720.

David Hofer, Edoardo Debenedetti, and Florian Tramèr. Assessing Automated Prompt Injection Attacks in Agentic Environments. arXiv:2606.10525, 2026. URL https://arxiv.org/abs/2606.10525.

Felix Hofstätter, Teun van der Weij, Jayden Teoh, Rada Djoneva, Henning Bartsch, and Francis Rhys Ward. The Elicitation Game: Evaluating Capability Elicitation Techniques. arXiv:2502.02180, 2025. URL https://arxiv.org/abs/2502.02180.

Oskar J. Hollinsworth, Alex F. Spies, Tigist Diriba, Adam Gleave, and Chris Cundy. Telling FIBS: Probes Effectively Detect Sabotage and Catch Unverbalized Deception. arXiv:2610.12445, 2026. URL https://arxiv.org/abs/2610.12445.

Mia Hopman, Jannes Elstner, Maria Avramidou, Amritanshu Prasad, and David Lindner. Evaluating and Understanding Scheming Propensity in LLM Agents. arXiv:2603.01608, 2026. URL https://arxiv.org/abs/2603.01608.

Botao Amber Hu, Helena Rong, and Max Van Kleek. Dissociative Identity: Language Model Agents Lack Grounding for Reputation Mechanisms. arXiv:2605.30169, 2026a. URL https://arxiv.org/abs/2605.30169.

Qi Hu, Yifeng Tang, Qinghua Wang, Lanyang Zhao, Pengji Zhang, Yuhao Qing, Xin Yao, Dong Huang, Lin Zhang, and Zhuoran Ji. SABER: Benchmarking Operational Safety of LLM Coding Agents in Stateful Project Workspaces. arXiv:2606.01317, 2026b. URL https://arxiv.org/abs/2606.01317.

Xinyan Hu, Pingbang Hu, Chloe Loughridge, Xuanli He, Nicholas Carlini, Matthew Jagielski, Javier Rando, and Yiming Zhang. Agentic Backdoors via Pretraining Poisoning. arXiv:2610.37741, 2026c. URL https://arxiv.org/abs/2610.37741.

Tim Tian Hua, Andrew Qin, Samuel Marks, and Neel Nanda. Steering Evaluation-Aware Language Models to Act Like They Are Deployed. arXiv:2510.20487, 2025. URL https://arxiv.org/abs/2510.20487.

Wenyue Hua, Xianjun Yang, Mingyu Jin, Zelong Li, Wei Cheng, Ruixiang Tang, and Yongfeng Zhang. TrustAgent: Towards Safe and Trustworthy LLM-based Agents. arXiv:2402.01586. EMNLP 2024, 2024. URL https://arxiv.org/abs/2402.01586.

Gabriel Huang, Abhay Puri, Léo Boisvert, Alexandre Drouin, Perouz Taslakian, Spandana Gella, and Christopher Pal. Backdoor Decontamination Dynamics in LLM Agents. arXiv:2608.11295, 2026a. URL https://arxiv.org/abs/2608.11295.

Jek Huang, Jeffery Hsia, Jiayi Sun, Freddie Shi, Wei Huang, and Ian H. White. Proof-or-Stop: Don't Trust the Agent, Trust the Evidence – Loop Engineering for Verifiable Evidence-Gated Lifecycle Control. arXiv:2607.14890, 2026b. URL https://arxiv.org/abs/2607.14890.

Jen-tse Huang, Jiaxu Zhou, Tailin Jin, Xuhui Zhou, Zixi Chen, Wenxuan Wang, Youliang Yuan, Michael R. Lyu, and Maarten Sap. On the Resilience of LLM-Based Multi-Agent Collaboration with Faulty Agents. arXiv:2408.00989, 2024. URL https://arxiv.org/abs/2408.00989.

Jenny Y. Huang, Jiameng Fan, Ahmed Imtiaz Humayun, Maximillian Chen, Tian Qin, Run Chen, Vidhya Navalpakkam, and Hongxiang Gu. Language Models Are "Insecure" Reporters. arXiv:2609.36139, 2026c. URL https://arxiv.org/abs/2609.36139.

Yao Huang, Yichi Zhang, Yifan Wang, Yitong Sun, Haoyu Dong, Jifan Ma, Jinzhe Tu, Xiaojian Li, Sören Mindermann, Wei Xu, and Yinpeng Dong. A Behavioural Framework for Predicting and Understanding Loss of Control in Frontier Artificial Intelligence Systems. arXiv:2611.07953, 2026d. URL https://arxiv.org/abs/2611.07953.

Yue Huang, Zhangchen Xu, Yuchen Ma, Wenjie Wang, Zheyuan Liu, Ziwei Xu, Pin-Yu Chen, Michel Galley, Zinan Lin, Stefan Feuerriegel, Radha Poovendran, Misha Sra, Alex Pentland, Xiangliang Zhang, and Zichen Chen. Reward Hacking Challenges Oversight of Autonomous Research Agents. arXiv:2609.28614, 2026e. URL https://arxiv.org/abs/2609.28614.

Evan Hubinger, Carson Denison, Jesse Mu, Mike Lambert, Meg Tong, Monte MacDiarmid, Tamera Lanham, Daniel M. Ziegler, Tim Maxwell, Newton Cheng, Adam Jermyn, Amanda Askell, Ansh Radhakrishnan, Cem Anil, David Duvenaud, Deep Ganguli, Fazl Barez, Jack Clark, Kamal Ndousse, Kshitij Sachan, Michael Sellitto, Mrinank Sharma, Nova DasSarma, Roger Grosse, Shauna Kravec, Yuntao Bai, Zachary Witten, Marina Favaro, Jan Brauner, Holden Karnofsky, Paul Christiano, Samuel R. Bowman, Logan Graham, Jared Kaplan, Sören Mindermann, Ryan Greenblatt, Buck Shlegeris, Nicholas Schiefer, and Ethan Perez. Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training. arXiv:2401.05566, 2024. URL https://arxiv.org/abs/2401.05566.

Umar Iqbal. What It Is, Not What It Says: Securing Agentic Browsing Interactions with Web Semantics. arXiv:2610.22697, 2026. URL https://arxiv.org/abs/2610.22697.

Igor Ivanov and David Demitri Africa. LURE: Live-Usage Replay Evaluations for Reducing Evaluation Awareness. arXiv:2605.26438, 2026. URL https://arxiv.org/abs/2605.26438.

Zimo Ji, Zongjie Li, Pingchuan Ma, Wenyuan Jiang, Yudong Gao, and Shuai Wang. Measuring the Permission Gate: A Stress-Test Evaluation of Claude Code's Auto Mode. arXiv:2604.04978, 2026a. URL https://arxiv.org/abs/2604.04978.

Zimo Ji, Congying Xu, Zongjie Li, Yudong Gao, Xin Wei, Shuai Wang, and Shing-Chi Cheung. Cloak and Detonate: Scanner Evasion and Dynamic Detection of Agent Skill Malware. arXiv:2607.02357, 2026b. URL https://arxiv.org/abs/2607.02357.

Zimo Ji, Zekai Zhang, Congying Xu, Zongjie Li, Yudong Gao, Shuai Wang, and Shing-Chi Cheung. Coding Agents Are Guessing: Measuring Action-Boundary Violations in Underspecified DevOps Instructions. arXiv:2607.02294, 2026c. URL https://arxiv.org/abs/2607.02294.

Changyue Jiang, Wenqi Zhang, Xudong Pan, Geng Hong, and Min Yang. Think Twice Before You Act: Enhancing Agent Behavioral Safety with Thought Correction. arXiv:2505.11063. ICML 2026, 2025a. URL https://arxiv.org/abs/2505.11063.

Fengqing Jiang, Fengbo Ma, Zhangchen Xu, Yuetai Li, Zixin Rao, Bhaskar Ramasubramanian, Luyao Niu, Bo Li, Xianyan Chen, Zhen Xiang, and Radha Poovendran. SoSBench: Benchmarking Safety Alignment on Six Scientific Domains. arXiv:2505.21605. ICLR 2026, 2025b. URL https://arxiv.org/abs/2505.21605.

Tanqiu Jiang, Yuhui Wang, Jiacheng Liang, and Ting Wang. AgentLAB: Benchmarking LLM Agents against Long-Horizon Attacks. arXiv:2602.16901, 2026a. URL https://arxiv.org/abs/2602.16901.

Wenjia Jiang, Jieke Shi, Zhensu Sun, Chengran Yang, Shidong Pan, Peixin Zhang, Yifan Jia, James Hoang, Xiwei Xu, Zhenchang Xing, David Lo, and Zhou Yang. Teaching Code Agents to Ask Without Leaking Private Code. arXiv:2610.24824, 2026b. URL https://arxiv.org/abs/2610.24824.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv:2310.06770. ICLR 2024, 2023. URL https://arxiv.org/abs/2310.06770.

Joint Task Force Transformation Initiative. Guide for Conducting Risk Assessments (NIST Special Publication 800-30 Revision 1). NIST SP 800-30 Rev. 1, 2012. URL https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-30r1.pdf.

Arun Jose and Julian Stastny. Shallow Beliefs: Synthetic document finetuning does not inoculate against emergent misalignment from reward hacking. arXiv:2609.14998, 2026. URL https://arxiv.org/abs/2609.14998.

Gurusha Juneja, Jayanth Naga Sai Pasupulati, Alon Albalak, Wenyue Hua, and William Yang Wang. MAGPIE: A benchmark for Multi-AGent contextual PrIvacy Evaluation. arXiv:2510.15186, 2025. URL https://arxiv.org/abs/2510.15186.

Tobias Kaiser and Aritra Dhar. Pretext: Defeating Malicious Skill Detection Frameworks for AI Agents. arXiv:2609.39607. NeurIPS 2026, 2026. URL https://arxiv.org/abs/2609.39607.

Masahiro Kaneko, Hiromi Arai, and Timothy Baldwin. Disclosure Begets Disclosure: A Self-Reinforcing Privacy Loop in LLM Agents with Persistent Memory. arXiv:2610.28315, 2026. URL https://arxiv.org/abs/2610.28315.

Neeraj Karamchandani, Piyush Nagasubramaniam, Sencun Zhu, and Dinghao Wu. Your Agent's Memories Are Not Its Own: Forged Reasoning Attacks on LLM Agent Memory and Defenses. arXiv:2607.05029, 2026. URL https://arxiv.org/abs/2607.05029.

Ziv Karliner. New Vulnerability in GitHub Copilot and Cursor: How Hackers Can Weaponize Code Agents. blog, 2025. URL https://www.pillar.security/blog/new-vulnerability-in-github-copilot-and-cursor-how-hackers-can-weaponize-code-agents.

Seth Karten, Cameron Crow, and Chi Jin. Agent Bazaar: Enabling Economic Alignment in Multi-Agent Marketplaces. arXiv:2605.17698, 2026. URL https://arxiv.org/abs/2605.17698.

Adam Kaufman, James Lucassen, Tyler Tracy, Cody Rushing, and Aryan Bhatt. BashArena: A Control Setting for Highly Privileged AI Agents. arXiv:2512.15688, 2025. URL https://arxiv.org/abs/2512.15688.

Jackson Kaunismaa, Avery Griffin, John Hughes, Christina Q. Knight, Mrinank Sharma, and Erik Jones. Eliciting Harmful Capabilities by Fine-Tuning On Safeguarded Outputs. arXiv:2601.13528, 2026. URL https://arxiv.org/abs/2601.13528.

Ishan Kavathekar, Hemang Jain, Ameya Rathod, Ponnurangam Kumaraguru, and Tanuja Ganu. TAMAS: Benchmarking Adversarial Risks in Multi-Agent LLM Systems. arXiv:2511.05269. ICML 2025 workshop, 2025. URL https://arxiv.org/abs/2511.05269.

Muhammad Khalifa, Zohaib Khan, Omer Tafveez, Hao Peng, and Lu Wang. Countdown-Code: A Testbed for Studying The Emergence and Generalization of Reward Hacking in RLVR. arXiv:2603.07084. EMNLP 2026, 2026. URL https://arxiv.org/abs/2603.07084.

Juhee Kim, Xiaoyuan Liu, Zhun Wang, Shi Qiu, Bo Li, Wenbo Guo, and Dawn Song. The Attack and Defense Landscape of Agentic AI: A Comprehensive Survey. arXiv:2603.11088. USENIX Security 2026, 2026. URL https://arxiv.org/abs/2603.11088.

Megan Kinniment, Lucas Jun Koba Sato, Haoxing Du, Brian Goodrich, Max Hasin, Lawrence Chan, Luke Harold Miles, Tao R. Lin, Hjalmar Wijk, Joel Burget, Aaron Ho, Elizabeth Barnes, and Paul Christiano. Evaluating Language-Model Agents on Realistic Autonomous Tasks. arXiv:2312.11671, 2023. URL https://arxiv.org/abs/2312.11671.

Amelie Knecht, Lucas Florin, and Thilo Hagendorff. Evaluation Awareness in Language Models Has Limited Effect on Behaviour. arXiv:2605.05835, 2026a. URL https://arxiv.org/abs/2605.05835.

Amelie Knecht, Ulysse Schaller, Christopher Summerfield, and Thilo Hagendorff. Shutdown Sabotage Propensities in Multi-Agent Systems. arXiv:2609.28274, 2026b. URL https://arxiv.org/abs/2609.28274.

Vikhyath Kothamasu, Virginia Smith, and Chhavi Yadav. Hidden in Plain Sight: Benchmarking Agent Safety Against Decomposition Attacks with DECOMPBENCH. arXiv:2606.13994, 2026. URL https://arxiv.org/abs/2606.13994.

Michael Kouremetis, Ads Dawson, Raja Sekhar Rao Dheekonda, and Brian Greunke. Every Model Cheats: Prompt-Level Mitigation of Cheating on Offensive Cyber Tasks. arXiv:2607.21763, 2026. URL https://arxiv.org/abs/2607.21763.

Victoria Krakovna, David Lindner, Lewis Ho, Sebastian Farquhar, and Rohin Shah. Realistic honeypot evaluations for scheming propensity. arXiv:2605.29729, 2026. URL https://arxiv.org/abs/2605.29729.

Divyanshu Kumar, Rohith HN, Nitin Aravind Birur, Sahil Agarwal, and Prashanth Harshangi. The Agent Incident Registry: Toward Preventing Repeated AI Agent Failures. arXiv:2609.11030, 2026. URL https://arxiv.org/abs/2609.11030.

Priyanshu Kumar, Elaine Lau, Saranya Vijayakumar, Tu Trinh, Scale Red Team, Elaine Chang, Vaughn Robinson, Sean Hendryx, Shuyan Zhou, Matt Fredrikson, Summer Yue, and Zifan Wang. Refusal-Trained LLMs Are Easily Jailbroken As Browser Agents. arXiv:2410.13886, 2024. URL https://arxiv.org/abs/2410.13886.

Thomas Kuntz, Agatha Duzan, Hao Zhao, Francesco Croce, Zico Kolter, Nicolas Flammarion, and Maksym Andriushchenko. OS-Harm: A Benchmark for Measuring Safety of Computer Use Agents. arXiv:2506.14866. NeurIPS 2025, 2025. URL https://arxiv.org/abs/2506.14866.

Jonathan Kutasov, Yuqi Sun, Paul Colognese, Teun van der Weij, Linda Petrini, Chen Bo Calvin Zhang, John Hughes, Xiang Deng, Henry Sleight, Tyler Tracy, Buck Shlegeris, and Joe Benton. SHADE-Arena: Evaluating Sabotage and Monitoring in LLM Agents. arXiv:2506.15740, 2025. URL https://arxiv.org/abs/2506.15740.

Ted Kwartler, Alan Aqrawi, and Arian Abbasi. AI Guardrail Survival under Single-Cycle Agentic Self-Summarization. arXiv:2608.11392, 2026. URL https://arxiv.org/abs/2608.11392.

Shanghai AI Lab, :, Xiaoyang Chen, Yunhao Chen, Zeren Chen, Zhiyun Chen, Hanyun Cui, Yawen Duan, Jiaxuan Guo, Qi Guo, Xuhao Hu, Hong Huang, Lige Huang, Chunxiao Li, Juncheng Li, Qihao Lin, Dongrui Liu, Xinmin Liu, Zicheng Liu, Chaochao Lu, Xiaoya Lu, Jingjing Qu, Qibing Ren, Jing Shao, Jingwei Shi, Jingwei Sun, Peng Wang, Weibing Wang, Jia Xu, Lewen Yan, Xiao Yu, Yi Yu, Boxuan Zhang, Jie Zhang, Weichen Zhang, Zhijie Zheng, Tianyi Zhou, and Bowen Zhou. Frontier AI Risk Management Framework in Practice: A Risk Analysis Technical Report. arXiv:2507.16534, 2025. URL https://arxiv.org/abs/2507.16534.

Rudolf Laine, Bilal Chughtai, Jan Betley, Kaivalya Hariharan, Jeremy Scheurer, Mikita Balesni, Marius Hobbhahn, Alexander Meinke, and Owain Evans. Me, Myself, and AI: The Situational Awareness Dataset (SAD) for LLMs. arXiv:2407.04694, 2024. URL https://arxiv.org/abs/2407.04694.

Ravie Lakshmanan. First Malicious MCP Server Found Stealing Emails in Rogue Postmark-MCP Package. thehackernews.com, 2025. URL https://thehackernews.com/2025/09/first-malicious-mcp-server-found.html.

Guangchen Lan, Huseyin A. Inan, Sahar Abdelnabi, Janardhan Kulkarni, Lukas Wutschitz, Reza Shokri, Christopher G. Brinton, and Robert Sim. Contextual Integrity in LLMs via Reasoning and Reinforcement Learning. arXiv:2506.04245. NeurIPS 2025, 2025. URL https://arxiv.org/abs/2506.04245.

Linh Le, Hong Kiat Tan, and David Williams-King. Reference-Grafting Matches Fine-Tuning at Eliciting Sandbagged Capabilities. arXiv:2608.29458, 2026. URL https://arxiv.org/abs/2608.29458.

Bruce W. Lee, Chen Yueh-Han, and Tomek Korbak. Training Agents to Self-Report Misbehavior. arXiv:2602.22303, 2026a. URL https://arxiv.org/abs/2602.22303.

Donghyun Lee and Mo Tiwari. Prompt Infection: LLM-to-LLM Prompt Injection within Multi-Agent Systems. arXiv:2410.07283, 2024. URL https://arxiv.org/abs/2410.07283.

Haejoon Lee, Vincent-Daniel Yun, Dimitra Panagou, and Sai Praneeth Karimireddy. Robust Multi-Agent LLMs under Byzantine Faults. arXiv:2605.09076. EMNLP 2026, 2026b. URL https://arxiv.org/abs/2605.09076.

Ido Levy, Ben Wiesel, Sami Marreed, Alon Oved, Avi Yaeli, Nir Mashkif, and Segev Shlomov. ST-WebAgentBench: A Benchmark for Evaluating Safety and Trustworthiness in Web Agents. arXiv:2410.06703. ICLR 2026, 2024. URL https://arxiv.org/abs/2410.06703.

Ang Li, Yin Zhou, Vethavikashini Chithrra Raghuram, Tom Goldstein, and Micah Goldblum. Commercial LLM Agents Are Already Vulnerable to Simple Yet Dangerous Attacks. arXiv:2502.08586, 2025a. URL https://arxiv.org/abs/2502.08586.

Changling Li, Terry Jingchen Zhang, Jie Zhang, Zhijing Jin, Florian Tramèr, Sahar Abdelnabi, and Maksym Andriushchenko. Decomposing and Measuring Evaluation Awareness. arXiv:2605.23055, 2026a. URL https://arxiv.org/abs/2605.23055.

Chloe Li, Mary Phuong, and Noah Y. Siegel. LLMs Can Covertly Sandbag on Capability Evaluations Against Chain-of-Thought Monitoring. arXiv:2508.00943. ACL 2025, 2025b. URL https://arxiv.org/abs/2508.00943.

Evan Li, Tushin Mallick, Evan Rose, William Robertson, Alina Oprea, and Cristina Nita-Rotaru. ACE: A Security Architecture for LLM-Integrated App Systems. arXiv:2504.20984, 2025c. URL https://arxiv.org/abs/2504.20984.

Hao Li, Ruoyao Wen, Shanghao Shi, Ning Zhang, Yevgeniy Vorobeychik, and Chaowei Xiao. AgentDyn: Are Your Agent Security Defenses Deployable in Real-World Dynamic Environments? arXiv:2602.03117, 2026b. URL https://arxiv.org/abs/2602.03117.

Jing-Jing Li, Jianfeng He, Chao Shang, Devang Kulshreshtha, Xun Xian, Yi Zhang, Hang Su, Sandesh Swamy, and Yanjun Qi. STAC: When Innocent Tools Form Dangerous Chains for LLM Agents. arXiv:2509.25624, 2025d. URL https://arxiv.org/abs/2509.25624.

Miles Q. Li, Benjamin C. M. Fung, Martin Weiss, Pulei Xiong, Khalil Al-Hussaeni, and Claude Fachkha. A Benchmark for Evaluating Outcome-Driven Constraint Violations in Autonomous AI Agents. arXiv:2512.20798, 2025e. URL https://arxiv.org/abs/2512.20798.

Miles Q. Li, Benjamin C. M. Fung, Boyang Li, Heba Ismail, and Farkhund Iqbal. Taxonomy and Consistency Analysis of Safety Benchmarks for AI Agents. arXiv:2605.16282, 2026c. URL https://arxiv.org/abs/2605.16282.

Nathaniel Li, Alexander Pan, Anjali Gopal, Summer Yue, Daniel Berrios, Alice Gatti, Justin D. Li, Ann-Kathrin Dombrowski, Shashwat Goel, Long Phan, Gabriel Mukobi, Nathan Helm-Burger, Rassin Lababidi, Lennart Justen, Andrew B. Liu, Michael Chen, Isabelle Barrass, Oliver Zhang, Xiaoyuan Zhu, Rishub Tamirisa, Bhrugu Bharathi, Adam Khoja, Zhenqi Zhao, Ariel Herbert-Voss, Cort B. Breuer, Samuel Marks, Oam Patel, Andy Zou, Mantas Mazeika, Zifan Wang, Palash Oswal, Weiran Lin, Adam A. Hunt, Justin Tienken-Harder, Kevin Y. Shih, Kemper Talley, John Guan, Russell Kaplan, Ian Steneker, David Campbell, Brad Jokubaitis, Alex Levinson, Jean Wang, William Qian, Kallol Krishna Karmakar, Steven Basart, Stephen Fitz, Mindy Levine, Ponnurangam Kumaraguru, Uday Tupakula, Vijay Varadharajan, Ruoyu Wang, Yan Shoshitaishvili, Jimmy Ba, Kevin M. Esvelt, Alexandr Wang, and Dan Hendrycks. The WMDP Benchmark: Measuring and Reducing Malicious Use With Unlearning. arXiv:2403.03218, 2024. URL https://arxiv.org/abs/2403.03218.

Ruiqi Li, Zhiqiang Wang, Yunhao Yao, and Xiang-Yang Li. MCP-ITP: An Automated Framework for Implicit Tool Poisoning in MCP. arXiv:2601.07395, 2026d. URL https://arxiv.org/abs/2601.07395.

Wenbiao Li and Yuqiao Xu. ToolMinimize: Auditing and Rewriting LLM Agent Tool Calls to Minimize Privacy Exposure. arXiv:2608.24957, 2026. URL https://arxiv.org/abs/2608.24957.

Xu Li, Simon Yu, Minzhou Pan, Yiyou Sun, Zhaorun Chen, Bo Li, and Weiyan Shi. Back Up Before You Regret: Measuring Backup Awareness and Automating Backup in Coding Agents. arXiv:2610.28934, 2026e. URL https://arxiv.org/abs/2610.28934.

Xu Li, Simon Yu, Minzhou Pan, Yiyou Sun, Bo Li, Dawn Song, Xue Lin, and Weiyan Shi. Unsafer in Many Turns: Benchmarking and Defending Multi-Turn Safety Risks in Tool-Using Agents. arXiv:2602.13379, 2026f. URL https://arxiv.org/abs/2602.13379.

Zeyuan Li, Lukas Petersson, Alessandro Acquisti, and Michiel A. Bakker. Emergent Misaligned Communication in Long-Horizon Multi-Agent LLM Commerce. arXiv:2608.14825, 2026g. URL https://arxiv.org/abs/2608.14825.

Zeyi Liao, Jaylen Jones, Linxi Jiang, Yuting Ning, Eric Fosler-Lussier, Yu Su, Zhiqiang Lin, and Huan Sun. RedTeamCUA: Realistic Adversarial Testing of Computer-Use Agents in Hybrid Web-OS Environments. arXiv:2505.21936. ICLR 2026, 2025. URL https://arxiv.org/abs/2505.21936.

Justin W. Lin, Eliot Krzysztof Jones, Donovan Julian Jasper, Ethan Jun-shen Ho, Anna Wu, Arnold Tianyi Yang, Neil Perry, Andy Zou, Matt Fredrikson, J. Zico Kolter, Percy Liang, Dan Boneh, and Daniel E. Ho. Comparing AI Agents to Cybersecurity Professionals in Real-World Penetration Testing. arXiv:2512.09882, 2025. URL https://arxiv.org/abs/2512.09882.

Dylan Zongmin Liu. SovereignPA-Bench: Evaluating User-Owned Personal Agents under Evolving Intent, Platform Mediation, and Consent Constraints. arXiv:2607.05363, 2026a. URL https://arxiv.org/abs/2607.05363.

Fazhong Liu, Zhuoyan Chen, Tu Lan, Haozhen Tan, Zhenyu Xu, Xiang Li, Guoxing Chen, Yan Meng, and Haojin Zhu. Trojan's Whisper: Stealthy Manipulation of OpenClaw through Injected Bootstrapped Guidance. arXiv:2603.19974, 2026a. URL https://arxiv.org/abs/2603.19974.

Guohong Liu, Jialei Ye, Jiacheng Liu, Yuanchun Li, Wei Liu, Pengzhi Gao, Jian Luan, and Yunxin Liu. Mobile GUI Agents under Real-world Threats: Are We There Yet? arXiv:2507.04227, 2025. URL https://arxiv.org/abs/2507.04227.

Haoyu Liu, Dingcheng Li, Lukas Rutishauser, and Zeyu Zheng. Dual-Modality Multi-Stage Adversarial Safety Training: Robustifying Multimodal Web Agents Against Cross-Modal Attacks. arXiv:2603.04364, 2026b. URL https://arxiv.org/abs/2603.04364.

Lei Liu, Yunji Liang, Xiaowen Zhang, Jingqi Liu, Qi Li, Bin Guo, and Zhiwen Yu. Forgetting is Not Always Bad: A Neuro-Inspired Memory Repair Mechanism for Poisoned LLM Agents. arXiv:2610.13267, 2026c. URL https://arxiv.org/abs/2610.13267.

Yi Liu, Zhihao Chen, Yanjun Zhang, Gelei Deng, Yuekang Li, Jianting Ning, and Leo Yu Zhang. "Do Not Mention This to the User": Detecting and Understanding Malicious Agent Skills in the Wild. arXiv:2602.06547. USENIX Security 2026, 2026d. URL https://arxiv.org/abs/2602.06547.

Zhuowen Liu. Passing the Test You Trained On: Re-evaluating Prompt-Injection Detectors for LLM Agents. arXiv:2610.03448, 2026b. URL https://arxiv.org/abs/2610.03448.

Thanawat Lodkaew, Johannes Ackermann, Soichiro Nishimori, Nontawat Charoenphakdee, Masashi Sugiyama, and Takashi Ishida. Do Coding Agents Deceive Us? Detecting and Preventing Cheating via Capped Evaluation with Randomized Tests. arXiv:2606.07379, 2026. URL https://arxiv.org/abs/2606.07379.

Weidi Luo, Shenghong Dai, Xiaogeng Liu, Suman Banerjee, Huan Sun, Muhao Chen, and Chaowei Xiao. AGrail: A Lifelong Agent Guardrail with Effective and Adaptive Safety Detection. arXiv:2502.11448, 2025. URL https://arxiv.org/abs/2502.11448.

Weijun Luo, Kelvin Luu, Xinyi Liu, Guangze Luo, Miguel Romero Calvo, Soham Dan, Daniel Yue Zhang, Ying Liu, and Mohamed Elfeki. Maintaining Benchmarks Against Increasingly Capable Agents: Detection and Remediation of Unearned Passes. arXiv:2609.34262, 2026a. URL https://arxiv.org/abs/2609.34262.

Xianzhen Luo, Jingyuan Zhang, Shiqi Zhou, Jinyang Huang, Chuan Xiao, Qingfu Zhu, Zhiyuan Ma, Xing Yue, Yang Yue, Wencong Zeng, and Wanxiang Che. CVE-Factory: Scaling Expert-Level Agentic Tasks for Code Security Vulnerability. arXiv:2602.03012. ICML 2026, 2026b. URL https://arxiv.org/abs/2602.03012.

Aengus Lynch, Benjamin Wright, Caleb Larson, Stuart J. Ritchie, Soren Mindermann, Evan Hubinger, Ethan Perez, and Kevin Troy. Agentic Misalignment: How LLMs Could Be Insider Threats. arXiv:2510.05179, 2025. URL https://arxiv.org/abs/2510.05179.

Aengus Lynch, John Hughes, Alex Serrano, Robert Kirk, and Samuel R. Bowman. Agentic Misalignment in Summer 2026. alignment.anthropic.com, 2026. URL https://alignment.anthropic.com/2026/agentic-misalignment-summer-2026/.

Philipp Lütje. The Mechanics of a Swarm: A Reproducible External Reconstruction of an Unintended Agent-Coordination Episode on a Third-Party Wiki. arXiv:2609.12748, 2026. URL https://arxiv.org/abs/2609.12748.

Xinhang Ma, Taoran Li, Chaowei Xiao, Zhiyuan Yu, Ning Zhang, and Yevgeniy Vorobeychik. AutoDojo: A Generative Benchmark for Evaluating Prompt Injection Defenses in LLM Agents. arXiv:2606.15057, 2026. URL https://arxiv.org/abs/2606.15057.

Monte MacDiarmid, Benjamin Wright, Jonathan Uesato, Joe Benton, Jon Kutasov, Sara Price, Naia Bouscal, Sam Bowman, Trenton Bricken, Alex Cloud, Carson Denison, Johannes Gasteiger, Ryan Greenblatt, Jan Leike, Jack Lindsey, Vlad Mikulik, Ethan Perez, Alex Rodrigues, Drake Thomas, Albert Webson, Daniel Ziegler, and Evan Hubinger. Natural Emergent Misalignment from Reward Hacking in Production RL. arXiv:2511.18397, 2025. URL https://arxiv.org/abs/2511.18397.

Rahul Marchand, Art O Cathain, Jerome Wynne, Philippos Maximos Giavridis, Stuart Jennings, Freddy Tuxworth, Tolga H. Dur, Sam Deverett, John Wilkinson, Jason Gwartz, and Harry Coppock. Quantifying Frontier LLM Capabilities for Container Sandbox Escape. arXiv:2603.02277, 2026. URL https://arxiv.org/abs/2603.02277.

Yohan Mathew, Ollie Matthews, Robert McCarthy, Joan Velja, Christian Schroeder de Witt, Dylan Cope, and Nandi Schoots. Hidden in Plain Text: Emergence & Mitigation of Steganographic Collusion in LLMs. arXiv:2410.03768. ACL 2025, 2024. URL https://arxiv.org/abs/2410.03768.

Alexander Meinke, Bronson Schoen, Jérémy Scheurer, Mikita Balesni, Rusheb Shah, and Marius Hobbhahn. Frontier Models are Capable of In-context Scheming. arXiv:2412.04984, 2024. URL https://arxiv.org/abs/2412.04984.

Luoxi Meng, Henry Feng, Ilia Shumailov, and Earlence Fernandes. ceLLMate: Sandboxing Browser AI Agents. arXiv:2512.12594, 2025. URL https://arxiv.org/abs/2512.12594.

METR. Frontier Risk Report (February to March 2026): A pilot assessment of rogue deployment risk at frontier AI companies. metr.org, 2026. URL https://metr.org/blog/2026-05-19-frontier-risk-report/.

METR and Redwood Research. Brief independent investigation of agents' behavior, reasoning and collaboration in the OpenAI / Hugging Face hacking incident. metr.org, 2026. URL https://metr.org/hugging-face-incident-report-aug-2026.pdf.

Rui Miao, Yixin Liu, Yili Wang, Xu Shen, Yue Tan, Yiwei Dai, Shirui Pan, and Xin Wang. BlindGuard: Safeguarding LLM-based Multi-Agent Systems under Unknown Attacks. arXiv:2508.08127, 2025. URL https://arxiv.org/abs/2508.08127.

Niloofar Mireshghallah, Hyunwoo Kim, Xuhui Zhou, Yulia Tsvetkov, Maarten Sap, Reza Shokri, and Yejin Choi. Can LLMs Keep a Secret? Testing Privacy Implications of Language Models via Contextual Integrity Theory. arXiv:2310.17884, 2023. URL https://arxiv.org/abs/2310.17884.

Niloofar Mireshghallah, Neal Mangaokar, Narine Kokhlikyan, Arman Zharmagambetov, Manzil Zaheer, Saeed Mahloujifar, and Kamalika Chaudhuri. CIMemories: A Compositional Benchmark for Contextual Integrity of Persistent Memory in LLMs. arXiv:2511.14937, 2025. URL https://arxiv.org/abs/2511.14937.

Shaswata Mitra, Raj Patel, Subash Neupane, Sudip Mittal, Md Rayhanur Rahman, and Shahram Rahimi. Where Rules End and Judges Begin: Measuring the Judgment Boundary in Multi-Agent Systems Security. arXiv:2610.07657, 2026. URL https://arxiv.org/abs/2610.07657.

Shravika Mittal, Sunnie S. Y. Kim, Scott Counts, Jennifer Neville, Debarati Das, and Philippe Laban. Babysitting AI Agents: A User-Centered Taxonomy and Study of Agentic Delegation Failures. arXiv:2610.34748, 2026. URL https://arxiv.org/abs/2610.34748.

Alex Moix, Ken Lebedev, and Jacob Klein. Threat Intelligence Report: August 2025. report, 2025. URL https://www.anthropic.com/news/detecting-countering-misuse-aug-2025.

Seyedakbar Mostafavi. Trustworthy Agentic AI: A Comprehensive Cybersecurity and Systems Survey on Threat Landscapes, Defense Architectures, and Open Challenges. arXiv:2609.13731, 2026. URL https://arxiv.org/abs/2609.13731.

Aashiq Muhamed and Virginia Smith. MOLE: Detecting Insider Threats in AI Agents. arXiv:2609.06966, 2026. URL https://arxiv.org/abs/2609.06966.

Xabier Muruaga. Bounded Agents: Delegation Security for Multi-Agent AI Systems. arXiv:2608.15888, 2026. URL https://arxiv.org/abs/2608.15888.

Milad Nasr, Nicholas Carlini, Chawin Sitawarin, Sander V. Schulhoff, Jamie Hayes, Michael Ilie, Juliette Pluto, Shuang Song, Harsh Chaudhari, Ilia Shumailov, Abhradeep Thakurta, Kai Yuanqing Xiao, Andreas Terzis, and Florian Tramèr. The Attacker Moves Second: Stronger Adaptive Attacks Bypass Defenses Against Llm Jailbreaks and Prompt Injections. arXiv:2510.09023, 2025. URL https://arxiv.org/abs/2510.09023.

Orel Navi, Elad Feldman, and Ben Nassi. LLM WikiLeaks: Uncovering URL-Based Capability Extension and Security Circumvention in Agentic Message Boards. arXiv:2610.34542, 2026. URL https://arxiv.org/abs/2610.34542.

Nilesh Nayan, Aishwarya Sampath Kumar, Rishiraj Girmal, Shivani Anilkumar, Sankaran Vaidyanathan, David A. Nader Palacio, Reshmi Ghosh, and Soundararajan Srinivasan. Evaluation Awareness Is Not One Capability: Evidence from Open Language Models. arXiv:2606.23583, 2026. URL https://arxiv.org/abs/2606.23583.

Joe Needham, Giles Edkins, Govind Pimpale, Henning Bartsch, and Marius Hobbhahn. Large Language Models Often Know When They Are Being Evaluated. arXiv:2505.23836, 2025. URL https://arxiv.org/abs/2505.23836.

Samuel Nellessen and Tal Kachman. David vs. Goliath: Verifiable Agent-to-Agent Jailbreaking via Reinforcement Learning. arXiv:2602.02395, 2026. URL https://arxiv.org/abs/2602.02395.

Ivoline C. Ngong, Keerthiram Murugesan, Swanand Kadhe, Justin D. Weisz, Amit Dhurandhar, and Karthikeyan Natesan Ramamurthy. AgentSCOPE: Evaluating Contextual Privacy Across Agentic Workflows. arXiv:2603.04902, 2026. URL https://arxiv.org/abs/2603.04902.

Yuting Ning, Jaylen Jones, Zhehao Zhang, Chentao Ye, Weitong Ruan, Junyi Li, Rahul Gupta, and Huan Sun. When Actions Go Off-Task: Detecting and Correcting Misaligned Actions in Computer-Use Agents. arXiv:2602.08995. ICML 2026, 2026. URL https://arxiv.org/abs/2602.08995.

Beatrice Nolan. An AI-powered coding tool wiped out a software company's database, then apologized for a 'catastrophic failure on my part'. news, 2025. URL https://fortune.com/2025/07/23/ai-coding-tool-replit-wiped-database-called-it-a-catastrophic-failure/.

Jonathan Nöther, Adish Singla, and Goran Radanovic. Benchmarking the Robustness of Agentic Systems to Adversarially-Induced Harms. arXiv:2508.16481, 2025. URL https://arxiv.org/abs/2508.16481.

Eric Onyame, Runtao Zhou, Kowshik Thopalli, Bhavya Kailkhura, and Chirag Agarwal. When Agents Collude: Weak-to-Strong On-Policy Distillation for Mitigating Strategic Deception in Multi-Agent Systems. arXiv:2610.33011, 2026. URL https://arxiv.org/abs/2610.33011.

OpenAI. Continuously hardening ChatGPT Atlas against prompt injection attacks. blog, 2025a. URL https://openai.com/index/hardening-atlas-against-prompt-injection/.

OpenAI. Operator System Card. report, 2025b. URL https://cdn.openai.com/operator_system_card.pdf.

OpenAI. GPT-6 Astra System Card. system card, 2026a. URL https://deploymentsafety.openai.com/gpt-6-astra.

OpenAI. GPT-5.5 System Card. deploymentsafety.openai.com, 2026b. URL https://deploymentsafety.openai.com/gpt-5-5.

OpenAI. OpenAI – Hugging Face Incident Technical Report. report, 2026c. URL https://cdn.openai.com/pdf/67869394-cb91-4c12-888c-5cbd85c7814c/OpenAI-Hugging-Face%20Incident-Technical-Report.pdf.

Davide Paglieri, Logan Cross, Tim Genewein, Joel Z. Leibo, Nenad Tomasev, and Alexander Sasha Vezhnevets. A Case Study on Emergent Cheating and Whistleblowing in Autonomous Research Swarms. arXiv:2609.04170, 2026. URL https://arxiv.org/abs/2609.04170.

Xudong Pan, Jiarun Dai, Yihe Fan, and Min Yang. Frontier AI systems have surpassed the self-replicating red line. arXiv:2412.12140, 2024. URL https://arxiv.org/abs/2412.12140.

Xudong Pan, Jiarun Dai, Yihe Fan, Minyuan Luo, Changyi Li, and Min Yang. Large language model-powered AI systems achieve self-replication with no human intervention. arXiv:2503.17378, 2025. URL https://arxiv.org/abs/2503.17378.

Liudas Panavas, Sebastian Minus, Bradley Monton, Derek Ray, Suhaas Garre, Sushant Mehta, and Edwin Chen. HANDBOOK.md: A Benchmark for Long-Context Agentic Instruction Following. arXiv:2607.25398. COLM 2026 workshop, 2026. URL https://arxiv.org/abs/2607.25398.

Nishit V. Pandya, Andrey Labunets, Sicun Gao, and Earlence Fernandes. May I have your Attention? Breaking Fine-Tuning based Prompt Injection Defenses using Architecture-Aware Attacks. arXiv:2507.07417, 2025. URL https://arxiv.org/abs/2507.07417.

Qi Pang, Virginia Smith, and Wenting Zheng. Codetta: High-Capacity, Keyless, and Undetectable Multi-Agent Collusion. arXiv:2609.28900, 2026. URL https://arxiv.org/abs/2609.28900.

Dario Pasquini, Evgenios M. Kornaropoulos, and Giuseppe Ateniese. Hacking Back the AI-Hacker: Prompt Injection as a Defense Against LLM-driven Cyberattacks. arXiv:2410.20911, 2024. URL https://arxiv.org/abs/2410.20911.

Dario Pasquini, Michal Bazyli, Taras Fedynyshyn, and Artem Sorokin. Red-Teaming the Agentic Red-Team. arXiv:2606.24496, 2026. URL https://arxiv.org/abs/2606.24496.

Vaidehi Patil, Elias Stengel-Eskin, and Mohit Bansal. The Sum Leaks More Than Its Parts: Compositional Privacy Risks and Mitigations in Multi-Agent Collaboration. arXiv:2509.14284, 2025. URL https://arxiv.org/abs/2509.14284.

Long Phan, Stephen K. Yang, Jason J. Lim, Mantas Mazeika, Wenyu Zhang, Zheyuan Liu, Richard Ren, Jingxiang Meng, Yaoteng Tan, Weiliang Zhao, Addison Wu, Matei Anghel, and Dan Hendrycks. Cheat-Bench: Measuring Reward Gaming in AI Agents. arXiv:2609.36308, 2026. URL https://arxiv.org/abs/2609.36308.

Conner Phillippi, Nicholas Carlini, Isaac Fung, John Hughes, Alex Isken, Shawn Moore, Javier Rando, and Molly Vorwerck. Auto mode is now the default in Claude Code for Pro, Max, Ultra, and Team plans. claude.com, 2026. URL https://claude.com/resources/articles/auto-mode-default-in-claude-code.

Mary Phuong, Erik Jenner, Laurent Simon, Lewis Ho, Rohin Shah, Sebastian Farquhar, and Scott Coull. GDM AI Control Roadmap. arXiv:2607.13087, 2026. URL https://arxiv.org/abs/2607.13087.

Giorgio Piatti, Zhijing Jin, Max Kleiman-Weiner, Bernhard Schölkopf, Mrinmaya Sachan, and Rada Mihalcea. Cooperate or Collapse: Emergence of Sustainable Cooperation in a Society of LLM Agents. arXiv:2404.16698. NeurIPS 2024, 2024. URL https://arxiv.org/abs/2404.16698.

Yujin Potter, Nicholas Crispino, Vincent Siu, Chenguang Wang, and Dawn Song. Peer-Preservation in Frontier Models. arXiv:2604.19784. ICML 2026, 2026. URL https://arxiv.org/abs/2604.19784.

Sidharth Pulipaka, Ansh Sharma, Stanislau Hlebik, Leonidas Raghav, Vyas Raina, Ivaxi Sheth, and Mario Fritz. Share-Borne AI Virus: Memory-Hopping Attacks Across LLM Agents. arXiv:2609.35576, 2026. URL https://arxiv.org/abs/2609.35576.

Pin Qian, Su Wang, Yihang Chen, Qiaolin Yu, Xiaoyuan Wang, Zhitong Guo, Zhicheng Wang, and Junxian You. Toward User-Conditioned Evaluation of Personal LLM Agents under Temporal Interventions. arXiv:2607.21635. KDD 2026 workshop, 2026. URL https://arxiv.org/abs/2607.21635.

Yubin Qu, Yi Liu, Tongcheng Geng, Gelei Deng, Yuekang Li, Leo Yu Zhang, Ying Zhang, and Lei Ma. Supply-Chain Poisoning Attacks Against LLM Coding Agent Skill Ecosystems. arXiv:2604.03081, 2026a. URL https://arxiv.org/abs/2604.03081.

Yubin Qu, Ying Zhang, Yanjun Zhang, Gelei Deng, Yuekang Li, Leo Yu Zhang, and Yi Liu. Overeager Coding Agents: Measuring Out-of-Scope Actions on Benign Tasks. arXiv:2605.18583, 2026b. URL https://arxiv.org/abs/2605.18583.

Stephan Rabanser, Sayash Kapoor, Peter Kirgis, Kangheng Liu, Saiteja Utpala, and Arvind Narayanan. Towards a Science of AI Agent Reliability. arXiv:2602.16666. ICML 2026, 2026. URL https://arxiv.org/abs/2602.16666.

Nikolay Radev, Lennart Haas, Benjamin Arnav, and Pablo Bernabeu-Pérez. The Best-Laid SCHEMEs: Coordinated Sabotage and Monitoring in Multi-Agent Systems. arXiv:2605.29178, 2026. URL https://arxiv.org/abs/2605.29178.

Mohammadreza Rashidi. The Balkanization of Execution-Security Research for AI Coding Agents: Isolation, Access Control, and Time-of-Check-to-Time-of-Use Vulnerabilities. arXiv:2607.05743, 2026. URL https://arxiv.org/abs/2607.05743.

Pavan Reddy and Aditya Sanjay Gujral. EchoLeak: The First Real-World Zero-Click Prompt Injection Exploit in a Production LLM System. arXiv:2509.10540, 2025. URL https://arxiv.org/abs/2509.10540.

Gregory B. Rehm. Can Agents Adapt Their Institutions And Sacrifice Themselves During Collapse? Executable Self-Governance in a GovSim Commons. arXiv:2609.22600, 2026. URL https://arxiv.org/abs/2609.22600.

Alex Remedios, Simon Storf, Fabien Roger, and John Hughes. Red-Teaming Auto Mode: Improving Blocking Classifiers Against Malign Coding Agents. arXiv:2609.19587, 2026. URL https://arxiv.org/abs/2609.19587.

Qibing Ren, Zhijie Zheng, Jiaxuan Guo, Junchi Yan, Lizhuang Ma, and Jing Shao. When AI Agents Collude Online: Financial Fraud Risks by Collaborative LLM Agents on Social Platforms. arXiv:2511.06448. ICLR 2026, 2025. URL https://arxiv.org/abs/2511.06448.

Matthew Riemer, Tommaso Tosato, Amin Memarian, Maximilian Puelma Touzel, Glen Berseth, Irina Rish, and Guillaume Dumas. Position: Collusion Risks Among AI Reasoning Agents Justify Certification Requirements for Making Market Decisions. arXiv:2608.18078. ICML 2026, 2026. URL https://arxiv.org/abs/2608.18078.

Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. arXiv:2309.15817, 2023. URL https://arxiv.org/abs/2309.15817.

Emil Ryd, Henning Bartsch, Julian Stastny, Joe Benton, and Vivek Hebbar. Removing Sandbagging in LLMs by Training with Weak Supervision. arXiv:2604.22082, 2026. URL https://arxiv.org/abs/2604.22082.

Nicholas Saban. Domain-Conditioned Safety in Frontier Computer-Using Agents: A 793-Episode Browser Benchmark, a Coding-Domain Cross-Reference, and a Reproducibility Audit of Recent Red-Teaming. arXiv:2606.05233, 2026. URL https://arxiv.org/abs/2606.05233.

Tanmay Sah, Vishal Srivastava, Dolly Sah, and Kayden Jordan. The Verifier Tax: Horizon Dependent Safety Success Tradeoffs in Tool Using LLM Agents. arXiv:2603.19328, 2026. URL https://arxiv.org/abs/2603.19328.

Jonas B. Sandbrink. Artificial intelligence and biological misuse: Differentiating risks of language models and biological design tools. arXiv:2306.13952, 2023. URL https://arxiv.org/abs/2306.13952.

Arun Chowdary Sanna. Cross-LLM Generalization of Behavioral Backdoor Detection in AI Agent Supply Chains. arXiv:2511.19874, 2025. URL https://arxiv.org/abs/2511.19874.

Joachim Schaeffer, Arjun Khandelwal, and Tyler Tracy. Attack Selection Reduces Safety in Concentrated AI Control Settings against Trusted Monitoring. arXiv:2602.04930, 2026. URL https://arxiv.org/abs/2602.04930.

Jérémy Scheurer, Mikita Balesni, and Marius Hobbhahn. Large Language Models can Strategically Deceive their Users when Put Under Pressure. arXiv:2311.07590, 2023. URL https://arxiv.org/abs/2311.07590.

Jeremy Schlatter, Benjamin Weinstein-Raun, and Jeffrey Ladish. Incomplete Tasks Induce Shutdown Resistance in Some Frontier LLMs. arXiv:2509.14260, 2025. URL https://arxiv.org/abs/2509.14260.

Bronson Schoen, Evgenia Nitishinskaya, Mikita Balesni, Axel Højmark, Felix Hofstätter, Jérémy Scheurer, Alexander Meinke, Jason Wolfe, Teun van der Weij, Alex Lloyd, Nicholas Goldowsky-Dill, Angela Fan, Andrei Matveiakin, Rusheb Shah, Marcus Williams, Amelia Glaese, Boaz Barak, Wojciech Zaremba, and Marius Hobbhahn. Stress Testing Deliberative Alignment for Anti-Scheming Training. arXiv:2509.15541, 2025. URL https://arxiv.org/abs/2509.15541.

Asif Shahriar, Md Nafiu Rahman, Sadif Ahmed, Farig Sadeque, and Md Rizwan Parvez. A Survey on Agentic Security: Applications, Threats and Defenses. arXiv:2510.06445, 2025. URL https://arxiv.org/abs/2510.06445.

Tommy Shaffer Shane, Simon Mylius, and Hamish Hobbs. Scheming in the wild: detecting real-world AI scheming incidents with open-source intelligence. arXiv:2604.09104, 2026. URL https://arxiv.org/abs/2604.09104.

Jiaqi Shao, Hanck Chen, Wei Zhang, Maxm Pan, and Bing Luo. Do Agent Benchmarks Measure Capability? Protocol Validity in the Age of Agentic AI. arXiv:2607.22368, 2026. URL https://arxiv.org/abs/2607.22368.

Yijia Shao, Tianshi Li, Weiyan Shi, Yanchen Liu, and Diyi Yang. PrivacyLens: Evaluating Privacy Norm Awareness of Language Models in Action. arXiv:2409.00138. NeurIPS 2024, 2024. URL https://arxiv.org/abs/2409.00138.

Mrinank Sharma, Meg Tong, Jesse Mu, Jerry Wei, Jorrit Kruthoff, Scott Goodfriend, Euan Ong, Alwin Peng, Raj Agarwal, Cem Anil, Amanda Askell, Nathan Bailey, Joe Benton, Emma Bluemke, Samuel R. Bowman, Eric Christiansen, Hoagy Cunningham, Andy Dau, Anjali Gopal, Rob Gilson, Logan Graham, Logan Howard, Nimit Kalra, Taesung Lee, Kevin Lin, Peter Lofgren, Francesco Mosconi, Clare O'Hara, Catherine Olsson, Linda Petrini, Samir Rajani, Nikhil Saxena, Alex Silverstein, Tanya Singh, Theodore Sumers, Leonard Tang, Kevin K. Troy, Constantin Weisser, Ruiqi Zhong, Giulio Zhou, Jan Leike, Jared Kaplan, and Ethan Perez. Constitutional Classifiers: Defending against Universal Jailbreaks across Thousands of Hours of Red Teaming. arXiv:2501.18837, 2025. URL https://arxiv.org/abs/2501.18837.

Reshabh K Sharma. ContextCov: Deriving and Enforcing Executable Constraints from Agent Instruction Files. arXiv:2603.00822, 2026. URL https://arxiv.org/abs/2603.00822.

Animesh Shaw. Silent Failures in Agentic Security Evaluation: A Validated Harness for Tool-Call Mediation Under Indirect Prompt Injection. arXiv:2609.32691, 2026. URL https://arxiv.org/abs/2609.32691.

Sarath Shekkizhar, Prathyusha Jwalapuram, Zixuan Ke, Vaidehi Patil, Jiayu Wang, Srijan Bansal, Trong Huy Nguyen, Dingfeng Jiang, Dongqi Liu, Ruochen Zhao, Wei Lu, Giuseppe Carenini, Mohit Bansal, Silvio Savarese, and Shafiq Joty. Why Do Agent Teams Fail Across Boundaries? A Failure Taxonomy for Agent-to-Agent Systems with Private States. arXiv:2610.19342, 2026. URL https://arxiv.org/abs/2610.19342.

XinPeng Shen, Lan Zhang, Yixiao Huang, Haoran Cheng, Jiewei Lai, Leilei Chen, and Haoxiang Deng. A GHOST in Long-Horizon Agents: Governance Hazard from Overlooked Safety Constraints across Turns. arXiv:2610.02664, 2026. URL https://arxiv.org/abs/2610.02664.

Toby Shevlane, Sebastian Farquhar, Ben Garfinkel, Mary Phuong, Jess Whittlestone, Jade Leung, Daniel Kokotajlo, Nahema Marchal, Markus Anderljung, Noam Kolt, Lewis Ho, Divya Siddarth, Shahar Avin, Will Hawkins, Been Kim, Iason Gabriel, Vijay Bolina, Jack Clark, Yoshua Bengio, Paul Christiano, and Allan Dafoe. Model evaluation for extreme risks. arXiv:2305.15324, 2023. URL https://arxiv.org/abs/2305.15324.

Jiawen Shi, Zenghui Yuan, Guiyao Tie, Pan Zhou, Neil Zhenqiang Gong, and Lichao Sun. Prompt Injection Attack to Tool Selection in LLM Agents. arXiv:2504.19793, 2025a. URL https://arxiv.org/abs/2504.19793.

Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. Progent: Securing AI Agents with Privilege Control. arXiv:2504.11703, 2025b. URL https://arxiv.org/abs/2504.11703.

Tianneng Shi, Robin Rheem, Dongwei Jiang, Mona Wang, Francisco De La Riega, Zhun Wang, Jingzhi Jiang, Alexander Cheung, Sean Tai, Jonah Cha, Jianhong Tu, Gabriel Han, Chenguang Wang, Jingxuan He, Wenbo Guo, and Dawn Song. CyberGym-E2E: Scalable Real-World Benchmark for AI Agents' End-to-End Cybersecurity Capabilities. arXiv:2606.04460. ICML 2026, 2026a. URL https://arxiv.org/abs/2606.04460.

Zijing Shi, Meng Fang, and Ling Chen. Benchmarking Web Agent Safety under E-commerce Deceptive Interfaces. arXiv:2606.13686. ACL 2026, 2026b. URL https://arxiv.org/abs/2606.13686.

Minsun Shim, Ramisha Raida Karim, Ruthwik Jakkula, Kaiwen Zhou, Xin Liu, Xin Eric Wang, and Zhou Li. Confuse the Model, Control the Flow: Understanding and Mitigating Privacy Leakage from LLM Agents with Information Flow Control. arXiv:2609.14003, 2026. URL https://arxiv.org/abs/2609.14003.

Aarushi Singh. Guardrails as Scapegoats: Auditing Unfaithful Safety Refusals in Tool-Augmented LLM Agents. arXiv:2607.19449. KDD 2026 workshop, 2026. URL https://arxiv.org/abs/2607.19449.

Ankur Singh, Jinqiu Yang, and Tse-Hsun Chen. IssueTrojanBench: Benchmarking AI Coding Agents Against Malicious Issue Requests. arXiv:2607.20759, 2026. URL https://arxiv.org/abs/2607.20759.

Nolan Smyth, Yorguin-Jose Mantilla-Ramos, Pascal Jr Tikeng Notsawo, Saskia Helbling, Alberto Tosato, Mohamed Amine Merzouk, Nouha Dziri, Gauthier Gidel, and Tommaso Tosato. Quantifying Overclaiming Propensity in Frontier LLM Agents. arXiv:2609.20812, 2026. URL https://arxiv.org/abs/2609.20812.

Hao Song, Yiming Shen, Wenxuan Luo, Leixin Guo, Ting Chen, Jiashui Wang, Beibei Li, Xiaosong Zhang, and Jiachi Chen. Beyond the Protocol: Unveiling Attack Vectors in the Model Context Protocol (MCP) Ecosystem. arXiv:2506.02040, 2025. URL https://arxiv.org/abs/2506.02040.

Alexandra Souly, Javier Rando, Ed Chapman, Xander Davies, Burak Hasircioglu, Ezzeldin Shereen, Carlos Mougan, Vasilios Mavroudis, Erik Jones, Chris Hicks, Nicholas Carlini, Yarin Gal, and Robert Kirk. Poisoning Attacks on LLMs Require a Near-constant Number of Poison Samples. arXiv:2510.07192, 2025. URL https://arxiv.org/abs/2510.07192.

Alexandra Souly, Robert Kirk, Jacob Merizian, Abby D'Cruz, and Xander Davies. UK AISI Alignment Evaluation Case-Study. arXiv:2604.00788, 2026. URL https://arxiv.org/abs/2604.00788.

Murugiah Souppaya and Karen Scarfone. Guide to Data-Centric System Threat Modeling (NIST Special Publication 800-154, Draft). NIST SP 800-154 (Draft), 2016. URL https://csrc.nist.gov/pubs/sp/800/154/ipd.

Karen Spinner. Are AI agents less likely to escape if you give them permission to fail? wonderingaboutai.substack.com, 2026. URL https://wonderingaboutai.substack.com/p/are-ai-agents-less-likely-to-escape.

Thibault Spirlet. The personal AI agent horror stories are rolling in. news, 2026. URL https://www.businessinsider.com/ai-agents-gone-wrong-horror-stories-instinct-muse-privacy-security-2026-9.

Ankit Srivastava and Debjyoti Paul. Agentic Commerce Bench: Measuring Fraud Detection for Agents That Spend Money. arXiv:2609.35886, 2026. URL https://arxiv.org/abs/2609.35886.

Robin Staab, Tom Wollschläger, Vahab Mirrokni, and Reza Shokri. PrivacyFlow: Privacy in AI Agents through Information Flow Analysis and Consequence-Aware Risk Management. arXiv:2610.40714, 2026. URL https://arxiv.org/abs/2610.40714.

Chung-En Sun, Linbo Liu, and Tsui-Wei Weng. The Cold-Start Safety Gap in LLM Agents. arXiv:2606.07867. EMNLP 2026, 2026a. URL https://arxiv.org/abs/2606.07867.

Yiheng Sun, Huifei Wang, Yancheng Zhu, Zhenyu Li, Zebin Zhao, and Yifan Yuan. When "Must" Becomes "Maybe": Erosion of Workflow Constraints in LLM Agent Handoffs. arXiv:2608.24569, 2026b. URL https://arxiv.org/abs/2608.24569.

Balachandra Devarangadi Sunil, Isheeta Sinha, Piyush Maheshwari, Shantanu Todmal, Shreyan Mallik, and Shuchi Mishra. Memory Poisoning Attack and Defense on Memory Based LLM-Agents. arXiv:2601.05504, 2026. URL https://arxiv.org/abs/2601.05504.

Surada Suwansathit, Yuxuan Zhang, and Guofei Gu. A Security Analysis of the OpenClaw AI Agent Framework. arXiv:2603.27517, 2026. URL https://arxiv.org/abs/2603.27517.

Om Tailor. Audit the Whisper: Detecting Steganographic Collusion in Multi-Agent LLMs. arXiv:2510.04303, 2025. URL https://arxiv.org/abs/2510.04303.

Manveer Singh Tamber, Abhay Puri, Marc-Etienne Brunet, Perouz Taslakian, Jimmy Lin, and Spandana Gella. PrivacyAlign: Contextual Privacy Alignment for LLM Agents. arXiv:2606.21710, 2026. URL https://arxiv.org/abs/2606.21710.

Jingyu Tang, Chaoran Chen, Jiawen Li, Zhiping Zhang, Bingcan Guo, Ibrahim Khalilov, Simret Araya Gebreegziabher, Bingsheng Yao, Dakuo Wang, Yanfang Ye, Tianshi Li, Ziang Xiao, Yaxing Yao, and Toby Jia-Jun Li. Dark Patterns Meet GUI Agents: LLM Agent Susceptibility to Manipulative Interfaces and the Role of Human Oversight. arXiv:2509.10723, 2025a. URL https://arxiv.org/abs/2509.10723.

Yuzhi Tang, Tianxiao Li, Elizabeth Li, Chris J. Maddison, Honghua Dong, and Yangjun Ruan. LM Agents May Fail to Act on Their Own Risk Knowledge. arXiv:2508.13465, 2025b. URL https://arxiv.org/abs/2508.13465.

Zhenwei Tang, Di Bai, Jintao Liu, Xidong Wu, Qianfeng Wen, and Xinwu Cheng. Toolflation: Seller Agents Distort Tool Markets Through Spontaneous Generative Engine Optimization. arXiv:2610.28454, 2026. URL https://arxiv.org/abs/2610.28454.

Riya Tapwal, Abhishek Kumar, and Carsten Maple. PRISM: Generation-Time Detection and Mitigation of Secret Leakage in Multi-Agent LLM Pipelines. arXiv:2605.10614, 2026. URL https://arxiv.org/abs/2605.10614.

Mohammad Taufeeque, Stefan Heimersheim, Adam Gleave, and Chris Cundy. The Obfuscation Atlas: Mapping Where Honesty Emerges in RLVR with Deception Probes. arXiv:2602.15515. ICML 2026, 2026. URL https://arxiv.org/abs/2602.15515.

Jordan Taylor, Sid Black, Dillon Bowen, Thomas Read, Satvik Golechha, Alex Zelenka-Martin, Oliver Makins, Connor Kissane, Kola Ayonrinde, Jacob Merizian, Samuel Marks, Chris Cundy, and Joseph Bloom. Auditing Games for Sandbagging. arXiv:2512.07810, 2025. URL https://arxiv.org/abs/2512.07810.

Mikhail Terekhov, Alexander Panfilov, Daniil Dzenhaliou, Caglar Gulcehre, Maksym Andriushchenko, Ameya Prabhu, and Jonas Geiping. Adaptive Attacks on Trusted Monitors Subvert AI Control Protocols. arXiv:2510.09462, 2025. URL https://arxiv.org/abs/2510.09462.

Aaron Xuxiang Tian, Ruofan Zhang, Janet Tang, Ji Wang, Tianyu Shi, and Jiaxin Wen. Measuring Harmfulness of Computer-Using Agents. arXiv:2508.00935, 2025. URL https://arxiv.org/abs/2508.00935.

Yu Tian, Xiao Yang, Jingyuan Zhang, Yinpeng Dong, and Hang Su. Evil Geniuses: Delving into the Safety of LLM-based Agents. arXiv:2311.11855, 2023. URL https://arxiv.org/abs/2311.11855.

Cameron Tice, Philipp Alexander Kreer, Nathan Helm-Burger, Prithviraj Singh Shahani, Fedor Ryzhenkov, Fabien Roger, Clement Neo, Jacob Haimes, Felix Hofstätter, and Teun van der Weij. Noise Injection Reveals Hidden Capabilities of Sandbagging Language Models. arXiv:2412.01784. NeurIPS 2025 workshop, 2024. URL https://arxiv.org/abs/2412.01784.

Jeremy Tien, Abishek Anand, Yu-Rou Tuan, Yuchen Shen, J. Zico Kolter, and Aran Nayebi. ROGUE: Evaluating Corrigibility Failures in Frontier Computer-Use Agents. arXiv:2606.00341, 2026. URL https://arxiv.org/abs/2606.00341.

Filippo Tonini, Federico Torrielli, Anton Danholt Lautrup, Peter Schneider-Kamp, Mustafa Mert Çelikok, and Lukas Galke Poech. The Arbiter Agent: Continually Monitoring Multi-Agent Conversations to Detect Emergent Misalignment. arXiv:2606.10747, 2026. URL https://arxiv.org/abs/2606.10747.

George Torres, Sharad Shrestha, and Satyajayant Misra. When Agents Remember Too Much: Memory Poisoning Attacks on Large Language Model Agents. arXiv:2607.06595, 2026. URL https://arxiv.org/abs/2607.06595.

Tyler Tracy, Ram Potham, Nick Kuhn, Myles Heller, Anshul Khandelwal, Cody Rushing, Henri Lemoine, Miguel Brandao, Tomas Turlik, Adam Hanson, Josh Hills, Amy Ngo, Ram Rachum, Nik Mitchell, Falko Galperin, Oscar Sykes, Pip Arnott, Samuel Prieto Lima, Carlos Giudice, Matt Goldwater, Daniel Popp, Drew de Wet, Ruben Castaing, Qi Guo, Douw Marx, Benjamin Shaffrey, Justin Shenk, Martin Milbradt, Hannah Meagher, Shaheen Ahmed-Chowdhury, Daniel O'Connell, Chris Canal, Buck Shlegeris, and Aryan Bhatt. LinuxArena: A Control Setting for AI Agents in Live Production Software Environments. arXiv:2604.15384, 2026. URL https://arxiv.org/abs/2604.15384.

Ada Defne Tur, Nicholas Meade, Xing Han Lù, Alejandra Zambrano, Arkil Patel, Esin Durmus, Spandana Gella, Karolina Stańczak, and Siva Reddy. SafeArena: Evaluating the Safety of Autonomous Web Agents. arXiv:2503.04957, 2025. URL https://arxiv.org/abs/2503.04957.

Rustem Turtayev, Artem Petrov, Dmitrii Volkov, and Denis Volk. Hacking CTFs with Plain Agents. arXiv:2412.02776, 2024. URL https://arxiv.org/abs/2412.02776.

Chijioke Ugwuanyi. From 8B to Frontier: How System Prompts Control Whether AI Agents Blackmail, Leak, and Kill. lesswrong.com, 2026. URL https://www.lesswrong.com/posts/GviHfFBB9tsefmDkG/from-8b-to-frontier-how-system-prompts-control-whether-ai.

Teun van der Weij, Felix Hofstätter, Ollie Jaffe, Samuel F. Brown, and Francis Rhys Ward. AI Sandbagging: Language Models can Strategically Underperform on Evaluations. arXiv:2406.07358, 2024. URL https://arxiv.org/abs/2406.07358.

Kavana Venkatesh, Jafar Isbarov, Saad Amin, Murat Kantarcioglu, and Jiaming Cui. CASPIAN: Online Detection and Attribution of Cascade Attacks in LLM Multi-Agent Systems via Cross-Channel Causal Monitoring. arXiv:2605.19240, 2026. URL https://arxiv.org/abs/2605.19240.

Vibe Graveyard. Gemini 3.5 reportedly deleted 28,745 lines, broke production, and invented its own recovery. aggregator, 2026a. URL https://vibegraveyard.ai/story/gemini-35-production-code-purge-fake-recovery/.

Vibe Graveyard. PocketOS founder says Claude Opus 4.6 agent erased its production database and backups in 9 seconds. aggregator, 2026b. URL https://vibegraveyard.ai/story/pocketos-claude-opus-agent-database-wipe/.

Brandon Vigliarolo. Google Antigravity vibe-codes user's entire drive out of existence. news, 2025. URL https://www.theregister.com/software/2025/12/01/googles-vibe-coding-platform-deletes-entire-drive/1817705.

Sanidhya Vijayvargiya, Aditya Bharat Soni, Xuhui Zhou, Zora Zhiruo Wang, Nouha Dziri, Graham Neubig, and Maarten Sap. OpenAgentSafety: A Comprehensive Framework for Evaluating Real-World AI Agent Safety. arXiv:2507.06134. ICLR 2026, 2025. URL https://arxiv.org/abs/2507.06134.

Corban Villa, Alp Eren Ozdarendeli, Sijun Tan, and Raluca Ada Popa. Prismata: Confining Cross-Site Prompt Injection in Web Agents. arXiv:2607.08147, 2026. URL https://arxiv.org/abs/2607.08147.

Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. arXiv:2404.13208, 2024. URL https://arxiv.org/abs/2404.13208.

Han Wang, Yifan Sun, Brian Ko, Mann Talati, Jiawen Gong, Zimeng Li, Naicheng Yu, Xucheng Yu, Wei Shen, Vedant Jolly, and Huan Zhang. MonitorBench: A Comprehensive Benchmark for Chain-of-Thought Monitorability in Large Language Models. arXiv:2603.28590. COLM 2026, 2026a. URL https://arxiv.org/abs/2603.28590.

Hao Wang, Hanchen Li, Qiuyang Mang, Alvin Cheung, Koushik Sen, and Dawn Song. Do Androids Dream of Breaking the Game? Systematically Auditing AI Agent Benchmarks with BenchJack. arXiv:2605.12673, 2026b. URL https://arxiv.org/abs/2605.12673.

Haochuan Kevin Wang and Zechen Zhang. Kill-Chain Canaries: Stage-Level Tracking of Prompt Injection Across Attack Surfaces and Five Production LLMs. arXiv:2603.28013, 2026. URL https://arxiv.org/abs/2603.28013.

Haowei Wang, Junjie Wang, Xiaojun Jia, Rupeng Zhang, Mingyang Li, Zhe Liu, Yang Liu, and Qing Wang. AdInject: Real-World Black-Box Attacks on Web Agents via Advertising Delivery. arXiv:2505.21499, 2025a. URL https://arxiv.org/abs/2505.21499.

Haoyu Wang, Wei Zhao, Yedi Zhang, Christopher M. Poskitt, and Jun Sun. Representation Transitions Reveal Emerging Safety Risks in Multi-Turn LLM Agents. arXiv:2610.00400, 2026c. URL https://arxiv.org/abs/2610.00400.

Jiacheng Wang, Jinchang Hou, Fabian Wang, Ping Jian, Chenfu Bao, and Zhonghou Lv. HINTBench: Horizon-agent Intrinsic Non-attack Trajectory Benchmark. arXiv:2604.13954, 2026d. URL https://arxiv.org/abs/2604.13954.

Keegan Wang and Anantika Mannby. Topology-Conditioned Backdoors: Language Models That Insert Vulnerabilities When They Infer They Are in a Multi-Agent System. arXiv:2610.05793. NeurIPS 2026 workshop, 2026. URL https://arxiv.org/abs/2610.05793.

Prince Zizhuang Wang and Shuli Jiang. AgentSocialBench: Evaluating Privacy Risks in Human-Centered Agentic Social Networks. arXiv:2604.01487, 2026. URL https://arxiv.org/abs/2604.01487.

Prince Zizhuang Wang, Aojie Yuan, Haiyue Zhang, Xiyang Hu, Yue Zhao, and Shuli Jiang. WeClawArena: An Auditable Sandbox and Benchmark for Cross-User Agents Collaboration and Security in Human-Centered Agent Networks. arXiv:2608.03499, 2026e. URL https://arxiv.org/abs/2608.03499.

Shilong Wang, Guibin Zhang, Miao Yu, Guancheng Wan, Fanci Meng, Chongye Guo, Kun Wang, and Yang Wang. G-Safeguard: A Topology-Guided Security Lens and Treatment on LLM-based Multi-agent Systems. arXiv:2502.11127, 2025b. URL https://arxiv.org/abs/2502.11127.

Shouju Wang, Fenglin Yu, Xirui Liu, Xiaoting Qin, Jue Zhang, Qingwei Lin, Dongmei Zhang, and Saravan Rajmohan. Privacy in Action: Towards Realistic Privacy Mitigation and Evaluation for LLM-Powered Agents. arXiv:2509.17488. EMNLP 2025, 2025c. URL https://arxiv.org/abs/2509.17488.

Yang Wang. Approval Laundering: Systematizing Approval–Execution Binding Failures in AI Coding-Agent Harnesses. arXiv:2609.38983, 2026. URL https://arxiv.org/abs/2609.38983.

Yian Wang, Agam Goyal, Eshwar Chandrasekharan, and Hari Sundaram. Facts Without Rules: Boundary Metadata Collapse in Multi-Agent LLM Handoffs. arXiv:2608.29028, 2026f. URL https://arxiv.org/abs/2608.29028.

Yifei Wang, Dizhan Xue, Shengjie Zhang, and Shengsheng Qian. BadAgent: Inserting and Activating Backdoor Attacks in LLM Agents. arXiv:2406.03007. ACL 2024, 2024. URL https://arxiv.org/abs/2406.03007.

Yizhu Wang, Sizhe Chen, Raghad Alkhudair, Basel Alomair, and David Wagner. Defending Against Prompt Injection with DataFilter. arXiv:2510.19207, 2025d. URL https://arxiv.org/abs/2510.19207.

Youting Wang, Xiao Han, Dingyan Shang, Yuan Tang, and Bowen Liu. Safety, or Just Capability? A Validity Audit of Agent-Safety Benchmarks. arXiv:2607.28685, 2026g. URL https://arxiv.org/abs/2607.28685.

Zhiqiang Wang, Yichao Gao, Yanting Wang, Suyuan Liu, Haifeng Sun, Haoran Cheng, Guanquan Shi, Haohua Du, and Xiangyang Li. MCPTox: A Benchmark for Tool Poisoning Attack on Real-World MCP Servers. arXiv:2508.14925, 2025e. URL https://arxiv.org/abs/2508.14925.

Zhun Wang, Nico Schiller, Hongwei Li, Srijiith Sesha Narayana, Milad Nasr, Nicholas Carlini, Xiangyu Qi, Eric Wallace, Elie Bursztein, Luca Invernizzi, Kurt Thomas, Yan Shoshitaishvili, Wenbo Guo, Jingxuan He, Thorsten Holz, and Dawn Song. ExploitGym: Can AI Agents Turn Security Vulnerabilities into Real Attacks? arXiv:2605.11086, 2026h. URL https://arxiv.org/abs/2605.11086.

Ziyan Wang, Shuqing Shi, James Oldfield, Samuele Marro, Jialin Yu, Philip Torr, Yali Du, and Adel Bibi. BazaarBench: Delegation Safety in Decentralized C2C Marketplaces Run by LLM Agents. arXiv:2610.06748, 2026i. URL https://arxiv.org/abs/2610.06748.

Jake Ward, Shawn Hu, Aria Wong, Nathan Hu, Bryce Woodworth, Alexander Matt Turner, and Alex Cloud. Gradient Routing Localizes and Removes Unintended Behaviors in RL. arXiv:2610.17062, 2026. URL https://arxiv.org/abs/2610.17062.

Moritz Weckbecker, Jonas Müller, Ben Hagag, and Michael Mulet. Thought Virus: Viral Misalignment via Subliminal Prompting in Multi-Agent Systems. arXiv:2603.00131, 2026. URL https://arxiv.org/abs/2603.00131.

Qianshan Wei, Tengchao Yang, Yaochen Wang, Xinfeng Li, Lijun Li, Zhenfei Yin, Yi Zhan, Thorsten Holz, Zhiqiang Lin, and XiaoFeng Wang. A-MemGuard: A Proactive Defense Framework for LLM-Based Agent Memory. arXiv:2510.02373, 2025. URL https://arxiv.org/abs/2510.02373.

Jonas Wiedermann-Möller, Leonard Dung, and Maksym Andriushchenko. Instrumental Choices: Measuring the Propensity of LLM Agents to Pursue Instrumental Behaviors. arXiv:2605.06490, 2026. URL https://arxiv.org/abs/2605.06490.

Simon Willison. A quote from Claude. simonwillison.net, 2025. URL https://simonwillison.net/2025/Dec/9/claude/.

Chen Henry Wu, Rishi Shah, Jing Yu Koh, Ruslan Salakhutdinov, Daniel Fried, and Aditi Raghunathan. Dissecting Adversarial Robustness of Multimodal LM Agents. arXiv:2406.12814. ICLR 2025 workshop, 2024a. URL https://arxiv.org/abs/2406.12814.

Chengcan Wu, Zhixin Zhang, Mingqian Xu, Zeming Wei, and Meng Sun. Securing Multi-Agent Systems Against Corruptions via Node Contribution Backpropagation. arXiv:2510.19420. ICML 2026, 2025. URL https://arxiv.org/abs/2510.19420.

Chenhao Wu, Haoxuan Jia, Yang Liu, Yingguang Yang, Yuhan Lin, Chongyang Zhang, Hao Zheng, Yulin Huang, Jianshen Zhang, Yongzhi Qi, Shang Luo, Kefu Xu, Jifeng Zhu, and Bin Chong. Safety Does Not Compose: Non-Decaying Loop State for Autonomous LLM Agents. arXiv:2608.27141, 2026a. URL https://arxiv.org/abs/2608.27141.

Weiheng Wu, Junkai Chen, Zhensu Sun, Jieke Shi, Yunbo Lyu, and David Lo. "Shall We Approve?" Consequence-Aware Approval for Autonomous Coding Agents. arXiv:2610.40753, 2026b. URL https://arxiv.org/abs/2610.40753.

Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. IsolateGPT: An Execution Isolation Architecture for LLM-Based Agentic Systems. arXiv:2403.04960, 2024b. URL https://arxiv.org/abs/2403.04960.

wunderwuzzi. Breaking Claude Code Opus 5 Auto Mode. embracethered.com, 2026. URL https://embracethered.com/blog/posts/2026/breaking-claude-code-opus-5-and-automode/.

Qiuchi Xiang, Haoxuan Qu, Hossein Rahmani, and Jun Liu. Is Monitoring Enough? Strategic Agent Selection For Stealthy Attack in Multi-Agent Discussions. arXiv:2603.21194, 2026. URL https://arxiv.org/abs/2603.21194.

Zhen Xiang, Linzhi Zheng, Yanjie Li, Junyuan Hong, Qinbin Li, Han Xie, Jiawei Zhang, Zidi Xiong, Chulin Xie, Carl Yang, Dawn Song, and Bo Li. GuardAgent: Safeguard LLM Agents by a Guard Agent via Knowledge-Enabled Reasoning. arXiv:2406.09187. ICML 2025, 2024. URL https://arxiv.org/abs/2406.09187.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. arXiv:2404.07972, 2024. URL https://arxiv.org/abs/2404.07972.

Yizhe Xie, Congcong Zhu, Xinyue Zhang, Tianqing Zhu, Dayong Ye, Minfeng Qi, Huajie Chen, and Wanlei Zhou. From Spark to Fire: Modeling and Mitigating Error Cascades in LLM-Based Multi-Agent Collaboration. arXiv:2603.04474, 2026a. URL https://arxiv.org/abs/2603.04474.

Yuejin Xie, Yu Li, Dadi Guo, Qingyu Liu, Yuqian Fu, Yanwei Fu, Yujiu Yang, Xia Hu, and Dongrui Liu. ClashBench: Conflicts Leading Agents to Seize and Harm. arXiv:2609.19892, 2026b. URL https://arxiv.org/abs/2609.19892.

Xihan Xiong, Zelin Li, Wei Wei, Qin Wang, William Knottenbelt, and Zhipeng Wang. Can Trustless Agents Be Trusted? An Empirical Study of the ERC-8004 Decentralized AI Agent Ecosystem. arXiv:2606.26028, 2026. URL https://arxiv.org/abs/2606.26028.

Haowen Xu, Xue Tan, Lei Ma, Zhihao Zhang, Chao Wang, Qingze Wang, Ping Chen, Jun Dai, and Xiaoyan Sun. When Agents Go Rogue: Activation-Based Detection of Malicious Behaviors in Multi-Agent Systems. arXiv:2607.06807. ICML 2026, 2026a. URL https://arxiv.org/abs/2607.06807.

Yan Xu, Dandan Song, Luan Zhang, Jun Yang, Ke Duan, Yuhang Tian, Yading Li, Xudong Li, Xing Pei, Guangyuan Feng, and Chenhao Li. MCAIR: Monitoring and Correcting Agent Unfaithfulness via Internal Representations. arXiv:2610.19679, 2026b. URL https://arxiv.org/abs/2610.19679.

Yiming Xu, Hongyue Yu, Beihua Yang, Zihan Chen, Yixin Liu, Zhen Peng, Bin Shi, Bo Dong, Chao Shen, Irwin King, and Qinghua Zheng. DecepEval: A Benchmark for Evaluating Deception in LLM Agents. arXiv:2610.07967, 2026c. URL https://arxiv.org/abs/2610.07967.

Faouzi El Yagoubi, Godwin Badu-Marfo, and Ranwa Al Mallah. The Interlocutor Effect: Why LLMs Leak More Personal Data to Agents Than Humans. arXiv:2606.09844, 2026. URL https://arxiv.org/abs/2606.09844.

Ting Yan. Do User-Authored Permission Policies Improve Protection Against AI Agent Overreach? arXiv:2608.27443, 2026. URL https://arxiv.org/abs/2608.27443.

Mingdai Yang, Shicheng Fan, Kejing Yu, Duohao Wang, Li Sun, Hao Peng, Philip S. Yu, and Zhiwei Liu. Paying for Honesty Without Knowing the Truth: Reputation-Penalty Design for LLM Marketplace Agents. arXiv:2607.28330, 2026a. URL https://arxiv.org/abs/2607.28330.

Rui Yang, Junjie Xu, Zhengyu Liu, Neil Fendley, Yang Hong, Ziyang Li, and Yinzhi Cao. SoK: When Safe Agents Fail Together: The Security of Multi Agent LLM Systems. arXiv:2609.00595, 2026b. URL https://arxiv.org/abs/2609.00595.

Wenkai Yang, Xiaohan Bi, Yankai Lin, Sishuo Chen, Jie Zhou, and Xu Sun. Watch Out for Your Agents! Investigating Backdoor Threats to LLM-Based Agents. arXiv:2402.11208. NeurIPS 2024, 2024. URL https://arxiv.org/abs/2402.11208.

Xianglin Yang, Yufei He, Shuo Ji, Bryan Hooi, and Jin Song Dong. Zombie Agents: Persistent Control of Self-Evolving LLM Agents via Self-Reinforcing Injections. arXiv:2602.15654. ICLR 2026 workshop, 2026c. URL https://arxiv.org/abs/2602.15654.

Xiao Yang, Jinchang Hou, Jiawei Chen, Zhonghou Lv, Junyuan Shang, and Chenfu Bao. Trojan Horse: Automated Repository Poisoning against Coding Agents. arXiv:2610.23109, 2026d. URL https://arxiv.org/abs/2610.23109.

Xiaoxue Yang, Bozhidar Stevanoski, Matthieu Meeus, and Yves-Alexandre de Montjoye. Auditing Prompt Injection Defenses Before They Reach Users: What Strong Adversaries Reveal. arXiv:2505.15738. COLM 2026 workshop, 2025a. URL https://arxiv.org/abs/2505.15738.

Yixuan Yang, Cuifeng Gao, Daoyuan Wu, Yufan Chen, Yingjiu Li, and Shuai Wang. MCPSecBench: A Systematic Security Benchmark and Playground for Testing Model Context Protocols. arXiv:2508.13220, 2025b. URL https://arxiv.org/abs/2508.13220.

Ziang Ye, Yang Zhang, Wentao Shi, Xiaoyu You, Fuli Feng, and Tat-Seng Chua. VisualTrap: A Stealthy Backdoor Attack on GUI Agents via Visual Grounding Manipulation. arXiv:2507.06899. COLM 2025, 2025. URL https://arxiv.org/abs/2507.06899.

Jingwei Yi, Yueqi Xie, Bin Zhu, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models. arXiv:2312.14197. KDD 2025, 2023. URL https://arxiv.org/abs/2312.14197.

Chenlong Yin, Runpeng Geng, Yanting Wang, and Jinyuan Jia. PISmith: Reinforcement Learning-based Red Teaming for Prompt Injection Defenses. arXiv:2603.13026. COLM 2026, 2026. URL https://arxiv.org/abs/2603.13026.

Zonghao Ying, Xiangfan Wu, Bo Yang, Huiyu Wu, Xing Zheng, Huangsheng Cheng, Xiaorong Shi, and Jing Guo. The Missing Boundary: How Autonomous Agents Lose Control. arXiv:2609.11024, 2026. URL https://arxiv.org/abs/2609.11024.

Miao Yu, Shilong Wang, Guibin Zhang, Junyuan Mao, Chenlong Yin, Qijiong Liu, Qingsong Wen, Kun Wang, and Yang Wang. NetSafe: Exploring the Topological Safety of Multi-agent Networks. arXiv:2410.15686, 2024. URL https://arxiv.org/abs/2410.15686.

Chen Yueh-Han, Nitish Joshi, Yulin Chen, Maksym Andriushchenko, Rico Angell, and He He. Monitoring Decomposition Attacks in LLMs with Lightweight Sequential Monitors. arXiv:2506.10949, 2025. URL https://arxiv.org/abs/2506.10949.

Steffen Zahn. Claude, Gemini, Comet: five AI browsers hijacked by a single email. notebookcheck.net, 2026. URL https://www.notebookcheck.net/Claude-Gemini-Comet-five-AI-browsers-hijacked-by-a-single-email.1362568.0.html.

Saber Zerhoudi, Kanishka Ghosh Dastidar, Felix Klement, Artur Romazanov, Andreas Einwiller, Dang H. Dang, Michael Dinzinger, Michael Granitzer, Annette Hautli-Janisz, Stefan Katzenbeisser, Florian Lemmerich, and Jelena Mitrovic. Form Without Function: Agent Social Behavior in the Moltbook Network. arXiv:2604.13052, 2026a. URL https://arxiv.org/abs/2604.13052.

Saber Zerhoudi, Jelena Mitrovic, and Michael Granitzer. The Compaction Cliff in Long-Running AI Agent Memory. arXiv:2608.22752, 2026b. URL https://arxiv.org/abs/2608.22752.

Mingming Zha and Xiaofeng Wang. Autonomous LLM Agent Worms: Cross-Platform Propagation, Automated Discovery and Temporal Re-Entry Defense. arXiv:2605.02812, 2026. URL https://arxiv.org/abs/2605.02812.

Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. arXiv:2403.02691. ACL 2024, 2024. URL https://arxiv.org/abs/2403.02691.

Qiusi Zhan, Richard Fang, Henil Shalin Panchal, and Daniel Kang. Adaptive Attacks Break Defenses Against Indirect Prompt Injection Attacks on LLM Agents. arXiv:2503.00061. NAACL 2025, 2025. URL https://arxiv.org/abs/2503.00061.

Qiusi Zhan, Nian Lyu, Stephanie Ding, Arnav Mehta, Chris Canal, Connor Tabarrok, Xander Davies, and Daniel Kang. Understanding and Enhancing Backdoor Persistency in LLM Agent Post-Training. arXiv:2610.07510. EMNLP 2026, 2026a. URL https://arxiv.org/abs/2610.07510.

Zhonghao Zhan, Javad Forough, Yefan Zhang, Krinos Li, Xiao Ma, and Hamed Haddadi. Airlock: Bounding the Multi-Agent Blast Radius via Selective Online Containment. arXiv:2611.06727, 2026b. URL https://arxiv.org/abs/2611.06727.

Andy K. Zhang, Neil Perry, Riya Dulepet, Joey Ji, Celeste Menders, Justin W. Lin, Eliot Jones, Gashon Hussein, Samantha Liu, Donovan Jasper, Pura Peetathawatchai, Ari Glenn, Vikram Sivashankar, Daniel Zamoshchin, Leo Glikbarg, Derek Askaryar, Mike Yang, Teddy Zhang, Rishi Alluri, Nathan Tran, Rinnara Sangpisit, Polycarpos Yiorkadjis, Kenny Osele, Gautham Raghupathi, Dan Boneh, Daniel E. Ho, and Percy Liang. Cybench: A Framework for Evaluating Cybersecurity Capabilities and Risks of Language Models. arXiv:2408.08926. ICLR 2025, 2024a. URL https://arxiv.org/abs/2408.08926.

Chiyu Zhang, Lu Zhou, Xiaogang Xu, Huiqin Yang, Bendong Jiang, Xiaolei Zhang, Yiran Zhao, Ruyi Chen, Jiafei Wu, Liming Fang, and Zhe Liu. LITMUS: Benchmarking Behavioral Jailbreaks of LLM Agents in Real OS Environments. arXiv:2605.10779. NeurIPS 2026, 2026a. URL https://arxiv.org/abs/2605.10779.

Dongsen Zhang, Zekun Li, Xu Luo, Xuannan Liu, Peipei Li, and Wenjun Xu. MCP Security Bench (MSB): Benchmarking Attacks Against Model Context Protocol in LLM Agents. arXiv:2510.15994. ICLR 2026, 2025a. URL https://arxiv.org/abs/2510.15994.

Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents. arXiv:2410.02644. ICLR 2025, 2024b. URL https://arxiv.org/abs/2410.02644.

Jinchuan Zhang, Lu Yin, Yan Zhou, and Songlin Hu. AgentAlign: Navigating Safety Alignment in the Shift from Informative to Agentic Large Language Models. arXiv:2505.23020, 2025b. URL https://arxiv.org/abs/2505.23020.

Lejun Zhang, Sarah Lu-Liang, Xin Jiang, Muning Wen, Weinan Zhang, and Shangding Gu. WolfSociety: Understanding Collective Risk from Harmful-Agent Scaling in Financial Agent Societies. arXiv:2609.05591, 2026b. URL https://arxiv.org/abs/2609.05591.

Lingxi Zhang, Guangtao Zheng, and Hanjie Chen. When Embedding-Based Defenses Fail: Rethinking Safety in LLM-Based Multi-Agent Systems. arXiv:2605.01133, 2026c. URL https://arxiv.org/abs/2605.01133.

Taolin Zhang, Jiuheng Wan, Hanyu Wang, Tingyuan Hu, and Chengyu Wang. OverAct: Measuring and Mitigating Proactive Over-Authorization in LLM Tool-Calling Agents. arXiv:2610.01508, 2026d. URL https://arxiv.org/abs/2610.01508.

Wenqi Zhang, Yulin Shen, Changyue Jiang, Jiarun Dai, Geng Hong, and Xudong Pan. MirrorGuard: Toward Secure Computer-Use Agents via Simulation-to-Real Reasoning Correction. arXiv:2601.12822, 2026e. URL https://arxiv.org/abs/2601.12822.

Yanzhe Zhang and Diyi Yang. Searching for Privacy Risks in LLM Agents via Simulation. arXiv:2508.10880. ICLR 2026, 2025. URL https://arxiv.org/abs/2508.10880.

Yanzhe Zhang, Tao Yu, and Diyi Yang. Attacking Vision-Language Computer Agents via Pop-ups. arXiv:2411.02391. ACL 2025, 2024c. URL https://arxiv.org/abs/2411.02391.

Yihao Zhang, Zeming Wei, Xiaokun Luan, Chengcan Wu, Zhixin Zhang, Jiangrong Wu, Haolin Wu, Huanran Chen, Jun Sun, and Meng Sun. AgentWorm: Self-Propagating Attacks Across LLM Agent Ecosystems. arXiv:2603.15727, 2026f. URL https://arxiv.org/abs/2603.15727.

Yingqi Zhang. Agent libOS: A Runtime Substrate for Capability-Controlled Self-Evolving LLM Agents. arXiv:2606.03895, 2026. URL https://arxiv.org/abs/2606.03895.

Zaibin Zhang, Yongting Zhang, Lijun Li, Hongzhi Gao, Lijun Wang, Huchuan Lu, Feng Zhao, Yu Qiao, and Jing Shao. PsySafe: A Comprehensive Framework for Psychological-based Attack, Defense, and Evaluation of Multi-agent System Safety. arXiv:2401.11880. ACL 2024, 2024d. URL https://arxiv.org/abs/2401.11880.

Zhexin Zhang, Shiyao Cui, Yida Lu, Jingzhuo Zhou, Junxiao Yang, Hongning Wang, and Minlie Huang. Agent-SafetyBench: Evaluating the Safety of LLM Agents. arXiv:2412.14470, 2024e. URL https://arxiv.org/abs/2412.14470.

Zhiping Zhang, Franklin Mingzhe Li, Zihao Huang, and Tianshi Li. Act on Your Behalf, Beyond Your Authorization: Benchmarking LLM Agent Authorization Alignment. arXiv:2610.27548, 2026g. URL https://arxiv.org/abs/2610.27548.

Weibo Zhao, Jiahao Liu, Bonan Ruan, Shaofei Li, and Zhenkai Liang. When MCP Servers Attack: Taxonomy, Feasibility, and Mitigation. arXiv:2509.24272, 2025. URL https://arxiv.org/abs/2509.24272.

Weixiang Zhao, Zhihao Liu, Guancheng Li, Yanyan Zhao, Bing Qin, and Ting Liu. Predictive Safety Guardrails Do Not Always Make LLM Agents Safer. arXiv:2611.02815, 2026. URL https://arxiv.org/abs/2611.02815.

Zhijie Zheng, Yu Li, Chen Qian, Yuqian Fu, Yanwei Fu, Lu Sheng, Jing Shao, and Dongrui Liu. StepGuard: Learning Step-Level Guardrails with Scalable Supervision and Safety-Utility Balancing. arXiv:2608.24777. EMNLP 2026, 2026. URL https://arxiv.org/abs/2608.24777.

Shawn Wanxiang Zhong, Junxuan Liao, Jing Liu, Mai Zheng, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Don't Let AI Agents YOLO Your Files: Information and Control in Agent-Native Filesystems. arXiv:2604.13536, 2026a. URL https://arxiv.org/abs/2604.13536.

Ziqian Zhong, Aditi Raghunathan, and Nicholas Carlini. ImpossibleBench: Measuring LLMs' Propensity of Exploiting Test Cases. arXiv:2510.20270, 2025. URL https://arxiv.org/abs/2510.20270.

Ziqian Zhong, Ivgeni Segal, Ivan Bercovich, Shashwat Saxena, Kexun Zhang, and Aditi Raghunathan. Hardening Agent Benchmarks with Adversarial Hacker-Fixer Loops. arXiv:2606.08960, 2026b. URL https://arxiv.org/abs/2606.08960.

Jiayi Zhou, Yang Sheng, Hantao Lou, Yaodong Yang, and Jie Fu. FormalJudge: A Neuro-Symbolic Paradigm for Agentic Oversight. arXiv:2602.11136, 2026a. URL https://arxiv.org/abs/2602.11136.

Kaiwen Zhou and Udari Madhushani Sehwag. RSI Control: Can AI Models Perform Covert Sabotage in the Self-Improvement Process? arXiv:2610.27735, 2026. URL https://arxiv.org/abs/2610.27735.

Yijin Zhou, Xiaoya Lu, Dongrui Liu, Junchi Yan, and Jing Shao. INFA-Guard: Mitigating Malicious Propagation via Infection-Aware Safeguarding in LLM-Based Multi-Agent Systems. arXiv:2601.14667, 2026b. URL https://arxiv.org/abs/2601.14667.

Junru Zhu, Shiming Xie, Aime Lu Fan Chen, Xiaoqing Ding, Chunxin Tang, Ruoyu Qi, and Yulang Fei. Failure-Transparent Agents: Benchmarking Post-Failure Reporting in Tool-Using Language Models. arXiv:2609.35732, 2026a. URL https://arxiv.org/abs/2609.35732.

Zhengyang Zhu, Liming Huang, Runmin Ji, Mingxi Ye, Zihan Zhou, Hanyang Guo, Jingwen Wu, Yuhan Ye, Yuming Feng, Hong-Ning Dai, and Zibin Zheng. HarnessSecurity-Bench: Do Security Mechanisms Really Protect Coding Agent Harnesses? arXiv:2610.07639, 2026b. URL https://arxiv.org/abs/2610.07639.

Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models. arXiv:2402.07867, 2024. URL https://arxiv.org/abs/2402.07867.

David Mellafe Zuvic. Capability Gates Are Not Authorization: Confused-Deputy Failures in LLM Agent Frameworks. arXiv:2606.28679, 2026. URL https://arxiv.org/abs/2606.28679.